

SOLIDWORKS API (.NET VBA C#) — Examples (44/50): QueryInterface_Example_CPlusPlus_COM-Roll_Back_Model_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Use QueryInterface Example (C++ COM)

This example shows how to use QueryInterface.

```
LPUNKNOWN iUnk = NULL;
LPFEATURE m_Feature = NULL;
HRESULT hres = S_FALSE;

// Your code
...

// Get the underlying object
hres = m_Feature->IGetSpecificFeature( &iUnk );

LPREFPLANE m_RefPlane = NULL;
hres = iUnk->QueryInterface( IID_IRefPlane, (LPVOID*)&m_RefPlane);

// If Feature is a RefPlane
if (hres == S_OK && m_RefPlane != NULL)
{
    AfxMessageBox( _T("This feature is a reference plane") );

    // Use m_RefPlane object
    ...

    // Release m_RefPlane object
    m_RefPlane->Release();
}

// Release iUnk object
iUnk->Release();

/*
The call to IGetSpecificFeature increases the reference count on iUnk by 1. If the call to QueryInterface is successful, it also increases the reference count on iUnk by 1. The m_RefPlane and iUnk pointers are the same object; you can therefore release either one. m_RefPlane->Release() is shown in this example.
*/
```

Create Rack and Pinion Mate Example (C#)

This example shows how to create a rack and pinion mechanical mate.

```
// -----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\MechanicalMates\Rack And Pinion.sldasm.
// 2. Delete RackPinionMate1 from the Mates folder of the FeatureManager design tree.
//
// Postconditions: Inspect the Mates folder in the FeatureManager design tree.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace CreateRackPinionMate_CSharp
{
    partial class SolidWorksMacro
    {
        private ModelDoc2 Part;
        private AssemblyDoc swAssembly;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)Part;

            boolstatus = Part.Extension.SelectByRay(-0.0463518983485187, 0.061012999696942, -0.00642963015610576, -0.398796540128445);
            boolstatus = Part.Extension.SelectByRay(0.00790589165686129, -0.00993915877270979, -0.00438167267822109, -0.398796540128445);

            // Create RackPinionMateFeatureData
            RackPinionMateFeatureData MateData = (RackPinionMateFeatureData)swAssembly.CreateMateData(13);

            // Set the mate diameter type
            MateData.DiameterType = 0;

            // Set the mate diameter value
            MateData.DiameterVal = 0.0254;

            // Set the mate orientation direction
            MateData.Reverse = false;

            // Create the mate
            Feature MateFeature = (Feature)swAssembly.CreateMate(MateData);
            Part.ClearSelection2(true);
            Part.EditRebuild3();
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Rebuild All Configurations Example (VBA)

To rebuild all of the configurations in a part, assembly, or drawing document, see:

- [Forcibly Rebuild All Configurations Example \(VBA\)](#)
- [Get Active Document Dependents Example \(VBA\)](#)

Also, because each drawing view on each sheet can contain a different model and each in a different configuration, first you must find the parts or assemblies associated with the drawing. You can then rebuild them in the usual manner. Next, you must activate each drawing sheet and rebuild that sheet. The following example shows how to activate each sheet in a drawing document.

```
'-----  
' Preconditions:  
' 1. Open a drawing document that has multiple  
' sheets.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the names of each drawing sheet and  
' activates that sheet.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDraw As SldWorks.DrawingDoc  
    Dim swSheet As SldWorks.Sheet  
    Dim vSheetNameArr As Variant  
    Dim vSheetName As Variant  
    Dim bRet As Boolean  
    Dim i As Long  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
  
    Set swSheet = swDraw.GetCurrentSheet  
    Debug.Print "FileName = " & swModel.GetPathName  
    Debug.Print " Current sheet = " & swSheet.GetName  
    Debug.Print ""  
  
    vSheetNameArr = swDraw.GetSheetNames  
    For Each vSheetName In vSheetNameArr  
        Debug.Print " SheetName[" & i & "] = " & vSheetName  
        bRet = swDraw.ActivateSheet(vSheetName): Debug.Assert bRet  
        Next vSheetName  
  
End Sub
```

Rebuild All Configurations Without Activating Each Configuration Example (C#)

This example shows how to rebuild only those features that need to be rebuilt in all configurations without activating each configuration.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
// 2. Right-click membrane<1> and click Suppress in the active
// configuration, Dual Speaker.
// 3. Click Don't Save.
// 4. Click the ConfigurationManager tab and right-click Single
// Speaker Glueable and click Show Configuration to make this
// configuration the active configuration.
// 5. Open the Immediate window.
//
// Postconditions:
// 1. Gets whether each configuration needs to be rebuilt.
// 2. Rebuilds only those features that need to be rebuilt in all
// configurations without activating each configuration.
// 3. Gets whether each configuration needs to be rebuilt.
// 4. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        Configuration swConfiguration;
        string[] vConfNameArr;
        string vConfName;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Get whether each configuration needs to be rebuilt
            Debug.Print("Traverse assembly without activating other configurations...");
            TraverseConfigurations(swModel);

            //Rebuild only those features that need to be rebuilt in all configurations
            //without activating each configuration
            swModelDocExt.EditRebuildAll();

            //Get whether each configuration needs to be rebuilt
            Debug.Print("Traverse assembly without activating other configurations...");
            TraverseConfigurations(swModel);
        }

        public void TraverseConfigurations(ModelDoc2 swModel)
        {
            vConfNameArr = (string[])swModel.GetConfigurationNames();
            foreach (string config in vConfNameArr)
            {
                vConfName = (string)config;
                swConfiguration = (Configuration)swModel.GetConfigurationByName(vConfName);
                Debug.Print(" Name of the configuration: " + swConfiguration.Name);
                Debug.Print(" Does the configuration need to be rebuilt? " + swConfiguration.NeedsRebuild);
            }
            Debug.Print("");
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Rebuild All Configurations Without Activating Each Configuration Example (VBA)

This example shows how to rebuild only those features that need to be rebuilt in all configurations without activating each configuration.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
' 2. Right-click membrane<1> and click Suppress in the active
' configuration, Dual Speaker.
' 3. Click Don't Save.
' 4. Click the ConfigurationManager tab and right-click Single
' Speaker Glueable and click Show Configuration to make this
' configuration the active configuration.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Gets whether each configuration needs to be rebuilt.
' 2. Rebuilds only those features that need to be rebuilt in all
' configurations without activating each configuration.
' 3. Gets whether each configuration needs to be rebuilt.
' 4. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swConfiguration As SldWorks.Configuration
Dim vConfNameArr As Variant
Dim vConfName As Variant

Sub TraverseConfigurations(swModel As SldWorks.ModelDoc2)
    vConfNameArr = swModel.GetConfigurationNames
    For Each vConfName In vConfNameArr
        Set swConfiguration = swModel.GetConfigurationByName(vConfName)
        Debug.Print " Name of the configuration: " & swConfiguration.Name
        Debug.Print " Does the configuration need to be rebuilt? " & swConfiguration.NeedsRebuild
    Next
    Debug.Print ""
End Sub

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    'Get whether each configuration needs to be rebuilt
    Debug.Print "Traverse assembly without activating other configurations..."
    TraverseConfigurations swModel

    'Rebuild only those features that need to be rebuilt in all configurations
    'without activating each configuration
    swModelDocExt.EditRebuildAll

    'Get whether each configuration needs to be rebuilt
    Debug.Print "Traverse assembly without activating other configurations..."
    TraverseConfigurations swModel
End Sub
```

Rebuild All Configurations Without Activating Each Configuration Example (VB.NET)

This example shows how to rebuild only those features that need to be rebuilt in all configurations without activating each configuration.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
2. Right-click membrane<1> and click Suppress in the active configuration, Dual Speaker.
3. Click Don't Save.
4. Click the ConfigurationManager tab and right-click Single Speaker Glueable and click Show Configuration to make this configuration the active configuration.
5. Open the Immediate window.

Postconditions:
1. Gets whether each configuration needs to be rebuilt.
2. Rebuilds only those features that need to be rebuilt in all configurations without activating each configuration.
3. Gets whether each configuration needs to be rebuilt.
4. Examine the Immediate window.

NOTE: Because the assembly is used elsewhere, do not save changes.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swConfiguration As Configuration
    Dim vConfNameArr As Object
    Dim vConfName As Object

    Public Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        'Get whether each configuration needs to be rebuilt
        Debug.Print("Traverse assembly without activating other configurations...")
        TraverseConfigurations(swModel)

        'Rebuild only those features that need to be rebuilt in all configurations
        'without activating each configuration
        swModelDocExt.EditRebuildAll()

        'Get whether each configuration needs to be rebuilt
        Debug.Print("Traverse assembly without activating other configurations...")
        TraverseConfigurations(swModel)

    End Sub

    Sub TraverseConfigurations(ByVal swModel As ModelDoc2)
        vConfNameArr = swModel.GetConfigurationNames
        For Each vConfName In vConfNameArr
            swConfiguration = swModel.GetConfigurationByName(vConfName)
            Debug.Print("  Name of the configuration: " & swConfiguration.Name)
            Debug.Print("    Does the configuration need to be rebuilt? " & swConfiguration.NeedsRebuild)
        Next
        Debug.Print("")
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Rebuild All Features in All Configurations Example (C#)

This example shows how to rebuild all features in all configurations without activating each configuration.

```
//-----
// Preconditions:
// 1. Copy public_documents\samples\tutorial\PDMMworks to c:\temp.
// 2. Open c:\temp\membrane.sldprt.
//    a. Expand Revolve-Thin1, click Sketch1 > Edit Sketch.
//    b. Click R30 and change 30 to 60.
//    c. Click File > Save All > Rebuild and save document
//    (recommended).
//    d. Click File > Close.
// 3. Open c:\temp\speaker.sldasm.
// 4. Click Don't rebuild.
// 5. Open the Immediate window.
//
// Postconditions:
// 1. Rebuilds all features in all configurations without
//    activating each configuration.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        bool bRebuild;
        int ret;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            ret = swModelDocExt.NeedsRebuild2;
            Debug.Print("Features need to be rebuilt (1 = needs rebuild)? " + ret);

            if (ret == 1)
            {
                bRebuild = swModelDocExt.ForceRebuildAll();
                Debug.Print("    All features rebuilt in all configurations without activating each configuration? " + bRebuild);
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Rebuild All Features in All Configurations Example (VBA)

This example shows how to rebuild all features in all configurations without activating each configuration.

```
'-----  
' Preconditions:  
' 1. Copy public_documents\samples\tutorial\PDMWorks to c:\temp.  
' 2. Open c:\temp\membrane.sldprt.  
'   a. Expand Revolve-Thin1, click Sketch1 > Edit Sketch.  
'   b. Click R30 and change 30 to 60.  
'   c. Click File > Save All > Rebuild and save document  
'      (recommended).  
'   d. Click File > Close.  
' 3. Open c:\temp\speaker.sldasm.  
' 4. Click Don't rebuild.  
' 5. Open the Immediate window.  
'  
' Postconditions:  
' 1. Rebuilds all features in all configurations without  
'    activating each configuration.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim bRebuild As Boolean  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
  
    bRet = swModelDocExt.NeedsRebuild2  
    Debug.Print "Features need to be rebuilt? " & bRet  
  
    If bRet Then  
        bRebuild = swModelDocExt.ForceRebuildAll  
        Debug.Print "    All features rebuilt in all configurations without activating each configuration? " & bRebuild  
    End If  
  
End Sub
```

Rebuild All Features in All Configurations Example (VB.NET)

This example shows how to rebuild all features in all configurations without activating each configuration.

```

-----
Preconditions:
1. Copy public_documents\samples\tutorial\PDMWorks to c:\temp.
2. Open c:\temp\membrane.sldprt.
   a. Expand Revolve-Thin1, click Sketch1 > Edit Sketch.
   b. Click R30 and change 30 to 60.
   c. Click File > Save All > Rebuild and save document
      (recommended).
   d. Click File > Close.
3. Open c:\temp\speaker.sldasm.
4. Click Don't rebuild.
5. Open the Immediate window.
-----
Postconditions:
1. Rebuilds all features in all configurations without
   activating each configuration.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim bRebuild As Boolean
    Dim bRet As Boolean

    Public Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        bRet = swModelDocExt.NeedsRebuild2
        Debug.Print("Features need to be rebuilt? " & bRet)

        If bRet Then
            bRebuild = swModelDocExt.ForceRebuildAll
            Debug.Print("    All features rebuilt in all configurations without activating each configuration? " & bRebuild)
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Rebuild an Assembly Example (C#)

This example shows how to rebuild an assembly.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\wrench.sldasm.
//
// Postconditions:
// 1. Rebuilds the assembly.
// 2. Inspect the Immediate window for the result code.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace Rebuild_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 part;
        ModelDocExtension modelDocExt;

        bool ret;

        public void Main()
        {
            part = (ModelDoc2)swApp.ActiveDoc;
            modelDocExt = part.Extension;
            ret = modelDocExt.Rebuild((int)swRebuildOptions_e.swRebuildAll);

            Debug.Print("Successfully rebuilt? " + ret);
        }

        public SldWorks swApp;
    }
}
```

Rebuild an Assembly Example (VBA)

This example shows how to rebuild an assembly.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\wrench.sldasm.  
'  
' Postconditions:  
' 1. Rebuilds the assembly.  
' 2. Inspect the Immediate window for the result code.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim part As SldWorks.ModelDoc2  
Dim modelDocExt As SldWorks.ModelDocExtension  
Dim ret As Boolean  
  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set part = swApp.ActiveDoc  
    Set modelDocExt = part.Extension  
    ret = modelDocExt.Rebuild(swRebuildOptions_e.swRebuildAll)  
  
    Debug.Print "Successfully rebuilt? " & ret  
  
End Sub
```

Rebuild an Assembly Example (VB.NET)

This example shows how to rebuild an assembly.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\wrench.sldasm.  
'  
' Postconditions:  
' 1. Rebuilds the assembly.  
' 2. Inspect the Immediate window for the result code.  
'  
' NOTE: Because the model is used elsewhere,  
' do not save changes when closing it.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim part As ModelDoc2  
    Dim modelDocExt As ModelDocExtension  
    Dim ret As Boolean  
  
    Sub main()  
  
        part = swApp.ActiveDoc  
        modelDocExt = part.Extension  
        ret = modelDocExt.Rebuild(swRebuildOptions_e.swRebuildAll)  
  
        Debug.Print("Successfully rebuilt? " & ret)
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Rebuild a Document on Activation Example (C#)

This example shows how to activate and rebuild an assembly document.

```
//-----
// Preconditions:
// 1. Open an assembly.
// 2. Replace loaded_document with the assembly name.
// 3. Right-click a part in the FeatureManager design tree.
// 4. Select Open Part from the toolbar.
// 5. Edit a feature in the part.
// 6. Click OK in the PropertyManager to accept the changes.
//
// Postconditions:
// 1. Displays a dialog.
// 2. Click Yes to rebuild the assembly.
// 3. Activates the assembly document.
// 4. Inspect the Immediate Window.
//-----

using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        int Errors;
        bool bValue = false;
        int nValue;

        public void Main()
        {
            // Set user preference to not rebuild on activation
            bValue = swApp.SetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swRebuildOnActivation,
                (int)swRebuildOnActivation_e.swDontRebuildActiveDoc);
            Debug.Print("Rebuild user preference set to not rebuild on activation: " + bValue);

            nValue = swApp.GetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swRebuildOnActivation);
            Debug.Print("swRebuildOnActivation_e user preference is: " + nValue);

            // Ignore the user preference just set
            Debug.Print("Ignoring rebuild user preference.");

            // Activate the loaded document and prompt for rebuild if the model changed
            swModel = (ModelDoc2)swApp.ActivateDoc3("loaded_document", false, (int)swRebuildOnActivation_e.swUserDecision, ref
Errors);
            Debug.Print("Error code after document activation: " + Errors.ToString());

        }

        public SldWorks swApp;
    }
}
```

Rebuild a Document on Activation Example (VBA)

This example shows how to activate and rebuild an assembly document.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Replace loaded_document with the assembly name.
' 3. Right-click a part in the FeatureManager design tree.
' 4. Select Open Part from the toolbar.
' 5. Edit a feature in the part.
' 6. Click OK in the PropertyManager to accept the changes.
'
' Postconditions:
' 1. Displays a dialog.
' 2. Click Yes to rebuild the assembly.
' 3. Activates the assembly document.
' 4. Inspect the Immediate Window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim Errors As Long
```

Sub main()

```
    Dim bValue As Boolean
    Dim lValue As Long
    Dim nValue As SwConst.swRebuildOnActivation_e

    Set swApp = Application.SldWorks

    ' Set user preference to not rebuild on activation
    bValue = swApp.SetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swRebuildOnActivation,
swRebuildOnActivation_e.swDontRebuildActiveDoc)
    Debug.Print ("Rebuild user preference set to not rebuild on activation: " & bValue)
    nValue = swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swRebuildOnActivation)
    Debug.Print ("swRebuildOnActivation_e user preference is: " & nValue)

    ' Ignore the user preference just set
    Debug.Print ("Ignoring rebuild user preference.")

    ' Activate the loaded document and prompt for rebuild if the model is changed
    Set swModel = swApp.ActivateDoc3("loaded_document", False, swRebuildOnActivation_e.swUserDecision, Errors)
    Debug.Print ("Error code after document activation: " & Errors)
```

```
End Sub
```

Rebuild a Document on Activation Example (VB.NET)

This example shows how to activate and rebuild an assembly document.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Replace loaded_document with the assembly name.
' 3. Right-click a part in the FeatureManager design tree.
' 4. Select Open Part from the toolbar.
' 5. Edit a feature in the part.
' 6. Click OK in the PropertyManager to accept the changes.
'
```

```
' Postconditions:
' 1. Displays a dialog.
' 2. Click Yes to rebuild the assembly.
' 3. Activates the assembly document.
' 4. Inspect the Immediate Window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
```

```
    Dim Errors As Long
```

```
    Sub main()
```

```
        Dim bValue As Boolean
```

```
        Dim nValue As swRebuildOnActivation_e
```

```
        ' Set user preference to not rebuild on activation
```

```
        bValue = swApp.SetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swRebuildOnActivation,
swRebuildOnActivation_e.swDontRebuildActiveDoc)
```

```
        Debug.Print("Rebuild user preference set to not rebuild on activation: " & bValue)
```

```
        nValue = swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swRebuildOnActivation)
```

```
        Debug.Print("swRebuildOnActivation_e user preference is: " & nValue)
```

```
        ' Ignore the user preference just set
```

```
        Debug.Print("Ignoring rebuild user preference.")
```

```
        ' Activate the loaded document and prompt for rebuild if the model is changed
```

```
        swModel = swApp.ActivateDoc3("loaded_document", False, swRebuildOnActivation_e.swUserDecision, Errors)
```

```
        Debug.Print("Error code after document activation: " + Errors.ToString)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Rebuild Model Example (VBA)

The example shows how to rebuild an assembly by using IModelDoc2::EditRebuild3 and IModelDoc2::ForceRebuild3.

```
'-----  
' Preconditions: Open an assembly document.  
'  
' Preconditions:  
' 1. Attempts to rebuild only those features in the  
'    assembly that need to be rebuilt and attempts  
'    to force a rebuild of all of the features  
'    in the assembly regardless if they need to be rebuilt.  
' 2. Read each message box and click OK to close it.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim boolStatus As Boolean  
    Dim topOnly As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
  
    If swModel.EditRebuild3() Then  
        MsgBox "Rebuild successful."  
    Else  
        MsgBox "This model has rebuild errors."  
    End If  
  
    topOnly = True  
    If swModel.ForceRebuild3(topOnly) Then  
        MsgBox "Forced rebuild successful."  
    Else  
        MsgBox "This model has rebuild errors."  
    End If  
  
End Sub
```

Recalculate Bounding Box Example (C#)

This example shows how to recalculate the bounding box of an assembly.

```
//-----
// Preconditions:
// 1. Specified assembly document exists.
// 2. Open the Immediate window.
// 3. Run the macro.
//
// Postconditions:
// 1. Opens assembly document.
// 2. Gets the bounding box for the assembly.
// 3. Prints the lower- and upper-diagonal corner points
//   of the bounding box to the Immediate window.
// 4. Modifies a dimension in a component in the assembly.
// 5. Updates the bounding box.
// 6. Prints the lower- and upper-diagonal corner points
//   of the bounding box to the Immediate window.
// 7. Examine the values printed to the Immediate window
//   to verify that the bounding box was updated.
//
// NOTE: Because this assembly is used elsewhere,
// do not save any changes when closing it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace UpdateBoxCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void ProcessAssyBox(SldWorks swApp, AssemblyDoc swAssy)
        {
            object box = null;
            double[] boxArray = new double[6];
            box = (object)swAssy.GetBox((int)swBoundingBoxOptions_e.swBoundingBoxIncludeRefPlanes);
            boxArray = (double[])box;
            Debug.Print(" Min = (" + boxArray[0] * 1000.0 + ", " + boxArray[1] * 1000.0 + ", " + boxArray[2] * 1000.0 + ") mm");
            Debug.Print(" Max = (" + boxArray[3] * 1000.0 + ", " + boxArray[4] * 1000.0 + ", " + boxArray[5] * 1000.0 + ") mm");
        }

        public void Main()
        {
            ModelDoc2 swModel;
            AssemblyDoc swAssy;
            ModelDocExtension swModelDocExt;
            Dimension swDimension;
            string fileName;
            int errors = 0;
            int warnings = 0;
            bool status;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\key pad_1.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swAssy = (AssemblyDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            // Print the two diagonal corner points
            // of the bounding box before modifying the
            // assembly
            Debug.Print("Before:");
            ProcessAssyBox(swApp, swAssy);

            // Change a dimension of one of the assembly components
            status = swModelDocExt.SelectByID2("Sketch1@Pad_1-1@key pad_1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swModel.EditSketch0;
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("D1@Sketch1@Pad_1-1@key pad_1", "DIMENSION", 0.00306153201295202, 0.0373842545521677, -0.0323079625553351, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swDimension = (Dimension)swModel.Parameter("D1@Sketch1@pad_1.Part");
            errors = (int)swDimension.SetSystemValue3(0.04, (int)swSetValueInConfiguration_e.swSetValue_InThisConfiguration, null);
            swModel.ClearSelection2(true);

            // Update the bounding box
            swAssy.UpdateBox();

            // Print the two diagonal corner points of the
            // bounding box after modifying the assembly
            Debug.Print("After:");
            ProcessAssyBox(swApp, swAssy);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Recalculate Bounding Box Example (VBA)

This example shows how to recalculate the bounding box of an assembly.

```

'-----
' Preconditions:
' 1. Specified assembly document exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Opens assembly document.
' 2. Gets the bounding box for the assembly.
' 3. Prints the lower- and upper-diagonal corner points
'   of the bounding box to the Immediate window.
' 4. Modifies a dimension in a component in the assembly.
' 5. Updates the bounding box.
' 6. Prints the lower- and upper-diagonal corner points
'   of the bounding box to the Immediate window.
' 7. Examine the values printed to the Immediate window
'   to verify that the bounding box was updated.
'
' NOTE: Because this assembly is used elsewhere,
' do not save any changes when closing it.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDimension As SldWorks.Dimension
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim status As Boolean

Sub ProcessAssyBox(swApp As SldWorks.SldWorks, swAssy As SldWorks.AssemblyDoc)
    Dim vBox As Variant
    vBox = swAssy.GetBox(0)
    Debug.Print "  Min = (" & vBox(0) * 1000# & ", " & vBox(1) * 1000# & ", " & vBox(2) * 1000# & ") mm"
    Debug.Print "  Max = (" & vBox(3) * 1000# & ", " & vBox(4) * 1000# & ", " & vBox(5) * 1000# & ") mm"
End Sub

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\key pad_1.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swAssy = swModel
    Set swModelDocExt = swModel.Extension

    ' Print the two diagonal corner points
    ' of the bounding box before modifying the
    ' assembly
    Debug.Print "Before:"

    ProcessAssyBox swApp, swAssy

    ' Change a dimension of one of the assembly components
    status = swModelDocExt.SelectByID("Sketch1@Pad_1-1@key pad_1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditSketch
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("D1@Sketch1@Pad_1-1@key pad_1", "DIMENSION", 3.06153201295202E-03, 3.73842545521677E-02, -3.23079625553351E-02,
    False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    Set swDimension = swModel.Parameter("D1@Sketch1@pad_1.Part")
    errors = swDimension.SetSystemValue3(0.04, swSetValueInConfiguration_e.swSetValue_InThisConfiguration, Nothing)
    swModel.ClearSelection2 True

    ' Update the bounding box
    swAssy.UpdateBox

    ' Print the two diagonal corner points of the
    ' bounding box after modifying the assembly
    Debug.Print "After:"
    ProcessAssyBox swApp, swAssy

End Sub

```

Recalculate Bounding Box Example (VB.NET)

This example shows how to recalculate the bounding box of an assembly.

```
'-----
' Preconditions:
' 1. Specified assembly document exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Opens assembly document.
' 2. Gets the bounding box for the assembly.
' 3. Prints the lower- and upper-diagonal corner points
'   of the bounding box to the Immediate window.
' 4. Modifies a dimension in a component in the assembly.
' 5. Updates the bounding box.
' 6. Prints the lower- and upper-diagonal corner points
'   of the bounding box to the Immediate window.
' 7. Examine the values printed to the Immediate window
'   to verify that the bounding box was updated.
'
' NOTE: Because this assembly is used elsewhere,
' do not save any changes when closing it.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swAssy As AssemblyDoc
    Dim swModelDocExt As ModelDocExtension
    Dim swDimension As Dimension
    Dim fileName As String
    Dim errors As Integer
    Dim warnings As Integer
    Dim status As Boolean

    Sub ProcessAssyBox(ByVal swApp As SldWorks, ByVal swAssy As AssemblyDoc)
        Dim vBox As Object
        vBox = swAssy.GetBox(0)
        Debug.Print("  Min = (" & vBox(0) * 1000.0# & ", " & vBox(1) * 1000.0# & ", " & vBox(2) * 1000.0# & ") mm")
        Debug.Print("  Max = (" & vBox(3) * 1000.0# & ", " & vBox(4) * 1000.0# & ", " & vBox(5) * 1000.0# & ") mm")
    End Sub

    Sub Main()

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\key pad_1.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swAssy = swModel
        swModelDocExt = swModel.Extension

        ' Print the two diagonal corner points
        ' of the bounding box before modifying the
        ' assembly
        Debug.Print("Before:")
        ProcessAssyBox(swApp, swAssy)

        ' Change a dimension of one of the assembly components
        status = swModelDocExt.SelectByID2("Sketch1@Pad_1-1@key pad_1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditSketch()
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("D1@Sketch1@Pad_1-1@key pad_1", "DIMENSION", 0.00306153201295202, 0.0373842545521677, -0.0323079625553351, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swDimension = swModel.Parameter("D1@Sketch1@pad_1.Part")
        errors = swDimension.SetSystemValue3(0.04, swSetValueInConfiguration_e.swSetValue_InThisConfiguration, Nothing)
        swModel.ClearSelection2(True)

        ' Update the bounding box
        swAssy.UpdateBox()

        ' Print the two diagonal corner points of the
        ' bounding box after modifying the assembly
        Debug.Print("After:")
        ProcessAssyBox(swApp, swAssy)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Record Macros Example (C#)

This example shows how to add two lines to a macro recording.

```
//-----
// Preconditions:
// 1. Add a reference to SolidWorks.Interop.swcommands.dll.
//
// Postconditions:
// 1. Displays a message box informing you that
//    macro recording has started. Click OK.
// 2. Displays the Save as dialog.
// 3. Type Macro2 in File name and select the languages
//    to which to save macros in Save as type.
// 4. Writes two lines of text to the macros in the languages
//    corresponding to the Save as type you selected.
// 5. Displays a message box informing you that
//    macro recording has ended. Click OK.
// 6. Displays a message box asking you if you
//    want to stop debugging. Click Yes.
// 7. Run the just-recorded macros to verify.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swcommands;
using System;
using System.Windows.Forms;

namespace RecordLineCSharpSldWorksCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public SldWorks swAppEvents;

        public void Main()
        {
            // Set up events
            swAppEvents = (SldWorks)swApp;
            AttachEventHandlers();

            // Start macro recording
            swApp.RunCommand((int)swCommands_e.swCommands_RecordPauseMacro, "");

            // Write to VBA macro
            swApp.RecordLine(" Test");
            swApp.RecordLine("MsgBox(\"C:\\Test\\\")");

            // Write to C# macro
            swApp.RecordLineCSharp("// Test");
            swApp.RecordLineCSharp("System.Windows.Forms.MessageBox.Show(\"C:\\\\Test\\\\\\\")");

            // Write to VB.NET macro
            swApp.RecordLineVBnet(" Test");
            swApp.RecordLineVBnet("MsgBox(\"C:\\Test\\\")");

            //Stop the macro recordings
            swApp.RunCommand((int)swCommands_e.swCommands_StopMacro, "");
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAppEvents.BeginRecordNotify += this.swAppEvents_BeginRecordNotify;
            swAppEvents.EndRecordNotify += this.swAppEvents_EndRecordNotify;
        }

        private int swAppEvents_BeginRecordNotify()
        {
            //Send message when the macro recording starts
            MessageBox.Show("Macro recording starting.");
            return 1;
        }

        private int swAppEvents_EndRecordNotify()
        {
            //Send message when macro recording ends
            MessageBox.Show("Macro recording ended.");
            return 1;
        }
    }
}
```

```
/// <summary>  
/// The SldWorks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;
```

```
}  
}
```

Record Macros Example (VBA)

This example shows how to add two lines to one or more macro recording sessions.

```

'-----
' Preconditions:
' 1. Add a reference to SolidWorks version Commands type library.
' 2. Copy Macro to the main module.
' 3. Click Insert > Class Module and copy Class1 to that module.
'
' Postconditions:
' 1. Displays a message box informing you that
'    macro recording has started. Click OK.
' 2. Displays the Save as dialog.
' 3. Type Macro2 in File name and select the languages
'    to which to save macros in Save as type.
' 4. Writes two lines of text to the macros in the languages
'    corresponding to languages that you selected.
' 5. Displays a message box informing you that
'    macro recording has ended. Click OK.
' 6. Run the just-recorded macros to verify.
'-----
'Macro

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swAppEvents As New Class1

    Set swApp = Application.SldWorks

    ' Set up events
    Set swAppEvents.swApp = swApp

    ' Start macro recording
    swApp.RunCommand swCommands_RecordPauseMacro, ""

    ' Write to VBA macro
    swApp.RecordLine "" Test"
    swApp.RecordLine "MsgBox(" & Chr(34) & "C:\Test\" & Chr(34) & ")"

    ' Write to C# macro
    swApp.RecordLineCSharp "// Test"
    swApp.RecordLineCSharp "System.Windows.Forms.MessageBox.Show(" & Chr(34) & "C:\\Test\\" & Chr(34) & ");"

    ' Write to VB.NET macro
    swApp.RecordLineVBnet "" Test"
    swApp.RecordLineVBnet "MsgBox(" & Chr(34) & "C:\Test\" & Chr(34) & ")"

    ' Stop macro recording
    swApp.RunCommand swCommands_StopMacro, ""

End Sub

'Class1

Option Explicit

Public WithEvents swApp As SldWorks.SldWorks

Private Function swApp_BeginRecordNotify() As Long
    'Send message when the macro recording starts
    MsgBox "Macro recording starting."
End Function

Private Function swApp_EndRecordNotify() As Long
    'Send message when macro recording ends
    MsgBox "Macro recording ended."
End Function

```

Record Macros Example (VB.NET)

This example shows how to add two lines to one or more macro recording sessions.

```

'-----
' Preconditions: Add a reference to SolidWorks.Interop.swcommands.dll.
'
' Postconditions:
' 1. Displays a message box informing you that
'    macro recording has started. Click OK.
' 2. Displays the Save as dialog.
' 3. Type Macro2 in File name and select the languages
'    to which to save macros in Save as type.
' 4. Writes two lines of text to the macros in the languages
'    corresponding to the Save as type that you selected.
' 5. Displays a message box informing you that
'    macro recording has ended. Click OK.
' 6. Displays a message box asking you if you
'    want to stop debugging. Click Yes.
' 7. Run the just-recorded macros to verify.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swcommands
Imports System

Partial Class SolidWorksMacro

    Public WithEvents swAppEvents As SldWorks

    Public Sub Main()

        ' Set up events
        swAppEvents = swApp
        AttachEventHandlers()

        ' Start macro recording
        swApp.RunCommand(swCommands_e.swCommands_RecordPauseMacro, "")

        ' Write to VBA macro
        swApp.RecordLine(" Test")
        swApp.RecordLine("MsgBox(" & Chr(34) & "C:\Test\" & Chr(34) & ")")

        ' Write to C# macro
        swApp.RecordLineCSharp("// Test")
        swApp.RecordLineCSharp("System.Windows.Forms.MessageBox.Show(" & Chr(34) & "C:\\Test\\" & Chr(34) & ");")

        ' Write to VB.NET macro
        swApp.RecordLineVBnet(" Test")
        swApp.RecordLineVBnet("MsgBox(" & Chr(34) & "C:\Test\" & Chr(34) & ")")

        ' Stop macro recording
        swApp.RunCommand(swCommands_e.swCommands_StopMacro, "")

    End Sub

    Sub AttachEventHandlers()
        AttachSwEvents()
    End Sub

    Sub AttachSwEvents()
        AddHandler swAppEvents.BeginRecordNotify, AddressOf Me.swAppEvents_BeginRecordNotify
        AddHandler swAppEvents.EndRecordNotify, AddressOf Me.swAppEvents_EndRecordNotify
    End Sub

    Private Function swAppEvents_BeginRecordNotify() As Integer
        'Send message when the macro recording starts
        MsgBox("Macro recording starting.")
    End Function

    Private Function swAppEvents_EndRecordNotify() As Integer
        'Send message when macro recording ends
        MsgBox("Macro recording ended.")
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Redirect Spotlight Example (VBA)

This examples shows how select a spotlight and two sketch points. The spotlight is then redirected to point along a vector that is defined by the two points.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Adds a spotlight.
' 3. Inserts a 3D sketch.
' 4. Selects the spotlight.
' 5. Selects a new root point and direction point for the spotlight.
' 6. Gets the current position and direction of the spotlight.
' 7. Modifies the position and direction of the spotlight.
' 8. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swModelView As SldWorks.ModelView
    Dim swFeature As SldWorks.Feature
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSketchManager As SldWorks.SketchManager
    Dim swSketch As SldWorks.Sketch
    Dim swMath As SldWorks.MathUtility
    Dim swLightFeature As SldWorks.Feature
    Dim swLight As SldWorks.Light
    Dim selRootSkPt As SldWorks.SketchPoint
    Dim selDirSkPt As SldWorks.SketchPoint
    Dim swSketchTrans As SldWorks.MathTransform
    Dim ptArr(2) As Double
    Dim dirArr(2) As Double
    Dim rootPoint As SldWorks.MathPoint
    Dim dirPoint As SldWorks.MathPoint
    Dim dirVector As SldWorks.MathVector
    Dim params As Variant
    Dim skSegment As Object
    Dim status As Boolean
    Dim errors As Long
    Dim warnings As Long
    Dim fileName As String
    Dim rect As Variant

    Set swApp = Application.SldWorks

    'Open assembly
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\appearances\usb_flash_drive1.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Add spotlight
    status = swModel.AddLightSource("SW#5", 2, "Spot1")
    status = swModel.SetLightSourcePropertyValuesVB("SW#5", 2, 0.5, 8454143, 1, -9.999999999999991E-02, 0.1700000000000101, 1.109999999999999, 0.179999999999999, -9.000000000001008E-02, -1.029999999999999, 45, 0, 0, 0, 0.4, 0.4, 0, False)
    status = swModel.LockLightToModel(4, True)
    Set swModelView = swModel.ActiveView
    swModelView.GraphicsRedraw (rect)

    'Insert 3D sketch
    Set swSketchManager = swModel.SketchManager
    swSketchManager.Insert3DSketch True
    Set skSegment = swSketchManager.CreateLine(-0.038076, 0.043671, -0#, -0.01322, 0.054563, -0#)
    Set swSketch = swModel.GetActiveSketch2()
    status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0)
    Set skSegment = swSketchManager.CreateLine(-0.01322, 0.054563, -0#, -0.01322, 0.08124, 0.018547)
    Set swSketch = swModel.GetActiveSketch2()
    status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0)
    Set skSegment = swSketchManager.CreateLine(-0.01322, 0.08124, 0.018547, 0.000568, 0.08124, 0.004759)
    Set swSketch = swSketchManager.ActiveSketch
    status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    'Select the light
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    status = swModelDocExt.SelectByID2("Spot1", "LIGHTS", 0, 0, 0, False, 0, Nothing, 0)
    Set swLightFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swLight = swLightFeature.GetSpecificFeature2

    'Select a root point
    status = swModel.Extension.SelectByID2("Point3@3DSketch1", "EXTSKETCHPOINT", 0, 0, 0, False, 0, Nothing, 0)
    Set selRootSkPt = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketch = selRootSkPt.GetSketch

    'Need the sketch to model transform
    Set swMath = swApp.GetMathUtility
    Set swSketchTrans = swSketch.ModelToSketchTransform.Inverse

```

```

ptArr(0) = selRootSkPt.X
ptArr(1) = selRootSkPt.Y
ptArr(2) = selRootSkPt.Z
Set rootPoint = swMath.CreatePoint((ptArr))

'Make sure that the point is in model space
Set rootPoint = rootPoint.MultiplyTransform(swSketchTrans)

'Select a direction point
status = swModelDocExt.SelectByID2("Point1@3DSketch1", "EXTSKETCHPOINT", 0, 0, 0, False, 0, Nothing, 0)
Set selDirSkPt = swSelMgr.GetSelectedObject6(1, -1)
Set swSketch = selDirSkPt.GetSketch
Set swSketchTrans = swSketch.ModelToSketchTransform.Inverse
ptArr(0) = selDirSkPt.X
ptArr(1) = selDirSkPt.Y
ptArr(2) = selDirSkPt.Z
Set dirPoint = swMath.CreatePoint((ptArr))
Set dirPoint = dirPoint.MultiplyTransform(swSketchTrans)

'Determine the direction of the spotlight
Set dirVector = dirPoint.Subtract(rootPoint)
params = rootPoint.ArrayData
ptArr(0) = params(0)
ptArr(1) = params(1)
ptArr(2) = params(2)
params = dirVector.ArrayData
dirArr(0) = params(0)
dirArr(1) = params(1)
dirArr(2) = params(2)

'Get the current parameters for the spotlight
params = swModel.LightSourcePropertyValues(swLight.GetID())
Dim nbrParams As Variant
nbrParams = UBound(params)
Dim i As Long
Debug.Print ("Current position(3) and direction(3)")
For i = 5 To (nbrParams - 8)
    Debug.Print params(i)
Next i

'Modify the root point and direction vector, then apply the changes
params(5) = ptArr(0)
params(6) = ptArr(1)
params(7) = ptArr(2)
params(8) = dirArr(0)
params(9) = dirArr(1)
params(10) = dirArr(2)
swModel.LightSourcePropertyValues(swLight.GetID()) = params
Debug.Print ""
nbrParams = UBound(params)
Debug.Print ("Modified position(3) and direction(3)")
For i = 5 To (nbrParams - 8)
    Debug.Print params(i)
Next i

End Sub

```

Redraw Graphics Example (VBA)

This example shows how to redraw the graphics area.

```
'-----  
' Preconditions: Open a model document.  
'  
' Postconditions: Redraws the graphics area.  
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    swModel.GraphicsRedraw2  
  
End Sub
```

Fire Assembly Rebuild Events Example (C#)

This example shows how to handle pre-notification and post-notification events for an assembly, which are fired when the assembly is rebuilt.

```
// -----
// Preconditions:
// 1. Verify that the specified assembly document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly document.
// 2. Inserts a cut-extrude assembly feature.
// 3. Pops up a dialog with the rebuild pre-notification.
// 4. Click OK to close the dialog.
// 5. Rebuilds the assembly.
// 6. Pops up a dialog with the rebuild post-notification.
// 7. Click OK to close the dialog.
// 8. Rolls back the rollback bar and pops up a dialog with the
//    rebuild post-notification.
// 9. Click OK to close the dialog.
// 10. Examine the Immediate window and FeatureManager design tree.
// 11. Click Tools > Options > System Options > Stop VSTA debugger on macro exit > OK.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
// -----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace Macro1.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        public AssemblyDoc swAssembly;
        SketchManager swSketchManager;
        ModelDocExtension swModelDocExt;
        SketchSegment swSketchSegment;
        FeatureManager swFeatureManager;
        Feature swFeature;
        SelectionMgr swSelectionMgr;
        string fileName;
        int errors;
        int warnings;

        public void Main()
        {
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, false);

            // Open assembly
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\smartcomponents\\stepped_shaft.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref
            errors, ref warnings);

            // Insert a cut-extrude assembly feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(-0.016076, -0.532382, 0.0, 0.044465, -0.546417, 0.0);
            swModel.ClearSelection2(true);
            swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureCut3(true, false, false, 0, 0, 0.00254, 0.00254, false, false, false, false, 0.0174532925199433,
            0.0174532925199433, false, false, false, false, false, true, true, true, true, false, 0, 0, false);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swSelectionMgr.EnableContourSelection = false;
            swModel.ClearSelection2(true);

            // Event notification
            swAssembly = (AssemblyDoc)swModel;
            AttachEventHandlers();

            // Rebuild the model
            swModel.ForceRebuild3(true);

            // Roll back the rollback bar
            swFeatureManager.EditRollback((int)swMoveRollbackBarTo_e.swMoveRollbackBarToBeforeFeature, swFeature.Name);
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssembly.RegenNotify += this.swAssembly_RegenNotify;
            swAssembly.RegenPostNotify2 += this.swAssembly_RegenPostNotify2;
        }

        private int swAssembly_RegenNotify()
        {
            // Display message before rebuild
            System.Windows.Forms.MessageBox.Show("A rebuild pre-notification event was fired.");
            return 0;
        }

        private int swAssembly_RegenPostNotify2(object stopFeature)
        {
        }
    }
}
```

```

    {
        // Display message after rebuild
        if ((stopFeature != null))
        {
            Feature feature = default(Feature);
            feature = (Feature)stopFeature;
            Debug.Print("The rollback bar is above " + feature.Name + " in the FeatureManager design tree.");
        }
        System.Windows.Forms.MessageBox.Show("A rebuild post-notification event was fired.");
        return 0;
    }

    public SldWorks swApp;
}

```

Fire Assembly Rebuild Events Example (VBA)

This example shows how to handle pre-notification and post-notification events for an assembly, which are fired when the assembly is rebuilt.

```
'-----
' Preconditions:
' 1. Copy and paste this code in the main module.
' 2. Click Insert > Class module and copy and paste this code in that module.
' 3. Verify that the specified assembly document to open exists.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly document.
' 2. Inserts a cut-extrude assembly feature.
' 3. Pops up a dialog with the rebuild pre-notification.
' 4. Click OK to close the dialog.
' 5. Rebuilds the assembly.
' 6. Pops up a dialog with the rebuild post-notification.
' 7. Click OK to close the dialog.
' 8. Rolls back the rollback bar and pops up a dialog with the
'    rebuild post-notification.
' 9. Click OK to close the dialog.
' 10. Examine the Immediate window and FeatureManager design tree.
' 11. Click Reset in the IDE to exit the macro.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

' Main module code
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssembly As SldWorks.AssemblyDoc
Dim swSketchManager As SldWorks.SketchManager
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.feature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim swAssemblyEvents As Class1

Sub main()

    Set swApp = Application.SldWorks

    ' Open assembly
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\stepped_shaft.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    ' Insert a cut-extrude assembly feature
    Set swModelDocExt = swModel.Extension
    swModelDocExt.SelectByID2 "Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0
    swModel.ClearSelection2 True
    Set swSketchManager = swModel.SketchManager
    swSketchManager.InsertSketch True
    Set swSketchSegment = swSketchManager.CreateCircle(-0.016076, -0.532382, 0#, 0.044465, -0.546417, 0#)
    swModel.ClearSelection2 True
    swModelDocExt.SelectByID2 "Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.FeatureCut3(True, False, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, True, False, 0, 0, False)
    Set swSelectionMgr = swModel.SelectionManager
    swSelectionMgr.EnableContourSelection = False
    swModel.ClearSelection2 True

    ' Event notification
    Set swAssembly = swModel
    Set swAssemblyEvents = New Class1
    Set swAssemblyEvents.swAssembly = swApp.ActiveDoc

    ' Rebuild the model
    swModel.ForceRebuild3 True

    ' Roll back the model
    swFeatureManager.EditRollback swMoveRollbackBarTo_e.swMoveRollbackBarToBeforeFeature, swFeature.Name

End Sub

Back to top

'Class1 module
Option Explicit

Public WithEvents swAssembly As SldWorks.AssemblyDoc

Private Function swAssembly_RegenNotify() As Long
    ' Display message before rebuild
    MsgBox "A rebuild pre-notification event was fired."
End Function

Private Function swAssembly_RegenPostNotify2(ByVal stopFeature As Object) As Long
    ' Display message after rebuild
    If Not stopFeature Is Nothing Then
```

```
Dim feature As SldWorks.feature
Set feature = stopFeature
Debug.Print "The rollback bar is above " & feature.Name & " in the FeatureManager design tree."
End if
MsgBox "A rebuild post-notification event was fired."
End Function
```

[Back to top](#)

Fire Assembly Rebuild Events Example (VB.NET)

This example shows how to handle pre-notification and post-notification events for an assembly, which are fired when the assembly is rebuilt.

```

-----
Preconditions:
1. Verify that the specified assembly document to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified assembly document.
2. Inserts a cut-extrude assembly feature.
3. Pops up a dialog with the rebuild pre-notification.
4. Click OK to close the dialog.
5. Rebuilds the assembly.
6. Pops up a dialog with the rebuild post-notification.
7. Click OK to close the dialog.
8. Rolls back the rollback bar and pops up a dialog with the rebuild
post-notification.
9. Click OK to close the dialog.
10. Examine the Immediate window and FeatureManager design tree.
11. Click Tools > Options > System Options > Stop VSTA debugger on macro exit > OK.

NOTE: Because the assembly is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swSketchManager As SketchManager
    Dim swModelDocExt As ModelDocExtension
    Dim swSketchSegment As SketchSegment
    Dim swFeatureManager As FeatureManager
    Dim swFeature As Feature
    Dim swSelectionMgr As SelectionMgr
    Dim fileName As String
    Dim errors As Integer
    Dim warnings As Integer
    Public WithEvents swAssembly As AssemblyDoc

    Sub main()

        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, False)

        ' Open assembly
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\smartcomponents\stepped_shaft.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        ' Insert a cut-extrude assembly feature
        swModelDocExt = swModel.Extension
        swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchManager = swModel.SketchManager
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateCircle(-0.016076, -0.532382, 0.0#, 0.044465, -0.546417, 0.0#)
        swModel.ClearSelection2(True)
        swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureCut3(True, False, 0, 0, 0.00254, 0.00254, False, False, False, False, 0.0174532925199433, 0.01745329251
99433, False, False, False, False, False, True, True, True, True, False, 0, 0, False)
        swSelectionMgr = swModel.SelectionManager
        swSelectionMgr.EnableContourSelection = False
        swModel.ClearSelection2(True)

        ' Event notification
        swAssembly = swModel
        AttachEventHandlers()

        ' Rebuild the model
        swModel.ForceRebuild3(True)

        ' Roll back the rollback bar
        swFeatureManager.EditRollback(swMoveRollbackBarTo_e.swMoveRollbackBarToBeforeFeature, swFeature.Name)

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swAssembly.RegenNotify, AddressOf Me.swAssembly_RegenNotify
        AddHandler swAssembly.RegenPostNotify2, AddressOf Me.swAssembly_RegenPostNotify2
    End Sub

    Private Function swAssembly_RegenNotify() As Integer
        ' Display message before rebuild
        MsgBox("A rebuild pre-notification event was fired.")
    End Function

    Private Function swAssembly_RegenPostNotify2(ByVal stopFeature As Object) As Integer
        ' Display message after rebuild
        If Not stopFeature Is Nothing Then
            Dim feature As Feature
            feature = stopFeature
            Debug.Print("The rollback bar is above " & feature.Name & " in the FeatureManager design tree.")
        End If
        MsgBox("A rebuild post-notification event was fired.")
    End Function

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

Release Notes

[Send Feedback](#)

This topic provides you with quick access to the enhancements in SOLIDWORKS API 2026.

Service Pack 3

New interfaces, methods, and properties

[IVersionCompatibilityItem](#) Interface

- [IVersionCompatibilityItem.IncompatibleObject](#)
- [IVersionCompatibilityItem.Version](#)
- [IVersionCompatibilityItem.ID](#)
- [IVersionCompatibilityItem.Message](#)
- [IVersionCompatibilityItem.Action](#)

IModelDocExtension Interface

- [IModelDocExtension.CheckVersionCompatibility](#)

DSldWorksEvents Handlers

- [DSldWorksEvents.FileSaveToVersionFailNotifyEventHandler](#)

DAssemblyDocEvents Handlers

- [DAssemblyDocEvents.AddDependencyNotifyEventHandler](#)
- [DAssemblyDocEvents.DeleteDependencyNotifyEventHandler](#)

DDrawingDocEvents Handlers

- [DDrawingDocEvents.AddDependencyNotifyEventHandler](#)
- [DDrawingDocEvents.DeleteDependencyNotifyEventHandler](#)

DPartDocEvents Handlers

- [DPartDocEvents.AddDependencyNotifyEventHandler](#)
- [DPartDocEvents.DeleteDependencyNotifyEventHandler](#)

Service Pack 2

New interfaces, methods, and properties

IComponent2 Interface

- [IComponent2.HasAssemblyFeatures](#)

[IDimXpertAngleBetweenCircularDimTol](#) Interface

- [IDimXpertAngleBetweenCircularDimTol.CenterFeature](#)
- [IDimXpertAngleBetweenCircularDimTol.GetActPlaneVector](#)
- [IDimXpertAngleBetweenCircularDimTol.GetNomPlaneVector](#)
- [IDimXpertAngleBetweenCircularDimTol.OriginFeature](#)
- [IDimXpertAngleBetweenCircularDimTol.Supplement](#)

[IDimXpertPart](#) Interface

- [IDimXpertPart.InsertAngleDimension](#)

[IDimXpertRefPlaneFeature](#) Interface

- [IDimXpertRefPlaneFeature.GetNominalPlane](#)

IGTol Interface

- [IGTol.GetAllOver](#)
- [IGTol.SetAllOver](#)

IView Interface

- [IView.RootDrawingComponent2](#) (obsoletes [IView.RootDrawingComponent](#))

Service Pack 1

New properties and method

IBoundingBoxFeatureData Interface

- [IBoundingBoxFeatureData.CoordinateSystem](#)
- [IBoundingBoxFeatureData.MatDirectionType](#)

IDSPBRMaterial Interface

- [IDSPBRMaterial.ClearcoatNormalStrength](#)
- [IDSPBRMaterial.EmissionColorTexture](#)
- [IDSPBRMaterial.MetallicTexture](#)
- [IDSPBRMaterial.NormalStrength](#)
- [IDSPBRMaterial.TransparencyTexture](#)

IModelDocExtension Interface

- [IModelDocExtension.ActivateFeatureMgrViewx64](#)
- [IModelDocExtension.AddFeatureMgrViewx64](#)
- [IModelDocExtension.DeActivateFeatureMgrViewx64](#)

IModelView Interface

- [IModelView.UnAxisFlyout](#)

Service Pack 0

- [New functionality](#)
- [New interfaces](#)
- [New delegates, methods, and properties](#)

New functionality

- Display the Dassault Systemes Physically Based Rendering (DSPBR) appearance visual style. See [IDSPBRMaterial.RenderMaterial.GetDSPBRMaterial](#) and [IRenderMaterial.HasDSPBRProperties](#).
- Add family tables to drawings and edit them. See [IFamilyTableAnnotation.FamilyTableFeature](#), [ISheet.InsertFamilyTable](#).
- Get top-level items in body folders. See [IBodyFolder.GetTopLevelFeatureCount](#) and [IBodyFolder.GetTopLevelFeatures](#).
- Receive notifications when reordering solid body folders. See [DAssemblyDocEvents.SolidBodyFolderReorderNotifyEventHandler](#) and [DPartDocEvents.SolidBodyFolderReorderNotifyEventHandler](#).
- Get and set the edge thickness when the display style is set to "Shaded with Edges". See [ISldWorks.ShadedEdgeThickness](#).
- Move parent drawing views independently of child views and receive notifications when changing view positions. See [IView.SetViewPosition](#) and [DDrawingDocEvents.ViewPositionChangeNotifyEventHandler](#).

New interfaces

[IDSPBRMaterial](#) Interface

- [IDSPBRMaterial.AlbedoNumber](#)
- [IDSPBRMaterial.Albedo](#)
- [IDSPBRMaterial.AlbedoTexture](#)

- [IDSPBRMaterial - Anisotropy](#)
- [IDSPBRMaterial - AnisotropyRotation](#)
- [IDSPBRMaterial - AttenuationColor](#)
- [IDSPBRMaterial - AttenuationDistance](#)
- [IDSPBRMaterial - ClearCoat](#)
- [IDSPBRMaterial - ClearCoatNormalTexture](#)
- [IDSPBRMaterial - ClearCoatRoughness](#)
- [IDSPBRMaterial - CutoutOpacity](#)
- [IDSPBRMaterial - CutoutOpacityTexture](#)
- [IDSPBRMaterial - Displacement](#)
- [IDSPBRMaterial - DisplacementTexture](#)
- [IDSPBRMaterial - EmissionColor](#)
- [IDSPBRMaterial - EmissionEnergyNormalization](#)
- [IDSPBRMaterial - EmissionMode](#)
- [IDSPBRMaterial - EmissionValue](#)
- [IDSPBRMaterial - FlakeColor](#)
- [IDSPBRMaterial - FlakeCoverage](#)
- [IDSPBRMaterial - FlakeRoughness](#)
- [IDSPBRMaterial - FlakeSize](#)
- [IDSPBRMaterial - FlakeTexture](#)
- [IDSPBRMaterial - FlipFlop](#)
- [IDSPBRMaterial - FlipFlopColor](#)
- [IDSPBRMaterial - IndexOfRefraction](#)
- [IDSPBRMaterial - Metallic](#)
- [IDSPBRMaterial - NormalTexture](#)
- [IDSPBRMaterial - RealViewToDSPBRTextureScale](#)
- [IDSPBRMaterial - Roughness](#)
- [IDSPBRMaterial - RoughnessTexture](#)
- [IDSPBRMaterial - SheenColor](#)
- [IDSPBRMaterial - SheenRoughness](#)
- [IDSPBRMaterial - Specular](#)
- [IDSPBRMaterial - SpecularTintColor](#)
- [IDSPBRMaterial - SubsurfaceAnisotropy](#)
- [IDSPBRMaterial - SubsurfaceColor](#)
- [IDSPBRMaterial - SubType](#)
- [IDSPBRMaterial - ThinFilm](#)
- [IDSPBRMaterial - ThinFilmIOR](#)
- [IDSPBRMaterial - ThinFilmThickness](#)
- [IDSPBRMaterial - ThinFilmThicknessTexture](#)
- [IDSPBRMaterial - ThinWalled](#)
- [IDSPBRMaterial - Translucency](#)
- [IDSPBRMaterial - TranslucencyColor](#)
- [IDSPBRMaterial - Transparency](#)
- [IDSPBRMaterial - Version](#)

[IFamilyTableAnnotation](#) Interface

- [IFamilyTableAnnotation - FamilyTableFeature](#)
- [IFamilyTableAnnotation - GetSheet](#)

[IFamilyTableFeature](#) Interface

- [IFamilyTableFeature - GetConfigurations](#)
- [IFamilyTableFeature - GetFeature](#)
- [IFamilyTableFeature - GetFollowConfigTreeOrder](#)
- [IFamilyTableFeature - GetModelName](#)
- [IFamilyTableFeature - GetSequenceNumberIncrement](#)
- [IFamilyTableFeature - GetSequenceStartNumber](#)
- [IFamilyTableFeature - GetTableAnnotationCount](#)
- [IFamilyTableFeature - GetTableAnnotations](#)
- [IFamilyTableFeature - GetUseAllConfigurations](#)
- [IFamilyTableFeature - SetConfigurations](#)
- [IFamilyTableFeature - SetFollowConfigTreeOrder](#)
- [IFamilyTableFeature - SetSequenceNumberIncrement](#)
- [IFamilyTableFeature - SetSequenceStartNumber](#)
- [IFamilyTableFeature - SetUseAllConfigurations](#)

New delegates, methods, and properties

DAssemblyDocEvents Handlers

- [DAssemblyDocEvents - SolidBodyFolderReorderNotifyEventHandler](#)

DDrawingDocEvents Handlers

- [DDrawingDocEvents - ViewPositionChangeNotifyEventHandler](#)

DPartDocEvents Handlers

- [DPartDocEvents - SolidBodyFolderReorderNotifyEventHandler](#)

IBodyFolder Interface

- [IBodyFolder - GetTopLevelFeatureCount](#)
- [IBodyFolder - GetTopLevelFeatures](#)

ISheet Interface

- [ISheet - InsertFamilyTable](#)

IRenderMaterial Interface

- [IRenderMaterial - GetDSPBRMaterial](#)
- [IRenderMaterial - HasDSPBRProperties](#)

ISldWorks Interface

- [ISldWorks - ShadedEdgeThickness](#)

IView Interface

- [IView - SetViewPosition](#)

[Back to top](#)

Reload Model Example (VBA)

This example shows how to reload the active model with the last saved version of the model.

```
'-----
' Preconditions:
' 1. Open a part or assembly document.
' 2. Suppress a feature in the part or a
'    component in the assembly.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Replaces the active part or assembly model
'    with last saved model.
' 2. Examine graphics area and the Immediate window.
'
' NOTE: Any changes made to the model in the current
' session are discarded if they were not saved
' previous to running this macro.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim nRetVal As Long

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc

    ' Fails for drawings because functionality is not supported in user interface
    nRetVal = swModel.Extension.ReloadOrReplace(False, swModel.GetPathName, True, True)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " ReloadOrReplace (0 = swComponentReloadError_e.swReloadOkay) = " & nRetVal

End Sub
```

Remove Edge from Edge Flange Feature Example (C#)

This example shows how to remove an edge from an edge flange feature in a sheet metal part.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\SM1-remove-edges.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Removes an edge from Edge-Flange1.
// 2. Examine the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace RemoveEdgesFromEdgeFlange_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swDocument = default(ModelDoc2);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            EdgeFlangeFeatureData swEdgeFlangeData = default(EdgeFlangeFeatureData);
            Feature swFeature = default(Feature);
            int lNumEdges = 0;
            Edge[] aEdgesToRemove = null;
            int lIdx = 0;
            bool bStatus = false;
            int lNumEdgesToRemove = 0;
            object[] vEdgesToRemove = null;
            object[] vEdges = null;
            swEdgeFlangeError_e nStatus = default(swEdgeFlangeError_e);
            ModelDocExtension modDocExt = default(ModelDocExtension);

            swDocument = (ModelDoc2)swApp.ActiveDoc;
            swSelectionManager = (SelectionMgr)swDocument.SelectionManager;

            modDocExt = (ModelDocExtension)swDocument.Extension;
            bStatus = modDocExt.SelectByID2("Edge-Flange1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelectionManager.GetSelectedObject6(1, 0);

            swEdgeFlangeData = (EdgeFlangeFeatureData)swFeature.GetDefinition();

            swEdgeFlangeData.AccessSelections(swDocument, null);

            vEdges = (object[])swEdgeFlangeData.Edges;
            lNumEdges = vEdges.GetUpperBound(0) - vEdges.GetLowerBound(0) + 1;
            Debug.Print("Number of edges in edge flange = " + swEdgeFlangeData.GetEdgeCount());

            //
            // Specify number of edges to remove
            //
            lNumEdgesToRemove = 1;

            vEdgesToRemove = null;
            aEdgesToRemove = new Edge[lNumEdgesToRemove];

            if ((lNumEdgesToRemove >= 1))
            {
                for (lIdx = 0; lIdx <= (lNumEdgesToRemove - 1); lIdx++)
                {
                    aEdgesToRemove[lIdx] = (Edge)vEdges[lIdx];
                }
                vEdgesToRemove = aEdgesToRemove;
            }

            nStatus = (swEdgeFlangeError_e)swEdgeFlangeData.RemoveEdges(vEdgesToRemove);
            Debug.Print("Edge removal error code = " + swEdgeFlangeError2String(nStatus));

            bStatus = swFeature.ModifyDefinition(swEdgeFlangeData, swDocument, null);
            Debug.Print("Status of modify definition = " + bStatus);

            swEdgeFlangeData.AccessSelections(swDocument, null);
            vEdges = null;
            vEdges = (object[])swEdgeFlangeData.Edges;
            if ((vEdges != null))
            {
                lNumEdges = vEdges.GetUpperBound(0) - vEdges.GetLowerBound(0) + 1;
                Debug.Print("Number of edges in edge flange = " + lNumEdges);
            }
            else
            {
                Debug.Print("Number of edges in edge flange = 0");
            }

            swEdgeFlangeData.ReleaseSelectionAccess();
        }

        private string swEdgeFlangeError2String(swEdgeFlangeError_e nStatus)
        {

```

```

string functionReturnValue = null;
switch ((nStatus))
{
    case 0:
        functionReturnValue = "no error";
        break;
    case swEdgeFlangeError_e.swEdgeFlangeError_EdgeAlreadyExists:
        functionReturnValue = "edge already exists";
        break;
    case swEdgeFlangeError_e.swEdgeFlangeError_EdgeNotSpecified:
        functionReturnValue = "edge not specified";
        break;
    case swEdgeFlangeError_e.swEdgeFlangeError_NumberOfEdgesAndSketchesNotEqual:
        functionReturnValue = "number mismatch";
        break;
    case swEdgeFlangeError_e.swEdgeFlangeError_SketchNotSpecified:
        functionReturnValue = "sketch not specified";
        break;
    default:
        functionReturnValue = "unknown error";
        break;
}
return functionReturnValue;
}

public SldWorks swApp;
}
}

```

Remove Edge from Edge Flange Feature Example (VBA)

This example shows how to remove an edge from an edge flange feature in a sheet metal part.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\SM1-remove-edges.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Removes an edge from Edge-Flange1.
' 2. Examine the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp           As SldWorks.SldWorks
    Dim swDocument      As SldWorks.PartDoc
    Dim swSelectionManager As SldWorks.SelectionMgr
    Dim swEdgeFlangeData As SldWorks.EdgeFlangeFeatureData
    Dim swFeature        As SldWorks.Feature
    Dim lNumEdges        As Long
    Dim aEdgesToRemove() As SldWorks.Edge
    Dim lIdx             As Long
    Dim bStatus          As Boolean
    Dim lNumEdgesToRemove As Long
    Dim vEdgesToRemove   As Variant
    Dim vEdges           As Variant
    Dim nStatus          As SwConst.swEdgeFlangeError_e

    Set swApp = Application.SldWorks
    Set swDocument = swApp.ActiveDoc
    Set swSelectionManager = swDocument.SelectionManager

    bStatus = swDocument.Extension.SelectByID2("Edge-Flange1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelectionManager.GetSelectedObject6(1, 0)

    swDocument.ClearSelection2 True

    Set swEdgeFlangeData = swFeature.GetDefinition
    swEdgeFlangeData.AccessSelections swDocument, Nothing
    Debug.Print "Number of edges: " & swEdgeFlangeData.GetEdgeCount
    vEdges = swEdgeFlangeData.Edges
    lNumEdges = UBound(vEdges) - LBound(vEdges) + 1

    '
    ' Select number of edges to remove
    '
    Select Case (1)
        Case 1
            ' Remove one edge
            lNumEdgesToRemove = 1

        Case 2
            ' Remove all edges
            lNumEdgesToRemove = lNumEdges

        Case 3
            ' Remove no edges
            lNumEdgesToRemove = 0

    End Select

    vEdgesToRemove = Empty

    If (lNumEdgesToRemove >= 1) Then
        ReDim aEdgesToRemove(lNumEdgesToRemove - 1)
        For lIdx = 0 To (lNumEdgesToRemove - 1)
            Set aEdgesToRemove(lIdx) = vEdges(lIdx)
        Next lIdx
        vEdgesToRemove = aEdgesToRemove
    End If

    nStatus = swEdgeFlangeData.RemoveEdges(vEdgesToRemove)
    Debug.Print "Edge removal error code = " & swEdgeFlangeError2String(nStatus)

    bStatus = swFeature.ModifyDefinition(swEdgeFlangeData, swDocument, Nothing)
    Debug.Print "Status of modify definition = " & bStatus

    swEdgeFlangeData.AccessSelections swDocument, Nothing

    vEdges = Empty
    vEdges = swEdgeFlangeData.Edges

    If (Not IsEmpty(vEdges)) Then
        lNumEdges = UBound(vEdges) - LBound(vEdges) + 1
        Debug.Print "Number of edges in edge flange = " & lNumEdges
    Else
        Debug.Print "Number of edges in edge flange = 0"
    End If
End Sub

```

swEdgeFlangeData.**ReleaseSelectionAccess**

End Sub

Private Function swEdgeFlangeError2String(ByVal nStatus As SwConst.swEdgeFlangeError_e) As String

```
Select Case (nStatus)
    Case 0
        swEdgeFlangeError2String = "no error"
    Case swEdgeFlangeError_e.swEdgeFlangeError_EdgeAlreadyExists
        swEdgeFlangeError2String = "edge already exists"
    Case swEdgeFlangeError_e.swEdgeFlangeError_EdgeNotSpecified
        swEdgeFlangeError2String = "edge not specified"
    Case swEdgeFlangeError_e.swEdgeFlangeError_NumberOfEdgesAndSketchesNotEqual
        swEdgeFlangeError2String = "number mismatch"
    Case swEdgeFlangeError_e.swEdgeFlangeError_SketchNotSpecified
        swEdgeFlangeError2String = "sketch not specified"
    Case Else
        swEdgeFlangeError2String = "unknown error"
End Select
```

End Function

Remove Edge from Edge Flange Feature Example (VB.NET)

This example shows how to remove an edge from an edge flange feature in a sheet metal part.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\api\SM1-remove-edges.sldprt.
2. Open the Immediate window.
-----
Postconditions:
1. Removes an edge from Edge-Flange1.
2. Examine the Immediate window.
-----
NOTE: Because the model is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub Main()

        Dim swDocument As PartDoc
        Dim swSelectionManager As SelectionMgr
        Dim swEdgeFlangeData As EdgeFlangeFeatureData
        Dim swFeature As Feature
        Dim lNumEdges As Long
        Dim aEdgesToRemove() As Edge
        Dim lIdx As Long
        Dim bStatus As Boolean
        Dim lNumEdgesToRemove As Long
        Dim vEdgesToRemove As Object
        Dim vEdges As Object
        Dim nStatus As swEdgeFlangeError_e
        Dim modDocExt As ModelDocExtension

        swDocument = swApp.ActiveDoc
        swSelectionManager = swDocument.SelectionManager

        modDocExt = swDocument.Extension
        bStatus = modDocExt.SelectByID2("Edge-Flange1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelectionManager.GetSelectedObject6(1, 0)

        swDocument.ClearSelection2(True)

        swEdgeFlangeData = swFeature.GetDefinition
        swEdgeFlangeData.AccessSelections(swDocument, Nothing)
        vEdges = swEdgeFlangeData.Edges
        lNumEdges = UBound(vEdges) - LBound(vEdges) + 1
        Debug.Print("Number of edges in edge flange = " & swEdgeFlangeData.GetEdgeCount())
        ' Select number of edges to remove.
        lNumEdgesToRemove = 1

        vEdgesToRemove = Nothing

        If (lNumEdgesToRemove >= 1) Then
            ReDim aEdgesToRemove(lNumEdgesToRemove - 1)
            For lIdx = 0 To (lNumEdgesToRemove - 1)
                aEdgesToRemove(lIdx) = vEdges(lIdx)
            Next lIdx
            vEdgesToRemove = aEdgesToRemove
        End If

        nStatus = swEdgeFlangeData.RemoveEdges(vEdgesToRemove)
        Debug.Print("Edge removal error code = " & swEdgeFlangeError2String(nStatus))
        bStatus = swFeature.ModifyDefinition(swEdgeFlangeData, swDocument, Nothing)
        Debug.Print("Status of modify definition = " & bStatus)

        swEdgeFlangeData.AccessSelections(swDocument, Nothing)
        vEdges = Nothing

        vEdges = swEdgeFlangeData.Edges
        If (Not IsNothing(vEdges)) Then
            lNumEdges = UBound(vEdges) - LBound(vEdges) + 1
            Debug.Print("Number of edges in edge flange = " & lNumEdges)
        Else
            Debug.Print("Number of edges in edge flange = 0")
        End If
        swEdgeFlangeData.ReleaseSelectionAccess()

    End Sub

    Private Function swEdgeFlangeError2String(ByVal nStatus As swEdgeFlangeError_e) As String
        Select Case (nStatus)
            Case 0
                swEdgeFlangeError2String = "no error"
            Case swEdgeFlangeError_e.swEdgeFlangeError_EdgeAlreadyExists
                swEdgeFlangeError2String = "edge already exists"
            Case swEdgeFlangeError_e.swEdgeFlangeError_EdgeNotSpecified
                swEdgeFlangeError2String = "edge not specified"
            Case swEdgeFlangeError_e.swEdgeFlangeError_NumberOfEdgesAndSketchesNotEqual
                swEdgeFlangeError2String = "number mismatch"
            Case swEdgeFlangeError_e.swEdgeFlangeError_SketchNotSpecified
                swEdgeFlangeError2String = "sketch not specified"
            Case Else
                swEdgeFlangeError2String = "unknown error"
        End Select
    End Function

    Public swApp As SldWorks

```

End Class

Remove Hyperlink from Note in Drawing Example (VBA)

This example shows how to remove a hyperlink from a note in a drawing.

```
'-----
' Preconditions:
' 1. Verify that the specified drawing to open exists.
' 2. Verify that you have internet connection.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Opens the drawing.
' 2. Inserts a note containing a hyperlink to http://www.solidworks.com.
' 3. At Stop, click the note in the drawing, which opens the
'    SOLIDWORKS home web page.
' 4. Close the web page, close the Note PropertyManager page, and
'    click Continue (F5) in the IDE.
' 5. Sets the hyperlink to non-hyperlinked text.
' 6. Click the note in the drawing. Although the text in the note still
'    appears as http://www.solidworks.com, the note is no longer hyperlinked.
' 7. Close the Note PropertyManager page.
' 8. Examine the Immediate window.
'
' NOTES:
' * You cannot remove a hyperlink from a note
'   in a drawing using the user interface; you must use
'   the SOLIDWORKS API.
' * Because the drawing is used elsewhere, do not save changes.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swNote As SldWorks.Note
    Dim swAnnotation As SldWorks.Annotation
    Dim swTextFormat As SldWorks.TextFormat
    Dim boolstatus As Boolean
    Dim longstatus As Long
    Dim longwarnings As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem20.slddrw", 3, 0, "", longstatus,
longwarnings)
    Set swNote = swModel.InsertNote("http://www.solidworks.com")
    If Not swNote Is Nothing Then
        swNote.LockPosition = False
        swNote.Angle = 0
        boolstatus = swNote.SetBalloon(0, 0)
        Set swAnnotation = swNote.GetAnnotation()
        If Not swAnnotation Is Nothing Then
            longstatus = swAnnotation.SetLeader3(swLeaderStyle.e.swNO_LEADER, 0, True, False, False, False)
            boolstatus = swAnnotation.SetPosition(2.99923871594043E-02, 0.130309614693878, 0)
            boolstatus = swAnnotation.SetTextFormat(0, True, swTextFormat)
        End If
    End If

    Stop
    ' Click the note, close the SOLIDWORKS home web
    ' page, close the Note PropertyManager page,
    ' and click Continue (F5) in the IDE

    swModel.ClearSelection2 True

    swModel.WindowRedraw

    Debug.Print "Note = " & swNote.GetName
    Debug.Print "Tag name = " & swNote.TagName
    Debug.Print "Hyperlink = " & swNote.GetHyperlinkText
    Debug.Print ""

    boolstatus = swNote.SetHyperlinkText("No longer a hyperlink")
    If boolstatus Then
        Debug.Print "Hyperlink set to non-hyperlinked text!"
    Else
        Debug.Print "Hyperlink not set to non-hyperlinked text!"
    End If

End Sub
```

Remove Material Properties from Bodies Example (C#)

This example shows how to remove materials from bodies in a multibody part. This example also works with a part with a single body and an assembly with single and multibody components.

```
//-----
// Preconditions:
// 1. Open public documents\samples\tutorial\multibody\multi_inter.sldprt.
// 2. Expand Solid Bodies(2) > right-click Extrude-Thin1 > Click
//    Appearances > Body > any color in Appearances(color) > OK.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the FeatureManager design tree.
// 2. Removes the color that you applied to Extrude-Thin1.
// 3. Examine the Immediate window and graphics area.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        PartDoc swPart;
        object vBody;
        bool bRet;

        public void Main()
        {
            int j = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModel.ClearSelection2(true);
            Debug.Print("File = " + swModel.GetPathName());
            Switch (swModel.GetType())
            {
                case (int)swDocumentTypes_e.swDocPART:
                    swPart = (PartDoc)swModel;
                    // Solid bodies
                    object[] vBodyArr = null;
                    Body2 swBody = default(Body2);
                    vBodyArr = (object[])swPart.GetBodies2((int)swBodyType_e.swSolidBody, true);
                    if ((vBodyArr != null))
                    {
                        Debug.Print("    Number of solid bodies: " + vBodyArr.Length);
                        Debug.Print("    Material removed from: ");
                        j = 1;
                        foreach (object vBody_loopVariable in vBodyArr)
                        {
                            vBody = vBody_loopVariable;
                            swBody = (Body2)vBody;
                            string[] vConfigName = null;
                            vConfigName = (string[])swModel.GetConfigurationNames();
                            bRet = swBody.RemoveMaterialProperty((int)swInConfigurationOpts_e.swAllConfiguration, (vConfigName));
                            Debug.Print("    Body " + j + "? " + bRet);
                            j = j + 1;
                        }
                    }
                    break;
                case (int)swDocumentTypes_e.swDocASSEMBLY:
                    ProcessAssembly(swApp, swModel);
                    break;
                default:
                    return;
                    break;
            }
        }

        public void ProcessAssembly(SldWorks swApp, ModelDoc2 swModel)
        {
            ConfigurationManager swConfigMgr = default(ConfigurationManager);
            Configuration swConf = default(Configuration);
            Component2 swRootComp = default(Component2);

            swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConf = (Configuration)swConfigMgr.ActiveConfiguration;
            swRootComp = (Component2)swConf.GetRootComponent3(true);
            ProcessComponent(swApp, swModel, swRootComp);
        }

        public void ProcessComponent(SldWorks swApp, ModelDoc2 swModel, Component2 swComp)
        {
            object[] vChildComp = null;
            Component2 swChildComp = null;
            object childComp = null;

            Debug.Print(swComp.Name2 + " <" + swComp.ReferencedConfiguration + ">");
            // Solid bodies
            object[] vBodyArr = null;
            Body2 swBody = default(Body2);
            vBodyArr = (object[])swComp.GetBodies2((int)swBodyType_e.swSolidBody);
            if ((vBodyArr != null))
            {
                Debug.Print("    Number of bodies: " + vBodyArr.Length);
                foreach (object vBody_loopVariable in vBodyArr)
                {
                    vBody = vBody_loopVariable;
                    swBody = (Body2)vBody;
                    string[] vConfigName = null;
                    Debug.Print("    Body name: " + swBody.Name);
                    vConfigName = (string[])swModel.GetConfigurationNames();
                }
            }
        }
    }
}
```

```

        bRet = swBody.RemoveMaterialProperty((int)swInConfigurationOpts_e.swThisConfiguration, (vConfigName));
        Debug.Print("Material removed from body? " + bRet);
    }
}
vChildComp = (object[])swComp.GetChildren();
foreach (object childComp_loopVariable in vChildComp)
{
    childComp = (object)childComp_loopVariable;
    swChildComp = (Component2)childComp;
    ProcessComponent(swApp, swModel, swChildComp);
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Remove Material Properties from Bodies Example (VBA)

This example shows how to remove materials from bodies in a multibody part. This example also works with a part with a single body and an assembly with single and multibody components.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_inter.sldprt.
' 2. Expand Solid Bodies(2) > right-click Extrude-Thin1 > click
'    Appearances > Body > any color in Appearances(color) > OK.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the FeatureManager design tree.
' 2. Removes the color that you applied to Extrude-Thin1.
' 3. Examine the Immediate window and graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim bRet As Boolean

Sub ProcessComponent(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2)
    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim vBody As Variant
    Dim childComp As Variant

    Debug.Print swComp.Name2 & " <" & swComp.ReferencedConfiguration & ">"

    ' Solid bodies
    Dim vBodyArr As Variant
    Dim swBody As Body2
    vBodyArr = swComp.GetBodies2(swSolidBody)
    If Not IsEmpty(vBodyArr) Then
        Debug.Print "  Number of bodies: " & UBound(vBodyArr) + 1
        For Each vBody In vBodyArr
            Set swBody = vBody
            Dim vConfigName As Variant
            Debug.Print "    Body name: " & swBody.Name
            vConfigName = swModel.GetConfigurationNames
            bRet = swBody.RemoveMaterialProperty(swThisConfiguration, (vConfigName))
            Debug.Print "      Material removed from body? " & bRet
        Next
    End If

    vChildComp = swComp.GetChildren
    For Each childComp In vChildComp
        Set swChildComp = childComp
        ProcessComponent swApp, swModel, swChildComp
    Next
End Sub

Sub ProcessAssembly(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2)
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConf As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2

    Set swConfigMgr = swModel.ConfigurationManager
    Set swConf = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConf.GetRootComponent3(True)
    ProcessComponent swApp, swModel, swRootComp
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim vBody As Variant
    Dim j As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    swModel.ClearSelection2 True

    Debug.Print "File = " & swModel.GetPathName

    Select Case swModel.GetType
        Case swDocPART
            Set swPart = swModel
            ' Solid bodies
            Dim vBodyArr As Variant
            Dim swBody As Body2
            vBodyArr = swPart.GetBodies2(swSolidBody, True)

            If Not IsEmpty(vBodyArr) Then
                Debug.Print "  Number of bodies: " & UBound(vBodyArr) + 1
                Debug.Print "    Material removed from: "
                j = 1
                For Each vBody In vBodyArr
                    Set swBody = vBody
                    Dim vConfigName As Variant
                    vConfigName = swModel.GetConfigurationNames
                    bRet = swBody.RemoveMaterialProperty(swAllConfiguration, (vConfigName))
                Next
            End If
        End Select
    End Sub
```

```
        Debug.Print "        Body " & j & "? " & bRet
        j = j + 1
    Next
End If
Case swDocASSEMBLY
    ProcessAssembly swApp, swModel
Case Else
    Exit Sub
End Select
End Sub
```

Remove Material Properties from Bodies Example (VB.NET)

This example shows how to remove materials from bodies in a multibody part. This example also works with a part with a single body and an assembly with single and multibody components.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_inter.sldprt.
' 2. Expand Solid Bodies(2) > right-click Extrude-Thin1 > click
'    Appearances > Body > any color in Appearances(color) > OK.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the FeatureManager design tree.
' 2. Removes the color that you applied to Extrude-Thin1.
' 3. Examine the Immediate window and graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swPart As PartDoc
    Dim vBody As Object
    Dim bRet As Boolean

    Public Sub main()

        Dim j As Integer

        swModel = swApp.ActiveDoc
        swModel.ClearSelection2(True)
        Debug.Print("File = " & swModel.GetPathName)
        Select Case swModel.GetType
            Case swDocumentTypes_e.swDocPART
                swPart = swModel
                ' Solid bodies
                Dim vBodyArr() As Object
                Dim swBody As Body2
                vBodyArr = swPart.GetBodies2(swBodyType_e.swSolidBody, True)
                If Not IsNothing(vBodyArr) Then
                    Debug.Print("    Number of solid bodies: " & UBound(vBodyArr) + 1)
                    Debug.Print("    Material removed from: ")
                    j = 1
                    For Each vBody In vBodyArr
                        swBody = vBody
                        Dim vConfigName() As Object
                        vConfigName = swModel.GetConfigurationNames
                        bRet = swBody.RemoveMaterialProperty(swInConfigurationOpts_e.swAllConfiguration, (vConfigName))
                        Debug.Print("    Body " & j & "?" & bRet)
                        j = j + 1
                    Next
                End If
            Case swDocumentTypes_e.swDocASSEMBLY
                ProcessAssembly(swApp, swModel)
            Case Else
                Exit Sub
        End Select

    End Sub

    Sub ProcessAssembly(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2)
        Dim swConfigMgr As ConfigurationManager
        Dim swConf As Configuration
        Dim swRootComp As Component2

        swConfigMgr = swModel.ConfigurationManager
        swConf = swConfigMgr.ActiveConfiguration
        swRootComp = swConf.GetRootComponent3(True)
        ProcessComponent(swApp, swModel, swRootComp)

    End Sub

    Sub ProcessComponent(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swComp As Component2)
        Dim vChildComp() As Object
        Dim swChildComp As Component2 = Nothing
        Dim childComp As Object

        Debug.Print(swComp.Name2 & " <" & swComp.ReferencedConfiguration & ">")
        ' Solid bodies
        Dim vBodyArr() As Object
        Dim swBody As Body2
        vBodyArr = swComp.GetBodies2(swBodyType_e.swSolidBody)
        If Not IsNothing(vBodyArr) Then
            Debug.Print("    Number of bodies: " & UBound(vBodyArr) + 1)
            For Each vBody In vBodyArr
                swBody = vBody
                Dim vConfigName() As Object
                Debug.Print("    Body name: " & swBody.Name)
                vConfigName = swModel.GetConfigurationNames
                bRet = swBody.RemoveMaterialProperty(swInConfigurationOpts_e.swThisConfiguration, (vConfigName))
                Debug.Print("    Material removed from body? " & bRet)
            Next
        End If
        vChildComp = swComp.GetChildren
        For Each childComp In vChildComp
            swChildComp = childComp
            ProcessComponent(swApp, swModel, swChildComp)
        Next
    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.

```

```
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Remove Material Properties from Assembly Component Example (VBA)

This example shows how to remove the material properties from the selected assembly component.

```
'-----
'
' Preconditions:
'   (1) Assembly is open.
'   (2) Component with changed material properties (for example, color)
'       is selected.
'
' Postconditions: Material properties from selected component are removed.
'
'-----

Option Explicit

Dim swApp           As SldWorks.SldWorks
Dim swModel         As SldWorks.ModelDoc2
Dim swSelMgr        As SldWorks.SelectionMgr
Dim swComp          As SldWorks.Component2
Dim swConfig        As SldWorks.Configuration
Dim vConfigName     As Variant
Dim bRet            As Boolean

Sub main()

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swComp = swSelMgr.GetSelectedObjectsComponent2(1)
    ' Get the names of the configurations in the assembly
    vConfigName = swModel.GetConfigurationNames

    ' Remove the material properties from the selected
    ' component in this configuration in this assembly
    bRet = swComp.RemoveMaterialProperty2(swThisConfiguration, (vConfigName))
    Debug.Print (" Material removed: "); bRet

    swModel.GraphicsRedraw2
End Sub

'-----
```

Remove Menu Commands, Menus, and Toolbar Buttons Example (VBA)

This example shows how to:

- retrieve the name of the parent menu.
- remove a toolbar button.
- remove a command from the main frame and context-sensitive menus.

```
'-----  
' Preconditions:  
' 1. Open a part document.  
' 2. Verify that the Filter Edges button appears on the  
'    Selection Filter toolbar.  
'  
' Postconditions:  
' 1. Removes the Filter Edges button on the Selection  
'    Filter toolbar for this SOLIDWORKS Design session.  
' 2. Removes the Suppress command from the main frame and  
'    context-sensitive menus for this SOLIDWORKS Design session.  
' 3. Examine the Selection Filter toolbar to verify step 1.  
' 4. Click or right-click a feature to verify step 2.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim parentName As String  
Dim name As String  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
  
    ' Get the name of the parent menu for the first occurrence  
    ' of the Suppress command in part documents  
    name = swApp.GetMenuStrings(swCommands_Suppress, 1, parentName)  
  
    ' Remove the Filter Edges button from the Selection Filter toolbar  
    ' in part documents  
    swApp.RemoveFromMenu swCommands_FilterEdges, 1, 3, True  
  
    ' Remove the Suppress command from all main frame menus in part documents; do  
    ' not remove the parent menus  
    swApp.RemoveFromMenu swCommands_Suppress, 1, 3, False  
  
    ' Remove the Suppress command from all context-sensitive menus; do not  
    ' remove the parent menus  
    swApp.RemoveFromPopupMenu swCommands_Suppress, 1, swSelEVERYTHING, False  
  
End Sub
```

Remove Textures from Assembly Components Example (VBA)

This example shows how to remove all textures from all components in an assembly.

```
'-----
' Preconditions:
' 1. Open an assembly with one or more components
'    with textures applied to them at the component (part)
'    level.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Removes all textures applied to all components.
' 2. Examine the Immediate window.
' 3. Click the graphics area and examine the assembly.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim boolstatus As Boolean

Function RemoveTexture(swDoc As SldWorks.ModelDoc2, configName As String) As Boolean

    Dim swDocExt As SldWorks.ModelDocExtension
    Dim swTexture As SldWorks.Texture

    Set swDocExt = swDoc.Extension
    Set swTexture = swDocExt.GetTexture(configName)
    If Not swTexture Is Nothing Then
        Debug.Print "    Texture removed: " & swTexture.MaterialName
        RemoveTexture = swDocExt.RemoveTexture2(configName)
        swDoc.SetSaveFlag
    End If
End Function

Function TraverseComponents(parentComp As SldWorks.Component2)

    Dim vChildComponents As Variant
    Dim vObj As Variant
    Dim childComp As SldWorks.Component2
    Dim childDoc As SldWorks.ModelDoc2
    Dim childConfigName As String

    vChildComponents = parentComp.GetChildren
    For Each vObj In vChildComponents
        Set childComp = vObj
        Set childDoc = childComp.GetModelDoc
        childConfigName = childComp.ReferencedConfiguration
        Debug.Print "Component name: " & childComp.Name2
        Debug.Print "    Configuration name: " & childConfigName
        boolstatus = RemoveTexture(childDoc, childConfigName)
        Call TraverseComponents(childComp)
    Next vObj
End Function

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Dim rootDoc As SldWorks.ModelDoc2
    Dim rootConfig As SldWorks.Configuration
    Dim rootComp As SldWorks.Component2
    Dim configMgr As SldWorks.ConfigurationManager

    Set rootDoc = swApp.ActiveDoc
    Set configMgr = Part.ConfigurationManager
    Set rootConfig = configMgr.ActiveConfiguration
    Set rootComp = rootConfig.GetRootComponent3(True)

    Call TraverseComponents(rootComp)
End Sub
```

Rename Active Sketch Example (VBA)

This example shows how to rename the active sketch.

```
'-----  
' Preconditions: Open the sketch that you want to rename.  
'  
' Postconditions:  
' 1. Changes the name of the open sketch to MySketch.  
' 2. Examine the FeatureManager design tree.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSketch As SldWorks.Sketch  
    Dim swFeat As SldWorks.Feature  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    ' Get the open sketch  
    Set swSketch = swModel.GetActiveSketch2  
    Set swFeat = swSketch  
  
    ' Change the name of the open sketch to MySketch  
    swFeat.Name = "MySketch"  
  
End Sub
```

Rename Assembly Components Example (VBA)

This example shows how to rename assembly components.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Appends 123 to the end of each assembly component's
'    name.
' 2. Examine the FeatureManager design tree and the
'    Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConfig As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2
    Dim Children As Variant
    Dim swChild As SldWorks.Component2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim ChildCount As Long
    Dim OldName As String
    Dim NewName As String
    Dim bOldSetting As Boolean
    Dim bRet As Boolean
    Dim i As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConfig.GetRootComponent3(True)

    bOldSetting = swApp.GetUserPreferenceToggle(swExtRefUpdateCompNames)

    swApp.SetUserPreferenceToggle swExtRefUpdateCompNames, False

    Children = swRootComp.GetChildren
    ChildCount = UBound(Children)
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    For i = 0 To ChildCount
        Set swChild = Children(i)
        ' Changing component name requires component to be selected
        bRet = swChild.Select4(False, swSelData, False)
        OldName = swChild.Name2
        NewName = OldName & " 123"
        Debug.Print "Old name = " & OldName
        Debug.Print "New name = " & NewName
        Debug.Print ""
        swChild.Name2 = NewName
    Next i

    swApp.SetUserPreferenceToggle swExtRefUpdateCompNames, bOldSetting

End Sub
```

Rename Component and Update References Example (C#)

This example shows how to rename a component and update its references.

```
//-----
// Preconditions:
// 1. Copy public_documents\samples\tutorial\EDraw\claw to c:\test\claw.
// 2. Open c:\test\claw\claw-mechanism.sldasm and save the file as
//    claw-mechanism-copy.sldasm.
// 3. Close claw-mechanism-copy.sldasm and reopen claw-mechanism.sldasm.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Renames the center component to centerXXX.
// 2. Fires the RenameItemNotify event.
// 3. Saves the assembly.
// 4. Fires the RenamedDocumentNotify event.
// 5. Updates references.
// 6. Examine the FeatureManager design tree and Immediate window.
// 7. Close claw-mechanism.sldasm and open
//    c:\test\claw\claw-mechanism-copy.sldasm to verify that the
//    center component was renamed to centerXXX.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public AssemblyDoc assy;

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssy = default(AssemblyDoc);
            Hashtable openAssembly = default(Hashtable);
            int errors = 0;
            int warnings = 0;
            bool status = false;

            swAssy = (AssemblyDoc)swApp.ActiveDoc;

            //Set up event
            assy = (AssemblyDoc)swAssy;
            openAssembly = new Hashtable();
            AttachEventHandlers();

            swModel = (ModelDoc2)swAssy;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("center-1@claw-mechanism", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            errors = swModelDocExt.RenameDocument("centerXXX");
            swModelDocExt.Rebuild((int)swRebuildOptions_e.swRebuildAll);
            status = swModel.Save3((int)swSaveAsOptions_e.swSaveAsOptions_Silent + (int)swSaveAsOptions_e.swSaveAsOptions_SaveReferenced, ref errors, ref
warnings);
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            if ((assy != null))
            {
                assy.RenameItemNotify += this.assy_RenameItemNotify;
                assy.RenamedDocumentNotify += this.assy_RenamedDocumentNotify;
            }
        }

        //Fire notification when item is renamed
        public int assy_RenameItemNotify(int entType, string oldName, string newName)
        {
            Debug.Print("RenameItemNotify fired");
            return 0;
        }

        //Fire notification for Rename Documents dialog
        public int assy_RenamedDocumentNotify(ref object swObj)
        {
            RenamedDocumentReferences swRenamedDocumentReferences = default(RenamedDocumentReferences);
            object[] searchPaths = null;
            object[] pathNames = null;
            int i = 0;
            int nbr = 0;

            swRenamedDocumentReferences = (RenamedDocumentReferences)swObj;

            swRenamedDocumentReferences.UpdateWhereUsedReferences = true;
            swRenamedDocumentReferences.IncludeFileLocations = true;

            searchPaths = (object[])swRenamedDocumentReferences.GetSearchPath();
            nbr = searchPaths.Length - 1;
            Debug.Print("Search paths:");
            for (i = 0; i <= nbr; i++)
            {
                Debug.Print(" " + searchPaths[i]);
            }

            swRenamedDocumentReferences.Search();
        }
    }
}
```

```

pathNames = (object[])swRenamedDocumentReferences.ReferencesArray();
nbr = pathNames.Length - 1;
Debug.Print("References:");
for (i = 0; i <= nbr; i++)
{
    Debug.Print(" " + pathNames[i]);
}

swRenamedDocumentReferences.CompletionAction = (int)swRenamedDocumentFinalAction_e.swRenamedDocumentFinalAction_0k;
Debug.Print("RenamedDocumentNotify fired");

return 0;
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Rename Component and Update References Example (VBA)

This example shows how to rename a component and update its references.

```

'-----
' Preconditions:
' 1. Copy and paste Main into your project.
' 2. Insert a class module and copy and paste Class1 into that module.
' 3. Copy public_documents\samples\tutorial\EDraw\claw to c:\test\claw.
' 4. Open c:\test\claw\claw-mechanism.sldasm and save the file as
'    claw-mechanism-copy.sldasm.
' 5. Close claw-mechanism-copy.sldasm and reopen claw-mechanism.sldasm.
' 6. Open the Immediate window.
'
' Postconditions:
' 1. Renames the center component to centerXXX.
' 2. Fires the RenameItemNotify event.
' 3. Saves the assembly.
' 4. Fires the RenamedDocumentNotify event.
' 5. Updates references.
' 6. Examine the FeatureManager design tree and Immediate window.
' 7. Close claw-mechanism.sldasm and open
'    c:\test\claw\claw-mechanism-copy.sldasm to verify that the
'    center component was renamed to centerXXX.
'-----
'Main
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssy As SldWorks.AssemblyDoc
Dim swAssyEvents As Class1
Dim errors As Long
Dim warnings As Long
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swAssy = swApp.ActiveDoc

    'Set up event
    Set swAssyEvents = New Class1
    Set swAssyEvents.swAssy = swApp.ActiveDoc

    Set swModel = swAssy
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("center-1@claw-mechanism", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    errors = swModelDocExt.RenameDocument("centerXXX")
    swModelDocExt.Rebuild swRebuildOptions_e.swRebuildAll
    status = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent + swSaveAsOptions_e.swSaveAsOptions_SaveReferenced, errors, warnings)

End Sub

'Class1
Option Explicit

Public WithEvents swAssy As SldWorks.AssemblyDoc

'Fire notification when item is renamed
Public Function swAssy_RenameItemNotify(ByVal entType As Long, ByVal oldName As String, ByVal newName As String) As Long
    Debug.Print "RenameItemNotify fired"
End Function

'Fire notification for Rename Documents dialog
Public Function swAssy_RenamedDocumentNotify(ByRef swObj As Object) As Long
    Dim swRenamedDocumentReferences As SldWorks.RenamedDocumentReferences
    Dim searchPaths As Variant
    Dim pathNames As Variant
    Dim i As Long
    Dim nbr As Long

    Set swRenamedDocumentReferences = swObj

    swRenamedDocumentReferences.UpdateWhereUsedReferences = True
    swRenamedDocumentReferences.IncludeFileLocations = True

    searchPaths = swRenamedDocumentReferences.GetSearchPath
    nbr = UBound(searchPaths)
    Debug.Print "Search paths:"
    For i = 0 To nbr
        Debug.Print (" " & searchPaths(i))
    Next i

    swRenamedDocumentReferences.Search

    pathNames = swRenamedDocumentReferences.ReferencesArray
    nbr = UBound(pathNames)
    Debug.Print "References:"
    For i = 0 To nbr
        Debug.Print (" " & pathNames(i))
    
```

```
Next i
```

```
swRenamedDocumentReferences.CompletionAction = swRenamedDocumentFinalAction_e.swRenamedDocumentFinalAction_0k
```

```
Debug.Print "RenamedDocumentNotify fired"
```

```
End Function
```

Rename Component and Update References Example (VB.NET)

This example shows how to rename a component and update its references.

```

-----
Preconditions:
1. Copy public_documents\samples\tutorial\EDraw\claw to c:\test\claw.
2. Open c:\test\claw\claw-mechanism.sldasm and save the file as
   claw-mechanism-copy.sldasm.
3. Close claw-mechanism-copy.sldasm and reopen claw-mechanism.sldasm.
4. Open the Immediate window.
Postconditions:
1. Renames the center component to centerXXX.
2. Fires the RenameItemNotify event.
3. Saves the assembly.
4. Fires the RenamedDocumentNotify event.
5. Updates references.
6. Examine the FeatureManager design tree and Immediate window.
7. Close claw-mechanism.sldasm and open
   c:\test\claw\claw-mechanism-copy.sldasm to verify that the
   center component was renamed to centerXXX.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
Imports System.Collections

Partial Class SolidWorksMacro

    Public WithEvents assy As AssemblyDoc

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssy As AssemblyDoc
        Dim openAssembly As Hashtable
        Dim errors As Integer
        Dim warnings As Integer
        Dim status As Boolean

        swAssy = swApp.ActiveDoc

        'Set up event
        assy = swAssy
        openAssembly = New Hashtable
        AttachEventHandlers()

        swModel = swAssy
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("center-1@claw-mechanism", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        errors = swModelDocExt.RenameDocument("centerXXX")
        swModelDocExt.Rebuild(swRebuildOptions_e.swRebuildAll)
        status = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent + swSaveAsOptions_e.swSaveAsOptions_SaveReferenced, errors, warnings)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        If Not assy Is Nothing Then
            AddHandler assy.RenameItemNotify, AddressOf Me.assy_RenameItemNotify
            AddHandler assy.RenamedDocumentNotify, AddressOf Me.assy_RenamedDocumentNotify
        End If
    End Sub

    'Fire notification when item is renamed
    Public Function assy_RenameItemNotify(ByVal entType As Integer, ByVal oldName As String, ByVal newName As String) As Integer
        Debug.Print("RenameItemNotify fired")
    End Function

    'Fire notification for Rename Documents dialog
    Public Function assy_RenamedDocumentNotify(ByRef swObj As Object) As Integer
        Dim swRenamedDocumentReferences As RenamedDocumentReferences
        Dim searchPaths As Object
        Dim pathNames As Object
        Dim i As Integer
        Dim nbr As Integer

        swRenamedDocumentReferences = swObj

        swRenamedDocumentReferences.UpdateWhereUsedReferences = True
        swRenamedDocumentReferences.IncludeFileLocations = True

        searchPaths = swRenamedDocumentReferences.GetSearchPath
        nbr = UBound(searchPaths)
        Debug.Print("Search paths:")
        For i = 0 To nbr
            Debug.Print(" " & searchPaths(i))
        Next i

        swRenamedDocumentReferences.Search()

        pathNames = swRenamedDocumentReferences.ReferencesArray
        nbr = UBound(pathNames)
        Debug.Print("References:")
        For i = 0 To nbr
            Debug.Print(" " & pathNames(i))
        Next i
    End Function
End Class

```

```
Next i

swRenamedDocumentReferences.CompletionAction = swRenamedDocumentFinalAction_e.swRenamedDocumentFinalAction_Ok

Debug.Print("RenamedDocumentNotify fired")

End Function

End Class
```

Rename Components and Save Assembly Example (C#)

This example shows how to rename components in an assembly and returns an error when attempting to save the assembly without first saving its references.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly.
// 2. Selects a component.
// 3. Renames the selected component and the other component with the
//    the same name.
// 4. Attempts to save the assembly.
// 5. Gets whether the assembly has renamed components.
// 6. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

Namespace Macro1CSharp.csproj
{
    Partial Public Class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            int errorsRename = 0;
            int errorsSave = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\beam_boltconnection.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
rrors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            status = swModelDocExt.SelectByID2("beam with holes-2@beam_boltconnection", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            errorsRename = swModelDocExt.RenameDocument("Renamed_beam_with_holes");
            Debug.Print("Rename document errors: " + errorsRename);
            status = swModel.Save3((int)swSaveAsOptions_e.swSaveAsOptions_Silent, ref errorsSave, ref warnings);
            if (status == false)
            {
                Debug.Print("Save errors (8192 = Saving an assembly with renamed components requires saving the references): " + errorsSave);
            }
            status = swModelDocExt.HasRenamedDocuments();
            Debug.Print("Assembly document has renamed components: " + status);

            swModel.ClearSelection2(true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Rename Components and Save Assembly Example (VBA)

This example shows how to rename components in an assembly and returns an error when attempting to save the assembly without first saving its references.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly.
' 2. Selects a component.
' 3. Renames the selected component and the other component with the
'    the same name.
' 4. Attempts to save the assembly.
' 5. Gets whether the assembly has renamed components.
' 6. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim errorsRename As Long
Dim errorsSave As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\beam_boltconnection.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    status = swModelDocExt.SelectByID2("beam with holes-2@beam_boltconnection", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    errorsRename = swModelDocExt.RenameDocument("Renamed_beam_with_holes")
    Debug.Print "Rename document errors: " & errorsRename
    status = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent, errorsSave, warnings)
    If status = False Then
        Debug.Print "Save errors (8192 = Saving an assembly with renamed components requires saving the references): " & errorsSave
    End If
    status = swModelDocExt.HasRenamedDocuments
    Debug.Print "Assembly document has renamed components: " & status

    swModel.ClearSelection2 True
End Sub
```

Rename Components and Save Assembly Example (VB.NET)

This example shows how to rename components in an assembly and returns an error when attempting to save the assembly without first saving its references.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Opens the specified assembly.
' 2. Selects a component.
' 3. Renames the selected component and the other component with the
'    the same name.
' 4. Attempts to save the assembly.
' 5. Gets whether the assembly has renamed components.
' 6. Examine the Immediate window.
'-----
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim errorsRename As Integer
        Dim errorsSave As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\beam_boltconnection.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        status = swModelDocExt.SelectByID("beam with holes-2@beam_boltconnection", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        errorsRename = swModelDocExt.RenameDocument("Renamed_beam_with_holes")
        Debug.Print("Rename document errors: " & errorsRename)
        status = swModel.Save3(swSaveAsOptions_e.swSaveAsOptions_Silent, errorsSave, warnings)
        If status = False Then
            Debug.Print("Save errors (8192 = Saving an assembly with renamed components requires saving the references): " & errorsSave)
        End If
        status = swModelDocExt.HasRenamedDocuments
        Debug.Print("Assembly document has renamed components: " & status)

        swModel.ClearSelection2(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Render Annotations to Separate Image File Example (C#)

This example shows how to render annotations and dimensions visible in a model to a separate image file.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Add a note to the model.
// 3. Right-click Annotations in the FeatureManager design tree
// and click Show Feature Dimensions, if dimensions
// are not visible in the model.
// 4. Verify that:
// * ray-trace rendering engine is loaded.
// * c:\temp exists.
//
// Preconditions:
// 1. Sets the options to include and render annotations and dimensions
// visible in a model to a separate image file.
// 2. If prompted to add camera or perspective, click Continue
// without Camera or Perspective.
// 3. Renders:
// * model to c:\temp\annotations.png
// * note and dimensions to c:\temp\annotations_1.png
// 4. Open both files to verify the previous step.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            RayTraceRenderer swRayTraceRenderer = default(RayTraceRenderer);
            RayTraceRendererOptions swRayTraceRenderOptions = default(RayTraceRenderOptions);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Access the ray-trace rendering engine
            //swRayTraceRenderer = (RayTraceRenderer)swApp.GetRayTraceRenderer();

            //Set options to include and render annotations and dimensions
            //visible in a model to a separate image file
            swRayTraceRenderOptions = (RayTraceRenderOptions)swRayTraceRenderer.RayTraceRenderOptions;
            swRayTraceRenderOptions.IncludeAnnotationsInRendering = true;
            swRayTraceRenderOptions.RenderAnnotationsToSeparateImage = true;
            status = swRayTraceRenderer.RenderToFile("c:\\temp\\annotations.png", 0, 0);
            status = swRayTraceRenderer.CloseFinalRenderWindow();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Render Annotations to Separate Image File Example (VBA)

This example shows how to render annotations and dimensions visible in a model to a separate image file.

```
'-----  
' Preconditions:  
' 1. Open a model document.  
' 2. Add a note to the model.  
' 3. Right-click Annotations in the FeatureManager design tree  
'    and click Show Feature Dimensions, if dimensions  
'    are not visible in the model.  
' 4. Verify that:  
'    * a ray-trace rendering engine is loaded.  
'    * c:\temp exists.  
'  
' Preconditions:  
' 1. Sets the options to include and render annotations and dimensions  
'    visible in a model to a separate image file.  
' 2. If prompted to add camera or perspective, click Continue  
'    without Camera or Perspective.  
' 3. Renders:  
'    * model to c:\temp\annotations.png  
'    * note and dimensions to c:\temp\annotations_1.png  
' 4. Open both files to verify the previous step.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swRayTraceRenderer As SldWorks.RayTraceRenderer  
    Dim swRayTraceRenderOptions As SldWorks.RayTraceRenderOptions  
    Dim status As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    'Access ray-trace rendering engine  
    Set swRayTraceRenderer = swApp.GetRayTraceRenderer()  
  
    'Set options to include and render annotations and dimensions  
    'visible in a model to a separate image file  
    Set swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRenderOptions  
    swRayTraceRenderOptions.IncludeAnnotationsInRendering = True  
    swRayTraceRenderOptions.RenderAnnotationsToSeparateImage = True  
    status = swRayTraceRenderer.RenderToFile("c:\temp\annotations.png", 0, 0)  
    status = swRayTraceRenderer.CloseFinalRenderWindow
```

End Sub

Render Annotations to Separate Image File Example (VB.NET)

This example shows how to render annotations and dimensions visible in a model to a separate image file.

```
-----
Preconditions:
1. Open a model document.
2. Add a note to the model.
3. Right-click Annotations in the FeatureManager design tree
   and click Show Feature Dimensions, if dimensions
   are not visible in the model.
4. Verify that:
   * a ray-trace rendering engine is loaded.
   * c:\temp exists.
-----
Preconditions:
1. Sets the options to include and render annotations and dimensions
   visible in a model to a separate image file.
2. If prompted to add camera or perspective, click Continue
   without Camera or Perspective.
3. Renders:
   * model to c:\temp\annotations.png
   * note and dimensions to c:\temp\annotations_1.png
4. Open both files to verify the previous step.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swRayTraceRenderer As RayTraceRenderer
        Dim swRayTraceRenderOptions As RayTraceRenderOptions
        Dim status As Boolean

        swModel = swApp.ActiveDoc

        'Access the ray-trace rendering engine
        swRayTraceRenderer = swApp.GetRayTraceRenderer()

        'Set options to include and render annotations and dimensions
        'visible in a model to a separate image file
        swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRenderOptions
        swRayTraceRenderOptions.IncludeAnnotationsInRendering = True
        swRayTraceRenderOptions.RenderAnnotationsToSeparateImage = True
        status = swRayTraceRenderer.RenderToFile("c:\temp\annotations.png", 0, 0)
        status = swRayTraceRenderer.CloseFinalRenderWindow

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Render Model (C#)

This example shows how to use a ray-trace rendering engine to render a model and save the rendered image in BMP and HDR formats. You must have a Premium license to run this example.

```
//-----
// Preconditions:
// 1. Verify that:
//    * specified part exists.
//    * c:\temp exists.
//    * a ray-trace rendering engine is loaded in SOLIDWORKS Design.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. If prompted to use perspective views in renderings, click
//    Continue without Camera or Perspective.
// 2. Changes the specified rendering options.
// 3. Creates c:\temp\Footer_1.bmp and c:\temp\Footer_2.hdr containing
//    rendered images of the part.
// 4. Examine the Immediate window and c:\temp.
//
// NOTE: Rendering can take several minutes to complete.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace RayTraceRendererCSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        RayTraceRenderer swRayTraceRenderer;
        RayTraceRendererOptions swRayTraceRenderOptions;
        int errors;
        int warnings;
        string fileName;

        bool status;

        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\routing-
pipes\\fittings\\filter.sldprt";
            swModel = swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, ""

            // Access the ray-trace rendering engine
            //swRayTraceRenderer = (RayTraceRenderer)swApp.GetRayTraceRenderer((int)swRayTraceRenderType_e.ray_trace_rendering_engin

            // Get and set rendering options
            swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRenderOptions;

            // Get current rendering values
            Debug.Print("Current rendering values");
            Debug.Print(" Image height = " + swRayTraceRenderOptions.ImageHeight);
            Debug.Print(" Image width = " + swRayTraceRenderOptions.ImageWidth);
            Debug.Print(" Image format = " + swRayTraceRenderOptions.ImageFormat);
            Debug.Print(" Preview render quality = " + swRayTraceRenderOptions.PreviewRenderQuality);
            Debug.Print(" Final render quality = " + swRayTraceRenderOptions.FinalRenderQuality);
            Debug.Print(" Bloom enabled? " + swRayTraceRenderOptions.BloomEnabled);
            Debug.Print(" Bloom threshold = " + swRayTraceRenderOptions.BloomThreshold);
            Debug.Print(" Bloom radius = " + swRayTraceRenderOptions.BloomRadius);
            Debug.Print(" Contour/cartoon rendering enabled? " + swRayTraceRenderOptions.ContourCartoonRenderingEnabled);
            Debug.Print(" Render type = " + swRayTraceRenderOptions.RenderType);
            Debug.Print(" Shaded contour? " + swRayTraceRenderOptions.ShadedContour);
            Debug.Print(" Contour line thickness = " + swRayTraceRenderOptions.ContourLineThickness);
            Debug.Print(" Contour line color = " + swRayTraceRenderOptions.ContourLineColor);
            Debug.Print(" Use SOLIDWORKS Design view aspect ratio? " + swRayTraceRenderOptions.UseSolidWorksViewAspectRatio);
            Debug.Print(" Alpha output? " + swRayTraceRenderOptions.AlphaOutput);
            Debug.Print(" ");

            // Change rendering values
            Debug.Print("New rendering values");
            swRayTraceRenderOptions.ImageHeight = 567;
            Debug.Print(" Image height = " + swRayTraceRenderOptions.ImageHeight);
            swRayTraceRenderOptions.ImageWidth = 678;
            Debug.Print(" Image width = " + swRayTraceRenderOptions.ImageWidth);
            swRayTraceRenderOptions.ImageFormat = (int)swRayTraceRenderImageFormat_e.swImageFormat_HDR;
            Debug.Print(" Image format = " + swRayTraceRenderOptions.ImageFormat);
            swRayTraceRenderOptions.PreviewRenderQuality = (int)swRayTraceRenderQuality_e.swRenderQuality_Better;
            Debug.Print(" Preview render quality = " + swRayTraceRenderOptions.PreviewRenderQuality);
            swRayTraceRenderOptions.FinalRenderQuality = (int)swRayTraceRenderQuality_e.swRenderQuality_Best;
            Debug.Print(" Final render quality = " + swRayTraceRenderOptions.FinalRenderQuality);
```

```

swRayTraceRenderOptions.BloomEnabled = false;
Debug.Print(" Bloom enabled? " + swRayTraceRenderOptions.BloomEnabled);
swRayTraceRenderOptions.BloomThreshold = 70;
Debug.Print(" Bloom threshold = " + swRayTraceRenderOptions.BloomThreshold);
swRayTraceRenderOptions.BloomRadius = 4;
Debug.Print(" Bloom radius = " + swRayTraceRenderOptions.BloomRadius);
swRayTraceRenderOptions.ContourCartoonRenderingEnabled = true;
Debug.Print(" Contour/cartoon rendering enabled? " + swRayTraceRenderOptions.ContourCartoonRenderingEnabled);
swRayTraceRenderOptions.RenderType = (int)swRayTraceRenderingType_e.swRayTraceContour;
Debug.Print(" Render type = " + swRayTraceRenderOptions.RenderType);
swRayTraceRenderOptions.ShadedContour = false;
Debug.Print(" Shaded contour? " + swRayTraceRenderOptions.ShadedContour);
swRayTraceRenderOptions.ContourLineThickness = 3;
Debug.Print(" Contour line thickness = " + swRayTraceRenderOptions.ContourLineThickness);
swRayTraceRenderOptions.ContourLineColor = 255;
Debug.Print(" Contour line color = " + swRayTraceRenderOptions.ContourLineColor);
swRayTraceRenderOptions.UseSolidWorksViewAspectRatio = true;
Debug.Print(" Use SOLIDWORKS Design view aspect ratio? " + swRayTraceRenderOptions.UseSolidWorksViewAspectRatio);

// Display the preview window
//status = swRayTraceRenderer.DisplayPreviewWindow
// Display final render window
//status = swRayTraceRenderer.InvokeFinalRender

// Render in HDR format
status = swRayTraceRenderer.RenderToFile("c:\\temp\\Filter_2", 0, 0);
// Render in BMP format
swRayTraceRenderOptions.ImageFormat = (int)swRayTraceRenderImageFormat_e.swImageFormat_WindowsBmp;
status = swRayTraceRenderer.RenderToFile("c:\\temp\\Filter_1", 0, 0);

// Abort final render
//status = swRayTraceRenderer.AbortFinalRender
// Close preview and final render windows
//status = swRayTraceRenderer.CloseRayTraceRender

```

```

}

```

```

public SldWorks swApp;

```

```

}

```

```

}

```

Render Model (VBA)

This example shows how to use a ray-trace rendering engine to render a model and save the rendered image in BMP and HDR formats. You must have a Premium license to run this example.

```

'-----
' Preconditions:
' 1. Verify that:
'   * specified part exists.
'   * c:\temp exists.
'   * ray-trace rendering engine is loaded in SOLIDWORKS Design.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. If prompted to use perspective views in renderings, click
'    Continue without Camera or Perspective.
' 2. Changes the specified rendering options.
' 3. Creates c:\temp\FILTER_1.bmp and c:\temp\FILTER_2.hdr containing
'    rendered images of the part.
' 4. Examine the Immediate window and c:\temp.
'
' NOTE: Rendering can take several minutes to complete.
'-----

Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swRayTraceRenderer As SldWorks.RayTraceRenderer
Dim swRayTraceRenderOptions As SldWorks.RayTraceRenderOptions
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\routing-pipes\fittings\filter.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

    ' Access ray-trace rendering engine
    ' Set swRayTraceRenderer = swApp.GetRayTraceRenderer(ray_trace_rendering_engine_member)

    ' Get and set rendering options
    Set swRayTraceRenderOptions = swRayTraceRenderer.RayTraceRenderOptions

    ' Get current rendering values
    Debug.Print "Current rendering values"
    Debug.Print " Image height = " & swRayTraceRenderOptions.ImageHeight
    Debug.Print " Image width = " & swRayTraceRenderOptions.ImageWidth
    Debug.Print " Image format = " & swRayTraceRenderOptions.ImageFormat
    Debug.Print " Preview render quality = " & swRayTraceRenderOptions.PreviewRenderQuality
    Debug.Print " Final render quality = " & swRayTraceRenderOptions.FinalRenderQuality
    Debug.Print " Bloom enabled? " & swRayTraceRenderOptions.BloomEnabled
    Debug.Print " Bloom threshold = " & swRayTraceRenderOptions.BloomThreshold
    Debug.Print " Bloom radius = " & swRayTraceRenderOptions.BloomRadius
    Debug.Print " Contour/cartoon enabled? " & swRayTraceRenderOptions.ContourCartoonRenderingEnabled
    Debug.Print " Render type = " & swRayTraceRenderOptions.RenderType
    Debug.Print " Shaded contour? " & swRayTraceRenderOptions.ShadedContour
    Debug.Print " Contour line thickness = " & swRayTraceRenderOptions.ContourLineThickness
    Debug.Print " Contour line color = " & swRayTraceRenderOptions.ContourLineColor
    Debug.Print " Use SOLIDWORKS Design view aspect ratio? " & swRayTraceRenderOptions.UseSolidWorksViewAspectRatio
    Debug.Print " Alpha output? " & swRayTraceRenderOptions.AlphaOutput
    Debug.Print " "

    ' Change rendering values
    Debug.Print "New rendering values"
    swRayTraceRenderOptions.ImageHeight = 567
    Debug.Print " Image height = " & swRayTraceRenderOptions.ImageHeight
    swRayTraceRenderOptions.ImageWidth = 678
    Debug.Print " Image width = " & swRayTraceRenderOptions.ImageWidth
    swRayTraceRenderOptions.ImageFormat = swImageFormat_HDR
    Debug.Print " Image format = " & swRayTraceRenderOptions.ImageFormat
    swRayTraceRenderOptions.PreviewRenderQuality = swRenderQuality_Better
    Debug.Print " Preview render quality = " & swRayTraceRenderOptions.PreviewRenderQuality
    swRayTraceRenderOptions.FinalRenderQuality = swRenderQuality_Best
    Debug.Print " Final render quality = " & swRayTraceRenderOptions.FinalRenderQuality
    swRayTraceRenderOptions.BloomEnabled = False
    Debug.Print " Bloom enabled? " & swRayTraceRenderOptions.BloomEnabled
    swRayTraceRenderOptions.BloomThreshold = 70
    Debug.Print " Bloom threshold = " & swRayTraceRenderOptions.BloomThreshold
    swRayTraceRenderOptions.BloomRadius = 4
    Debug.Print " Bloom radius = " & swRayTraceRenderOptions.BloomRadius
    swRayTraceRenderOptions.ContourCartoonRenderingEnabled = True
    Debug.Print " Contour/cartoon enabled? " & swRayTraceRenderOptions.ContourCartoonRenderingEnabled
    swRayTraceRenderOptions.RenderType = swRayTraceRenderingType_e.swRayTraceContour
    Debug.Print " Render type = " & swRayTraceRenderOptions.RenderType
    swRayTraceRenderOptions.ShadedContour = False

```

```
Debug.Print " Shaded contour? " & swRayTraceRenderOptions.ShadedContour  
swRayTraceRenderOptions.ContourLineThickness = 3  
Debug.Print " Contour line thickness = " & swRayTraceRenderOptions.ContourLineThickness  
swRayTraceRenderOptions.ContourLineColor = 255  
Debug.Print " Contour line color = " & swRayTraceRenderOptions.ContourLineColor  
swRayTraceRenderOptions.UseSolidWorksViewAspectRatio = True  
Debug.Print " Use SOLIDWORKS Design view aspect ratio? " & swRayTraceRenderOptions.UseSolidWorksViewAspectRatio
```

```
' Display the preview window  
'status = swRayTraceRenderer.DisplayPreviewWindow  
' Display final render window  
'status = swRayTraceRenderer.InvokeFinalRender  
  
' Render in HDR format  
status = swRayTraceRenderer.RenderToFile("c:\temp\FILTER_2", 0, 0)  
' Render in BMP format  
swRayTraceRenderOptions.ImageFormat = swImageFormat_WindowsBmp  
status = swRayTraceRenderer.RenderToFile("c:\temp\FILTER_1", 0, 0)  
  
' Abort final render  
'status = swRayTraceRenderer.AbortFinalRender  
' Close preview and final render windows  
'status = swRayTraceRenderer.CloseRayTraceRender
```

End Sub

Render Model (VB.NET)

This example shows how to use a ray-trace rendering engine to render a model and save the rendered image in BMP and HDR formats. You must have a Premium license to run this example.

```
'-----  
' Preconditions:  
' 1. Verify that:  
'    * specified part exists.  
'    * c:\temp exists.  
'    * ray-trace rendering engine is loaded in SOLIDWORKS Design.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. If prompted to use perspective views in renderings, click  
'    Continue without Camera or Perspective.  
' 2. Changes the specified rendering options.  
' 3. Creates c:\temp\Filter_1.bmp and c:\temp\Filter_2.hdr containing  
'    rendered images of the part.  
' 4. Examine the Immediate window and c:\temp.  
'  
' NOTE: Rendering can take several minutes to complete.  
'-----
```

Reorder Features Example (C#)

This example shows how to move a feature to another location in the FeatureManager design tree of a part.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\clamp2.sldprt.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Fillet5 moves to the end of the FeatureManager design tree.
// 2. Inspect the Immediate window.
//
// NOTE: Because the assembly document is used by elsewhere,
// do not save any changes when closing the document.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace ReorderFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 modelDoc2;
        ModelDocExtension modelDocExt;
        bool retVal;

        public void Main()
        {
            modelDoc2 = (ModelDoc2)swApp.ActiveDoc;
            modelDocExt = modelDoc2.Extension;

            retVal = modelDocExt.ReorderFeature2("Fillet5", "", (int)swMoveLocation_e.swMoveToEnd);
            Debug.Print("Fillet5 moved to the end of the FeatureManager design tree? " + retVal);
        }

        public SldWorks swApp;
    }
}
```

Reorder Features Example (VBA)

This example shows how to move a feature to another location in the FeatureManager design tree of a part.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\clamp2.sldprt.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Fillet5 moves to the end of the FeatureManager design tree.  
' 2. Inspect the Immediate window.  
'  
' NOTE: Because the assembly document is used by elsewhere,  
' do not save any changes when closing the document.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim modelDoc2 As SldWorks.modelDoc2  
Dim modelDocExt As SldWorks.ModelDocExtension  
Dim retVal As Boolean
```

Sub Main()

```
    Set swApp = Application.SldWorks  
    Set modelDoc2 = swApp.ActiveDoc  
    Set modelDocExt = modelDoc2.Extension  
  
    retVal = modelDocExt.ReorderFeature2("Fillet5", "", swMoveToEnd)  
    Debug.Print "Fillet5 moved to the end of the FeatureManager design tree? " & retVal
```

End Sub

Reorder Features Example (VB.NET)

This example shows how to move a feature to another location in the FeatureManager design tree of a part.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\clamp2.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Fillet5 moves to the end of the FeatureManager design tree.
' 2. Inspect the Immediate window.
'
' NOTE: Because the assembly document is used by elsewhere,
' do not save any changes when closing the document.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System.Runtime.InteropServices
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim modelDoc2 As ModelDoc2
```

```
    Dim featureMgr As FeatureManager
```

```
    Dim modelDocExt As ModelDocExtension
```

```
    Dim retVal As Boolean
```

```
    Sub Main()
```

```
        modelDoc2 = swApp.ActiveDoc
```

```
        modelDocExt = modelDoc2.Extension
```

```
        retVal = modelDocExt.ReorderFeature2("Fillet5", "", swMoveLocation_e.swMoveToEnd)
```

```
        Debug.Print("Fillet5 moved to the end of the FeatureManager design tree? " & retVal)
```

```
    End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Reorganize Components Example (VBA)

This example shows how to reorder components in an assembly.

```
'-----
' Preconditions:
' 1. Copy and paste this code in the main module.
' 2. Click Insert > Class module and copy and paste this code in that module.
' 3. Open public_documents\samples\tutorial\assemblyvisualize\food_processor.sldasm.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Run the macro.
' 2. Interactively select both shaft washer components in the FeatureManager design tree.
' 3. Press F5.
' 4. Interactively select the blade shaft subassembly in the FeatureManager design tree.
' 5. Press F5.
' 6. Expand and examine the blade shaft subassembly in the FeatureManager design tree to verify
'    that both shaft washer components moved to the subassembly.
' 7. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
```

```
'Module
Option Explicit

Dim myClass As Class1

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim myModel As SldWorks.ModelDoc2
    Dim myAssem As SldWorks.AssemblyDoc
    Dim selMgr As SldWorks.SelectionMgr
    Dim selCount As Long, selType As Long
    Dim selObj As Object
    Dim selSource() As SldWorks.Component2
    Dim vSource As Variant
    Dim selTarget As SldWorks.Component2
    Dim boolstatus As Boolean

    Set swApp = Application.SldWorks
    Set myModel = swApp.ActiveDoc
    Set myAssem = myModel

    Set myClass = New Class1
    Set myClass.msrcAssemblyDoc = myAssem

    ' Interactively select components to move
    ' The selected components must be from the
    ' same level of an assembly
    ' Press F5
    Stop

    Set selMgr = myModel.SelectionManager
    selCount = selMgr.GetSelectedObjectCount2(0)
    If (selCount = 0) Then
        Exit Sub
    End If

    ReDim selSource(0 To selCount - 1)
    Dim i As Long
    For i = 1 To selCount
        selType = selMgr.GetSelectedObjectType3(i, 0)
        If (selType = SwConst.swSelCOMPONENTS) Then
            Set selObj = selMgr.GetSelectedObject6(i, 0)
            Set selSource(i - 1) = selObj
        End If
    Next i

    vSource = selSource

    myModel.ClearSelection2 True

    ' Interactively select a top-level assembly or
    ' sub-assembly where to move the
    ' previously selected components
    ' Press F5
    Stop

    selCount = selMgr.GetSelectedObjectCount2(0)
    If (selCount > 0) Then
        selType = selMgr.GetSelectedObjectType3(1, 0)
        If selType = SwConst.swSelCOMPONENTS Then
            Set selObj = selMgr.GetSelectedObject6(1, 0)
            If Not selObj Is Nothing Then
                Set selTarget = selObj
            Else
                Set selTarget = myAssem.GetEditTargetComponent
            End If
        End If
    End If

    myModel.ClearSelection2 True

    If Not (selTarget Is Nothing) Then
        boolstatus = myAssem.ReorganizeComponents((vSource), selTarget)
```

```
' AssemblyDoc ComponentReorganizeNotify event is fired
If boolstatus = False Then
    Debug.Print "Reordering components failed."
Else
    Debug.Print "Reordering components succeeded."
End If
End Sub
```

[Back to top](#)

```
'Class1
Option Explicit

Public WithEvents msrAssemblyDoc As SldWorks.AssemblyDoc
Public Function msrAssemblyDoc_ComponentReorganizeNotify(ByVal sourceName As String, ByVal targetName As String) As Long
    Debug.Print "IAssemblyDocEvent ComponentReorganizeNotify"
    Debug.Print " Source component is: " & sourceName
    Debug.Print " Target component is: " & targetName
    msrAssemblyDoc_ComponentReorganizeNotify = 1
End Function
```

[Back to top](#)

Repair Mates Missing Same Mate Entity Example (C#)

This example shows how to repair all mates missing the same mate entity.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\RepairAssemMates.sldasm.
// 2. Click Close.
//
// Postconditions:
// 1. Selects the Concentric1 mate.
// 2. Selects a face on BoltHolder<1> and a face on Bolt<1>.
// 3. Edits and repairs all mates missing the same mate entity.
// 4. Examine the FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            bool status = false;
            int errors = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swAssembly = (AssemblyDoc)swModel;

            status = swModelDocExt.SelectByID2("Concentric1", "MATE", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", -0.0562994754449733, -0.000769308980977712, 0.0081518960735707, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", -0.0516662570718722, 0.0153992319276881, 0.0199034517498262, true, 1, null, 0);
            swAssembly.EditMate4((int)swMateType_e.swMateCONCENTRIC, (int)swMateAlign_e.swMateAlignALIGNED, false, 0.001, 0.001, 0.001, 0.001, 0.001, 0, 0.5235987755983, 0.5235987755983, false, false, 0, true, out errors);
            swModel.ClearSelection2(true);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Repair Mates Missing Same Mate Entity Example (VBA)

This example shows how to repair all mates missing the same mate entity.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\RepairAssemMates.sldasm.  
' 2. Click Close.  
'  
' Postconditions:  
' 1. Selects the Concentric1 mate.  
' 2. Selects a face on BoltHolder<1> and a face on Bolt<1>.  
' 3. Edits and repairs all mates missing the same mate entity.  
' 4. Examine the FeatureManager design tree.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim status As Boolean  
Dim errors As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swAssembly = swModel  
  
    status = swModelDocExt.SelectByID("Concentric1", "MATE", 0, 0, 0, False, 0, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "FACE", -5.62994754449733E-02, -7.69308980977712E-04, 8.1518960735707E-03, True, 1, Nothing, 0)  
    status = swModelDocExt.SelectByID("", "FACE", -5.16662570718722E-02, 1.53992319276881E-02, 1.99034517498262E-02, True, 1, Nothing, 0)  
    swAssembly.EditMate4 swMateType_e.swMateCONCENTRIC, swMateAlign_e.swMateAlignALIGNED, False, 0.001, 0.001, 0.001, 0.001, 0.001, 0, 0.5235987755983,  
0.5235987755983, False, False, 0, True, errors  
    swModel.ClearSelection2 True  
  
End Sub
```

Repair Mates Missing Same Mate Entity Example (VB.NET)

This example shows how to repair all mates missing the same mate entity.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\api\RepairAssemMates.sldasm.
2. Click Close.
Postconditions:
1. Selects the Concentric1 mate.
2. Selects a face on BoltHolder<1> and a face on Bolt<1>.
3. Edits and repairs all mates missing the same mate entity.
4. Examine the FeatureManager design tree.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssembly As AssemblyDoc
        Dim status As Boolean
        Dim errors As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swAssembly = swModel

        status = swModelDocExt.SelectByID2("Concentric1", "MATE", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", -0.0562994754449733, -0.000769308980977712, 0.0081518960735707, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", -0.0516662570718722, 0.0153992319276881, 0.0199034517498262, True, 1, Nothing, 0)
        swAssembly.EditMate4(swMateType_e.swMateCONCENTRIC, swMateAlign_e.swMateAlignALIGNED, False, 0.001, 0.001, 0.001, 0.001, 0.001, 0, 0.5235987755983
, 0.5235987755983, False, False, 0, True, errors)
        swModel.ClearSelection2(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Replace Component Example (C#)

This example shows how to replace all instances of a component with a different component.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\assem1.sldasm.
// 2. Select testpart1<1> in the FeatureManager design tree.
// 3. Verify that the specified part exists.
// 4. Open the Immediate window.
//
// Postconditions:
// 1. Replaces all instances of the selected component with the
//    specified replacement component.
// 2. Examine the Immediate window, FeatureManager design tree,
//    and assembly.
//
// NOTE: Because the assembly is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            const string fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\block.sldprt";
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssy = default(AssemblyDoc);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2 swSelComp = default(Component2);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelComp = (Component2)swSelMgr.GetSelectedObjectsComponent4(1, -1);
            Debug.Print("Old component = " + swSelComp.Name2);
            status = swAssy.ReplaceComponents2(fileName, "", true, 0, true);
            Debug.Print("Replacement component = " + swSelComp.Name2);
            Debug.Print("All instances of old component replaced? " + status);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Replace Component Example (VBA)

This example shows how to replace all instances of a component with a different component.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\assem1.sldasm.  
' 2. Select testpart1<1> in the FeatureManager design tree.  
' 3. Verify that the specified part exists.  
' 4. Open the Immediate window.  
'  
' Postconditions:  
' 1. Replaces all instances of the selected component with the  
'    specified replacement component.  
' 2. Examine the Immediate window, FeatureManager design tree,  
'    and assembly.  
'  
' NOTE: Because the assembly is used elsewhere, do not save  
'    changes.  
'-----  
Option Explicit  
  
Sub main()  
  
    Const fileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swAssy As SldWorks.AssemblyDoc  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelComp As SldWorks.Component2  
    Dim status As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swAssy = swModel  
    Set swSelMgr = swModel.SelectionManager  
    Set swSelComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)  
    Debug.Print "Old component = " & swSelComp.Name2  
    status = swAssy.ReplaceComponents2(fileName, "", True, 0, True)  
    Debug.Print "Replacement component = " & swSelComp.Name2  
    Debug.Print "All instances of old component replaced? " & status  
  
End Sub
```

Replace Component Example (VB.NET)

This example shows how to replace all instances of a component with a different component.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\assem1.sldasm.
' 2. Select testpart1<1> in the FeatureManager design tree.
' 3. Verify that the specified part exists.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. Replaces all instances of the selected component with the
'    specified replacement component.
' 2. Examine the Immediate window, FeatureManager design tree,
'    and assembly.
'
' NOTE: Because the assembly is used elsewhere, do not save
' changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Const fileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"
        Dim swModel As ModelDoc2
        Dim swAssy As AssemblyDoc
        Dim swSelMgr As SelectionMgr
        Dim swSelComp As Component2
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swAssy = swModel
        swSelMgr = swModel.SelectionManager
        swSelComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)
        Debug.Print("Old component = " & swSelComp.Name2)
        status = swAssy.ReplaceComponents2(fileName, "", True, 0, True)
        Debug.Print("Replacement component = " & swSelComp.Name2)
        Debug.Print("All instances of old component replaced? " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Replace Dimension with Text Example (VBA)

This example shows how to temporarily replace the selected dimension with text.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
'
' Postconditions:
' 1. Selects a dimension.
' 2. Replaces the selected dimension with display text.
' 3. Reselects the dimension.
' 4. Examine drawing.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim model As SldWorks.ModelDoc2
Dim selMgr As SldWorks.SelectionMgr
Dim selObj As Object
Dim selType As Long, selCount As Long
Dim selDim As SldWorks.DisplayDimension
Dim textFont As SldWorks.TextFormat
Dim entType As Long
Dim triData(0 To 8) As Double, vEntData As Variant
Dim dotData(0 To 3) As Double, arcData(0 To 11) As Double, textPos(0 To 2) As Double
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set model = swApp.ActiveDoc
    Set selMgr = model.SelectionManager
    boolstatus = model.Extension.SelectByID2("RD19@Drawing View1", "DIMENSION", 0.277820016554234, 0.29219884729064, 0, False, 0, Nothing, 0)
    selCount = selMgr.GetSelectedObjectCount()
    If selCount > 0 Then
        selType = selMgr.GetSelectedObjectType3(1, -1)
        Set selObj = selMgr.GetSelectedObject6(1, -1)
        If selType = SwConst.swSelDIMENSIONS Then
            Set selDim = selObj
            triData(0) = 0.12
            triData(1) = 0.1
            triData(2) = 0#
            triData(3) = 0.15
            triData(4) = 0.1
            triData(5) = 0#
            triData(6) = 0.13
            triData(7) = 0.13
            triData(8) = 0#
            vEntData = triData
            boolstatus = selDim.AddDisplayEnt(2, (vEntData)) ' Filled triangle
            dotData(0) = 0.11
            dotData(1) = 0.14
            dotData(2) = 0#
            dotData(3) = 0.005
            vEntData = dotData
            boolstatus = selDim.AddDisplayEnt(6, (vEntData)) ' Filled dot
            dotData(0) = 0.16
            dotData(1) = 0.14
            dotData(2) = 0#
            dotData(3) = 0.009
            vEntData = dotData
            boolstatus = selDim.AddDisplayEnt(6, (vEntData)) ' Filled dot
            arcData(0) = 0.135
            arcData(1) = 0.11
            arcData(2) = 0#
            arcData(3) = 0#
            arcData(4) = 0#
            arcData(5) = 1#
            arcData(6) = 0.09
            arcData(7) = 0.11
            arcData(8) = 0#
            arcData(9) = 0.18
            arcData(10) = 0.11
            arcData(11) = 0#
            vEntData = arcData
            boolstatus = selDim.AddDisplayEnt(4, (vEntData)) ' Arc
            textPos(0) = 0.09
            textPos(1) = 0.065
            textPos(2) = 0#
            vEntData = textPos
            Set textFont = model.GetUserPreferenceTextFormat(SwConst.swDetailingDimensionTextFormat)
            textFont.CharHeightInPts = 48
            boolstatus = selDim.AddDisplayText("Hi!", (vEntData), textFont, SwConst.swTextJustificationCenter, 1#)
        End If
    End If

    model.ClearSelection2 True
    boolstatus = model.Extension.SelectByID2("RD19@Drawing View1", "DIMENSION", 0.277820016554234, 0.29219884729064, 0, False, 0, Nothing, 0)

End Sub
```

Create Replace Face Feature Example (C#)

This example shows how to create a Replace Face feature.

```
//-----
// Preconditions:
// 1. Verify that the specified model document exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Opens the specified part.
// 2. Creates Plane1, Surface-Extrude1, and Replace Face1.
// 3. Inspect the FeatureManager design tree, the graphics area, and the
// Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace ReplaceFace_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        SelectionMgr selMgr;
        ModelDoc2 Part;
        Feature feat;
        ReplaceFaceFeatureData featData;
        bool boolstatus;
        int longstatus;
        int longwarnings;

        public void Main()
        {
            Part = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\block20.sldprt", 1, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("block20", false, ref longstatus);
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID2("", "FACE", 0.00687152192142548, 0.0256655537640995, 0.049345602378537, true, 0,
            RefPlane myRefPlane = default(RefPlane);
            myRefPlane = (RefPlane)Part.FeatureManager.InsertRefPlane(264, 0.05842, 0, 0, 0, 0);
            Part.ClearSelection2(true);

            object pointArray = null;
            double[] points = new double[15];
            points[0] = -0.0700496017443584;
            points[1] = 0.0582762055241233;
            points[2] = 0;
            points[3] = -0.0357558994484748;
            points[4] = 0.0853945497913173;
            points[5] = 0;
            points[6] = -0.00588719099721402;
            points[7] = 0.0671372129016845;
            points[8] = 0;
            points[9] = 0.0273002628375139;
            points[10] = 0.0878577815467452;
            points[11] = 0;
            points[12] = 0.0737626982062238;
            points[13] = 0.0582762055241233;
            points[14] = 0;
            pointArray = points;
            SketchSegment skSegment = default(SketchSegment);
            skSegment = Part.SketchManager.CreateSpline((pointArray));
            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("Spline1@Sketch2", "EXTSKETCHSEGMENT", -0.0549544681183813, 0.0875052976097064,
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 4, null, 0);
            ((SelectionMgr)Part.SelectionManager).EnableContourSelection = true;
            boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCHCONTOUR", 0, 0, 0, true, 4, null, 0);
            Part.FeatureExtruRefSurface2(true, false, false, 0, 0, 0.14478, 0.14478, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false);
            ((SelectionMgr)Part.SelectionManager).EnableContourSelection = false;
            boolstatus = Part.Extension.SelectByID2("", "FACE", 0.0585444908073214, 0.0396239999998329, -0.0518899759430838, true, 0,
            boolstatus = Part.Extension.SelectByID2("Surface-
Extrude1", "SURFACEBODY", -0.0189730427370591, 0.0726880897401543, 0.115671174990496, true, 0, null, 0);
            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("", "FACE", 0.0585444908073214, 0.0396239999998329, -0.0518899759430838, true, 1
```

```

        boolstatus = Part.Extension.SelectByID("Surface-
Extrude1", "SURFACEBODY", -0.0189730427370591, 0.0726880897401543, 0.115671174990496, true, 2, null, 0);
        Part.InsertFeatureReplaceFace();
        boolstatus = Part.Extension.SelectByID("", "FACE", -0.0362064915135534, 0.0856902732399476, 0.127037337239983, false, 0,
        Part.FeatureManager.HideBodies();
        boolstatus = Part.Extension.SelectByID("Plane1", "PLANE", -0.0693294107213475, 0.0872697709380442, -0.0300713252946179,
        Part.BlankRefGeom();

        boolstatus = Part.Extension.SelectByID("Replace Face1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
        selMgr = (SelectionMgr)Part.SelectionManager;
        feat = (Feature)selMgr.GetSelectedObject6(1, -1);
        featData = (ReplaceFaceFeatureData)feat.GetDefinition();

        featData.AccessSelections(Part, null);

        object[] vFacesToReplace = null;
        vFacesToReplace = (object[])featData.FacesForReplacement;
        Debug.Print(featData.GetFacesForReplacementCount() + " face replaced in " + ((Feature)
        ((Face2)vFacesToReplace[0]).GetFeature()).Name);
        Debug.Print(featData.GetReplacementSurfacesCount() + " replacement surface ");

        featData.ReleaseSelectionAccess();

    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
}
}

```

Create Replace Face Feature Example (VBA)

This example shows how to create a Replace Face feature.

```

' -----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Creates Plane1, Surface-Extrude1, and Replace Face1.
' 3. Inspect the FeatureManager design tree, the graphics area, and the
'    Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
' -----

Dim swApp As SldWorks.SldWorks
Dim selMgr As SldWorks.SelectionMgr
Dim Part As SldWorks.ModelDoc2
Dim feat As SldWorks.Feature
Dim featData As SldWorks.ReplaceFaceFeatureData
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long

Option Explicit

Sub main()
    Set swApp = Application.SldWorks

    Set Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt", 1, 0, "",
longstatus, longwarnings)
    swApp.ActivateDoc2 "block20", False, longstatus
    Set Part = swApp.ActiveDoc

    boolstatus = Part.Extension.SelectByID2("", "FACE", 6.87152192142548E-03, 2.56655537640995E-02, 0.049345602378537, True, 0,
Nothing, 0)
    Dim myRefPlane As SldWorks.RefPlane
    Set myRefPlane = Part.FeatureManager.InsertRefPlane(264, 0.05842, 0, 0, 0, 0)
    Part.ClearSelection2 True

    Dim pointArray As Variant
    Dim points() As Double
    ReDim points(0 To 14) As Double
    points(0) = -7.00496017443584E-02
    points(1) = 5.82762055241233E-02
    points(2) = 0
    points(3) = -3.57558994484748E-02
    points(4) = 8.53945497913173E-02
    points(5) = 0
    points(6) = -5.88719099721402E-03
    points(7) = 6.71372129016845E-02
    points(8) = 0
    points(9) = 2.73002628375139E-02
    points(10) = 8.78577815467452E-02
    points(11) = 0
    points(12) = 7.37626982062238E-02
    points(13) = 5.82762055241233E-02
    points(14) = 0
    pointArray = points
    Dim skSegment As SldWorks.SketchSegment
    Set skSegment = Part.SketchManager.CreateSpline((pointArray))
    Part.SketchManager.InsertSketch True
    boolstatus = Part.Extension.SelectByID2("Spline1@Sketch2", "EXTSKETCHSEGMENT", -5.49544681183813E-02, 8.75052976097064E-02, 0,
False, 0, Nothing, 0)
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    Part.SelectionManager.EnableContourSelection = True
    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCHCONTOUR", 0, 0, 0, True, 4, Nothing, 0)
    Part.FeatureExtrudeRefSurface2 True, False, 0, 0, 0.14478, 0.14478, False, False, False, False, 1.74532925199433E-02,
1.74532925199433E-02, False, False, False, False
    Part.SelectionManager.EnableContourSelection = False
    boolstatus = Part.Extension.SelectByID2("", "FACE", 5.85444908073214E-02, 3.96239999998329E-02, -5.18899759430838E-02, True, 0,
Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Surface-Extrude1", "SURFACEBODY", -1.89730427370591E-02, 7.26880897401543E-02,
0.115671174990496, True, 0, Nothing, 0)
    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("", "FACE", 5.85444908073214E-02, 3.96239999998329E-02, -5.18899759430838E-02, True, 1,
Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Surface-Extrude1", "SURFACEBODY", -1.89730427370591E-02, 7.26880897401543E-02,
0.115671174990496, True, 2, Nothing, 0)
    Part.InsertFeatureReplaceFace
    boolstatus = Part.Extension.SelectByID2("", "FACE", -3.62064915135534E-02, 8.56902732399476E-02, 0.127037337239983, False, 0,
Nothing, 0)
    Part.FeatureManager.HideBodies
    boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", -6.93294107213475E-02, 8.72697709380442E-02, -3.00713252946179E-02,

```

```
False, 0, Nothing, 0)
Part.BlankRefGeom

boolstatus = Part.Extension.SelectByID2("Replace Face1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set selMgr = Part.SelectionManager
Set feat = selMgr.GetSelectedObject6(1, -1)
Set featData = feat.GetDefinition

featData.AccessSelections Part, Nothing

Dim vFacesToReplace As Variant
vFacesToReplace = featData.FacesForReplacement
Debug.Print featData.GetFacesForReplacementCount & " face replaced in " & vFacesToReplace(0).GetFeature.Name
Debug.Print featData.GetReplacementSurfacesCount & " replacement surface "

featData.ReleaseSelectionAccess

End Sub
```

Create Replace Face Feature Example (VB.NET)

This example shows how to create a Replace Face feature.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Opens the specified part.
' 2. Creates Plane1, Surface-Extrude1, and Replace Face1.
' 3. Inspect the FeatureManager design tree, the graphics area, and the
'    Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim selMgr As SelectionMgr
    Dim Part As ModelDoc2
    Dim feat As Feature
    Dim featData As ReplaceFaceFeatureData
    Dim boolstatus As Boolean
    Dim longstatus As Integer, longwarnings As Integer

```

```
Sub main()
```

```

    Part = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\block20.sldprt", 1, 0, "", longstatus, longwarnings)
    swApp.ActivateDoc2("block20", False, longstatus)
    Part = swApp.ActiveDoc

    boolstatus = Part.Extension.SelectByID2("", "FACE", 0.00687152192142548, 0.0256655537640995, 0.049345602378537, True, 0, Not
    Dim myRefPlane As RefPlane
    myRefPlane = Part.FeatureManager.InsertRefPlane(264, 0.05842, 0, 0, 0, 0)
    Part.ClearSelection2(True)

    Dim pointArray As Object
    Dim points(0 To 14) As Double
    points(0) = -0.0700496017443584
    points(1) = 0.0582762055241233
    points(2) = 0
    points(3) = -0.0357558994484748
    points(4) = 0.0853945497913173
    points(5) = 0
    points(6) = -0.00588719099721402
    points(7) = 0.0671372129016845
    points(8) = 0
    points(9) = 0.0273002628375139
    points(10) = 0.0878577815467452
    points(11) = 0
    points(12) = 0.0737626982062238
    points(13) = 0.0582762055241233
    points(14) = 0
    pointArray = points
    Dim skSegment As SketchSegment
    skSegment = Part.SketchManager.CreateSpline((pointArray))
    Part.SketchManager.InsertSketch(True)
    boolstatus = Part.Extension.SelectByID2("Spline1@Sketch2", "EXTSKETCHSEGMENT", -0.0549544681183813, 0.0875052976097064, 0, F
    Part.ClearSelection2(True)
    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, 0, False, 4, Nothing, 0)
    Part.SelectionManager.EnableContourSelection = True
    boolstatus = Part.Extension.SelectByID2("Sketch2", "SKETCHCONTOUR", 0, 0, 0, 0, True, 4, Nothing, 0)
    Part.FeatureExtruRefSurface2(True, False, False, 0, 0, 0.14478, 0.14478, False, False, False, False, 0.0174532925199433, 0.0
    Part.SelectionManager.EnableContourSelection = False
    boolstatus = Part.Extension.SelectByID2("", "FACE", 0.0585444908073214, 0.0396239999998329, -0.0518899759430838, True, 0, Nc
    boolstatus = Part.Extension.SelectByID2("Surface-
Extrude1", "SURFACEBODY", -0.0189730427370591, 0.0726880897401543, 0.115671174990496, True, 0, Nothing, 0)
    Part.ClearSelection2(True)
    boolstatus = Part.Extension.SelectByID2("Surface-Extrude1", "BODYFEATURE", 0, 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("", "FACE", 0.0585444908073214, 0.0396239999998329, -0.0518899759430838, True, 1, Nc
    boolstatus = Part.Extension.SelectByID2("Surface-
Extrude1", "SURFACEBODY", -0.0189730427370591, 0.0726880897401543, 0.115671174990496, True, 2, Nothing, 0)
    Part.InsertFeatureReplaceFace()
    boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0362064915135534, 0.0856902732399476, 0.127037337239983, False, 0, Nc
    Part.FeatureManager.HideBodies()
    boolstatus = Part.Extension.SelectByID2("Plane1", "PLANE", -0.0693294107213475, 0.0872697709380442, -0.0300713252946179, Fal
    Part.BlankRefGeom()

```

```

boolstatus = Part.Extension.SelectByID2("Replace Face1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
selMgr = Part.SelectionManager
feat = selMgr.GetSelectedObject6(1, -1)
featData = feat.GetDefinition

featData.AccessSelections(Part, Nothing)

Dim vFacesToReplace As Object
vFacesToReplace = featData.FacesForReplacement
Debug.Print(featData.GetFacesForReplacementCount & " face replaced in " & vFacesToReplace(0).GetFeature.Name)
Debug.Print(featData.GetReplacementSurfacesCount & " replacement surface ")

featData.ReleaseSelectionAccess()

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Replace Referenced Document Example (VBA)

This example shows how to replace a referenced document with a different document in an assembly.

```
'-----  
' Preconditions:  
' 1. Create C:\temp\blade shaft.  
' 2. Copy these files from C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings  
'    to C:\temp\blade shaft:  
'    * blade shaft.sldasm  
'    * drive shaft.sldprt  
'    * drive shaft pin.sldprt  
'    * drive shaft plate.sldprt  
' 3. Open C:\temp\blade shaft\drive shaft.sldprt and save the file as drive shaft2.sldprt.  
' 4. Close drive shaft2.sldprt.  
' 5. Open the Immediate window.  
'  
' Postconditions:  
' 1. Replaces C:\temp\blade shaft\drive shaft.sldprt with C:\temp\blade shaft\drive shaft2.sldprt.  
' 2. Examine the Immediate window and open C:\temp\blade shaft\blade shaft.sldasm, click Rebuild,  
'    click File > Find References and examine the files listed under Name.  
'-----
```

Option Explicit

```
Const sReferencingDoc As String = "C:\temp\blade shaft\blade shaft.sldasm"  
Const sOldDoc As String = "C:\temp\blade shaft\drive shaft.sldprt"  
Const sNewDoc As String = "C:\temp\blade shaft\drive shaft2.sldprt"
```

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
  
    bRet = swApp.ReplaceReferencedDocument(sReferencingDoc, sOldDoc, sNewDoc)  
    Debug.Print ("Reference to drive shaft.sldprt replaced with reference to drive shaft2.sldprt? " & bRet)
```

End Sub

Replace Sketch Entity Example (C#)

This example shows how to replace a sketch entity in a model with another sketch entity.

```
//-----
// Preconditions: Verify that the specified template exists.
//
// Postconditions:
// 1. Opens a new part and creates Boss-Extrude1.
// 2. Replaces a sketch line with a sketch arc and modifies
//    Boss-Extrude1.
// 3. Examine the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace ReplaceSketch_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        Feature myFeature;
        SketchSegment skSegment;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2022\\templates\\Part.prtdot", 0, 0, 0);
            Part = (ModelDoc2)swApp.ActiveDoc;

            boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.048646278525398, 0.0222864804840025,
0.0105288722478765, false, 0, null, 0);

            object vSkLines = null;
            vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0338155129850894, 0.0167825138518592, 0, 0.0551067619016271,
-0.0245475575743612, 0);
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);
            Part.ShowNamedView2("Trimetric", 8);

            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 4, null, 0);

            myFeature = Part.FeatureManager.FeatureExtrusion2(true, false, false, 0, 0, 0.01778, 0.00254, false, false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
0, 0, false);

            boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            Part.EditSketch();
            Part.ClearSelection2(true);

            skSegment = Part.SketchManager.Create3PointArc(-0.033816, 0.016783, 0.0, 0.055107, 0.016783, 0.0, 0.016009, 0.025458,
0.0);
            Part.ClearSelection2(true);

            boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 0.00202904300411839, 0.0119654152286464,
-0.00709549576220667, true, 0, null, 0);
            boolstatus = Part.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", 0.00588878331207997, 0.0171023327681304,
-0.0126221740799126, true, 0, null, 0);

            // Replace Line1 with Arc1, delete Line1, and make Arc1 a contour
            boolstatus = Part.SketchManager.SketchReplace2(false, true);

            Part.SketchManager.InsertSketch(true);
        }

        public SldWorks swApp;
    }
}
```

Replace Sketch Entity Example (VBA)

This example shows how to replace a sketch entity in a model with another sketch entity.

```
'-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Opens a new part and creates Boss-Extrude1.
' 2. Replaces a sketch line with a sketch arc and modifies
'    Boss-Extrude1.
' 3. Examine the graphics area.
'-----
Dim swApp As SldWorks.SldWorks

Dim Part As SldWorks.ModelDoc2

Dim myFeature As SldWorks.Feature

Dim skSegment As SldWorks.SketchSegment

Dim boolstatus As Boolean

Option Explicit

Sub main()
    Set swApp = _
        Application.SldWorks

    Set Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2022\templates\Part.prtdot", 0, 0, 0)
    Set Part = swApp.ActiveDoc

    boolstatus = Part.Extension.SelectByID2("Front Plane", "PLANE", -0.048646278525398, 2.22864804840025E-02, 1.05288722478765E-02,
False, 0, Nothing, 0)
    Dim vSkLines As Variant
    vSkLines = Part.SketchManager.CreateCornerRectangle(-3.38155129850894E-02, 1.67825138518592E-02, 0, 5.51067619016271E-02,
-2.45475575743612E-02, 0)
    Part.ClearSelection2 True
    Part.SketchManager.InsertSketch True
    Part.ShowNamedView2 "**Trimetric", 8
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)

    Set myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0, 0, 0.01778, 0.00254, False, False, False, False,
1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, 0, 0, False)
    Part.SelectionManager.EnableContourSelection = False
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Part.EditSketch
    Part.ClearSelection2 True

    Set skSegment = Part.SketchManager.Create3PointArc(-0.033816, 0.016783, 0#, 0.055107, 0.016783, 0#, 0.016009, 0.025458, 0#)
    Part.ClearSelection2 True

    boolstatus = Part.Extension.SelectByID2("Line1", "SKETCHSEGMENT", 2.02904300411839E-03, 1.19654152286464E-02,
-7.09549576220667E-03, True, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", 5.88878331207997E-03, 1.71023327681304E-02,
-1.26221740799126E-02, True, 0, Nothing, 0)

    ' Replace Line1 with Arc1, delete Line1, and make Arc1 a contour
    boolstatus = Part.SketchManager.SketchReplace2(False, True)

    Part.SketchManager.InsertSketch True

End Sub
```

Replace Sketch Entity Example (VB.NET)

This example shows how to replace a sketch entity in a model with another sketch entity.

```
'-----
' Preconditions: Verify that the specified template exists.
'
' Postconditions:
' 1. Opens a new part and creates Boss-Extrude1.
' 2. Replaces a sketch line with a sketch arc and modifies
'    Boss-Extrude1.
' 3. Examine the graphics area.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2

    Dim myFeature As Feature

    Dim skSegment As SketchSegment

    Dim boolstatus As Boolean

    Sub main()

        Part = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS
2022\templates\Part.prtdot", 0, 0, 0)

        Part = swApp.ActiveDoc

        boolstatus = Part.Extension.SelectByID("Front
Plane", "PLANE", -0.048646278525398,
0.0222864804840025, 0.0105288722478765, False, 0, Nothing, 0)

        Dim vSkLines As Object

        vSkLines = Part.SketchManager.CreateCornerRectangle(-0.0338155129850894,
0.0167825138518592, 0, 0.0551067619016271, -0.0245475575743612, 0)

        Part.ClearSelection2(True)

        Part.SketchManager.InsertSketch(True)

        Part.ShowNamedView2("Trimetric",
8)

        boolstatus = Part.Extension.SelectByID("Sketch1", "SKETCH",
0, 0, 0, False,
4, Nothing,
0)

        myFeature = Part.FeatureManager.FeatureExtrusion2(True, False, False, 0,
0, 0.01778, 0.00254, False, False, False, False,
0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0, 0, False)

        Part.SelectionManager.EnableContourSelection = False

        boolstatus = Part.Extension.SelectByID("Sketch1", "SKETCH",
0, 0, 0, False,
0, Nothing,
0)

        Part.EditSketch()

        Part.ClearSelection2(True)

        skSegment = Part.SketchManager.Create3PointArc(-0.033816,
0.016783, 0.0#, 0.055107, 0.016783, 0.0#, 0.016009, 0.025458, 0.0#)

        Part.ClearSelection2(True)

        boolstatus = Part.Extension.SelectByID("Line1", "SKETCHSEGMENT",
0.00202904300411839, 0.0119654152286464, -0.00709549576220667, True, 0, Nothing, 0)
```

```
boolstatus = Part.Extension.SelectByID2("Arc1", "SKETCHSEGMENT",  
0.00588878331207997, 0.0171023327681304, -0.0126221740799126, True, 0, Nothing, 0)
```

```
'Replace Line1 with Arc1, delete Line1, and make Arc1 a contour  
boolstatus = Part.SketchManager.SketchReplace2(False, True)
```

```
Part.SketchManager.InsertSketch(True)
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Replace Sketch Relation Example (VBA)

This example shows how to reassign a sketch relation from one sketch line to another.

```

'-----
' Preconditions:
' 1. Verify that the specified part document template
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a new part document and inserts a sketch.
' 2. Gets the number of relations, each relation type and
'    number of entities associated with the relation in the
'    sketch.
' 3. Reassigns horizontal constraints.
' 4. Changes the suppression states of all of the relations
'    in the sketch.
' 5. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSketchMgr As SldWorks.SketchManager
    Dim swSketchSegment As SldWorks.SketchSegment
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim swSkRelMgr As SldWorks.SketchRelationManager
    Dim swSkRel As SldWorks.SketchRelation
    Dim vSkRelArr As Variant
    Dim vSkRel As Variant
    Dim i As Long
    Dim boolstatus As Boolean
    Dim result As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2014\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    Set swSketchMgr = swModel.SketchManager
    swSketchMgr.InsertSketch True
    boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchSegment = swSketchMgr.CreateLine(-0.040395, 0.028613, 0#, 0.031793, 0.028613, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(-0.040395, -0.033476, 0#, 0.031793, -0.033476, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(-0.040395, 0.028613, 0#, -0.040395, -0.033476, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(0.031793, 0.028613, 0#, 0.031793, -0.033476, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(-0.040395, 0.055823, 0#, 0.031793, 0.055823, 0#)
    Set swSketchMgr = swModel.SketchManager

    swModel.ClearSelection2 True

    boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -7.85461847389557E-03, 2.86132530120482E-02, 0, True, 0, Nothing, 1)

    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketch = swFeat.GetSpecificFeature2
    Set swSkRelMgr = swSketch.RelationManager
    Dim newEntity As Object
    Set newEntity = swSelMgr.GetSelectedObject6(2, -1)

    Debug.Print " Feature = " & swFeat.Name

    vSkRelArr = swSkRelMgr.GetRelations(swAll)
    Dim RelationsCount As Long
    RelationsCount = swSkRelMgr.GetRelationsCount(swAll)
    Debug.Print " Number of relations = " & RelationsCount
    For Each vSkRel In vSkRelArr
        Set swSkRel = vSkRel
        Debug.Print " Sketch relation(" & i & ") "
        Debug.Print " Type = " & swSkRel.GetRelationType
        Dim EntitiesCount As Long
        Dim vEntities As Variant
        EntitiesCount = swSkRel.GetEntitiesCount
        Debug.Print " Entities count = " & EntitiesCount

        If (swSkRel.GetRelationType = swConstraintType_e.swConstraintType_HORIZONTAL) Then
            vEntities = swSkRel.GetEntities
            swModel.ClearSelection2 True
            Dim Entity As SldWorks.Entity
            Dim oldEntity As Object
            Dim SketchSeg As SldWorks.SketchSegment
            Dim SketchPt As SldWorks.SketchPoint
            Set Entity = Nothing
            Set SketchSeg = Nothing
            Set SketchPt = Nothing
            On Error Resume Next
            Set oldEntity = vEntities(0)
            result = swSkRel.ReplaceEntity(oldEntity, newEntity)
            Debug.Print " Sketch entity replaced? " & result
        End If
    Next

```

```
        result = swSkRel.Suppressed
        Debug.Print "        Suppressed = " & result
        If (result) Then
            swSkRel.Suppressed = False
        Else
            swSkRel.Suppressed = True
        End If
        Debug.Print "        Suppressed = " & swSkRel.Suppressed
        i = i + 1
    Next
End Sub
```

Replace Sketch Text Example (VBA)

This example shows how to replace sketch text in a part.

```
'-----  
' Preconditions:  
' 1. Open a part that contains sketch text.  
' 2. Select the sketch that contains the sketch text.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Replaces the original sketch text with  
'    new sketch text, "New text".  
' 2. Examine the Immediate window and graphics area.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As Object  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swSketch As SldWorks.Sketch  
Dim swSketchText As SldWorks.SketchText  
Dim params As Variant  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
  
    ' Get the selected feature of the sketch text  
    Set swFeat = swSelMgr.GetSelectedObject6(1, 0)  
    Set swSketch = swFeat.GetSpecificFeature2  
  
    ' Edit the sketch of the sketch text  
    swModel.EditSketch  
  
    ' Get the sketch text  
    params = swSketch.GetSketchTextSegments  
  
    ' Only one instance of sketch text so  
    ' set SketchText to that instance  
    Set swSketchText = params(0)  
  
    ' Print the current sketched text  
    Debug.Print swSketchText.Text  
  
    'Change the sketched text  
    swSketchText.Text = "New text"  
  
    ' Print the changed text  
    Debug.Print swSketchText.Text  
  
    ' Insert the new text in the sketch,  
    ' rebuild the part with any changes  
    ' made to the sketch, and  
    ' exit sketch mode  
    swModel.InsertSketch2 True  
  
End Sub
```

Replace View Model Example (C#)

This example shows how to replace a model in drawing views.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\assem20.slddrw.
// 2. Verify that the specified replacement model exists.
//
// Postconditions: Replaces the specified component in Drawing View1
// with the specified model.
//
// NOTE: Because the model is used elsewhere, do not save changes
// when closing it.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace ReplaceViewModel_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            DrawingDoc swDrawingDoc;
            SelectionMgr swSelectionMgr;
            DrawingComponent swDrawingComponent;
            View swView;
            Component2 swComponent;
            object view;
            object instance;
            object[] views = new object[1];
            object[] instances = new object[1];
            Dispatcher[] viewsIn = new Dispatcher[1];
            Dispatcher[] instancesIn = new Dispatcher[1];
            bool status;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDrawingDoc = (DrawingDoc)swModel;
            status = swDrawingDoc.ActivateView("Drawing View1");

            //Select the view in which to replace the model
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swView = (View)swSelectionMgr.GetSelectedObject6(1, -1);
            view = (object)swView;
            views[0] = view;
            viewsIn[0] = new Dispatcher(views[0]);

            // Select the instance of the model to replace
            status = swModelDocExt.SelectByID2("Assem20-3@Drawing View1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swDrawingComponent = (DrawingComponent)swSelectionMgr.GetSelectedObject6(1, -1);
            swComponent = (Component2)swDrawingComponent.Component;
            instance = (object)swComponent;
            instances[0] = instance;
            instancesIn[0] = new Dispatcher(instances[0]);

            status = swDrawingDoc.ReplaceViewModel("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\bagel.sldprt", (viewsIn), (instancesIn));

        }

        public SldWorks swApp;
    }
}
```

Replace View Model Example (VBA)

This example shows how to replace a model in a drawing view.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\assem20.slddrw.
' 2. Verify that the specified replacement model exists.
'
' Postconditions: Replaces the specified component in Drawing View1
' with the specified model.
'
' NOTE: Because the model is used elsewhere, do not save changes
' when closing it.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swDrawingComponent As SldWorks.DrawingComponent
Dim views(0) As Object
Dim swView As SldWorks.View
Dim instances(0) As Object
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDrawingDoc = swModel

    status = swModel.ActivateView("Drawing View1")

    'Select the view in which to replace the model
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swView = swSelectionMgr.GetSelectedObject6(1, -1)
    Set views(0) = swView

    ' Select the instance of the model to replace
    status = swModelDocExt.SelectByID2("Assem20-3@Drawing View1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swDrawingComponent = swSelectionMgr.GetSelectedObject6(1, -1)
    Set instances(0) = swDrawingComponent.Component
    status = swDrawingDoc.ReplaceViewModel("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\bagel.sldprt", (views), (instances))

End Sub
```

Replace View Model Example (VB.NET)

This example shows how to replace a model in drawing views.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\assem20.slddrw.
' 2. Verify that the specified replacement model exists.
'
' Postconditions: Replaces the specified component in Drawing View1
' with the specified model.
'
' NOTE: Because the model is used elsewhere, do not save changes
' when closing it.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swDrawingDoc As DrawingDoc
    Dim swSelectionMgr As SelectionMgr
    Dim swDrawingComponent As DrawingComponent
    Dim swView As View
    Dim swComponent As Component2
    Dim views(0) As Object
    Dim instances(0) As Object
    Dim viewsIn(0) As DispatcherWrapper
    Dim instancesIn(0) As DispatcherWrapper
    Dim status As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swDrawingDoc = swModel
        status = swModel.ActivateView("Drawing View1")

        'Select the view in which to replace the model
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swView = swSelectionMgr.GetSelectedObject6(1, -1)
        views(0) = swView
        viewsIn(0) = New DispatcherWrapper(views(0))

        ' Select the instance of the model to replace
        status = swModelDocExt.SelectByID("Assem20-3@Drawing View1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swDrawingComponent = swSelectionMgr.GetSelectedObject6(1, -1)
        swComponent = swDrawingComponent.Component
        instances(0) = swComponent
        instancesIn(0) = New DispatcherWrapper(instances(0))
        status = swDrawingDoc.ReplaceViewModel("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\bagel.sldprt", (viewsIn), (instancesIn))

    End Sub

    Public swApp As SldWorks

End Class
```

Reposition Drawing Views to Avoid Overlap Example (VBA)

This example shows how to reposition drawing views to avoid overlap.

```
'-----
' Preconditions:
' 1. Open public_documents\introsw\bolt-assembly.slddrw.
' 2. Drag Drawing View3 so that it overlaps Drawing View2.
'
' Postconditions:
' 1. Repositions Drawing View2 and Drawing View2 so that they do
'    not overlap.
' 2. Examine the drawing and the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim vOutline() As Variant
    Dim vPos() As Variant
    Dim nNumView As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swDraw = swModel

    nNumView = 0

    Set swView = swDraw.GetFirstView
    Do While Not swView Is Nothing
        ReDim Preserve vOutline(nNumView)
        ReDim Preserve vPos(nNumView)
        vOutline(nNumView) = swView.GetOutline
        vPos(nNumView) = swView.Position
        Debug.Print "View = " & swView.GetName2
        Debug.Print "  Pos = (" & vPos(nNumView)(0) * 1000# & ", " & vPos(nNumView)(1) * 1000# & ") mm"
        Debug.Print "  Min = (" & vOutline(nNumView)(0) * 1000# & ", " & vOutline(nNumView)(1) * 1000# & ") mm"
        Debug.Print "  Max = (" & vOutline(nNumView)(2) * 1000# & ", " & vOutline(nNumView)(3) * 1000# & ") mm"
        nNumView = nNumView + 1
        Set swView = swView.GetNextView
    Loop

    ' Sheet
    Set swView = swDraw.GetFirstView

    ' View 1
    Set swView = swView.GetNextView

    'View 2 - vertically aligned to view 1
    Set swView = swView.GetNextView

    vPos(2)(1) = vPos(1)(1) + (vOutline(1)(3) - vPos(1)(1)) + (vPos(2)(1) - vOutline(2)(1))
    swView.Position = vPos(2)

    swModel.GraphicsRedraw2

    'View 3 - horizontally aligned to view 2
    Set swView = swView.GetNextView

    vPos(3)(0) = vPos(1)(0) + (vOutline(1)(1) - vPos(0)(0)) + (vPos(3)(0) - vOutline(3)(0))
    swView.Position = vPos(3)

    swModel.GraphicsRedraw2

End Sub
```

Reset Untitled Document Count Example (C#)

For new untitled documents, SOLIDWORKS Design increments the untitled document count and generates a document name by appending the count to the document type. By default, Part1 is for the first untitled document in a SOLIDWORKS Design session, Part2 for the next untitled document in that same SOLIDWORKS Design session, etc.). The untitled document count is automatically reset between SOLIDWORKS Design sessions.

You can use `ISldWorks::ResetUntitledCount` to reset the untitled document count within a single SOLIDWORKS Design session.

The following code example demonstrates how to use `ISldWorks::ResetUntitledCount` to reset the untitled document count. The code resets the untitled document count at the beginning of the SOLIDWORKS Design session, creates two untitled documents, maximizes one document, and minimizes the second document.

NOTE: Programs designed to run multiple times in the same SOLIDWORKS Design session should either assign titles to new documents or reset the untitled document count at the beginning of the SOLIDWORKS Design session. This ensures that SOLIDWORKS Design creates predictable names for untitled documents, resulting in predictable outcomes for each invocation of the program.

```
//-----
// Preconditions: Verify that the specified document template
// exists.
//
// Postconditions:
// 1. Creates two new untitled part documents, Part9 and Part10;
//    maximizes Part9 and minimizes Part10.
// 2. Minimize Part9 and maximize Part10.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = null;
            ModelDoc2 Part2 = null;
            bool silent = false;
            int longstatus = 0;
            int longwarnings = 0;

            // Reset the counts for untitled documents
            // (parts, assemblies, and drawings) to 8
            swApp.ResetUntitledCount(8, 8, 8);

            // When a new untitled document is created,
            // SOLIDWORKS Design increments the untitled document
            // count to 9
            // The next untitled document gets a name with
            // count = 9 ("Part9")
            Part = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swApp.ActivateDoc2("Part9", silent, ref longwarnings);
            ModelView myModelView = null;
            myModelView = (ModelView)Part.ActiveView;

            // Maximize Part9
            myModelView.FrameState = (int)swWindowState_e.swWindowStateMaximized;

            // When a new untitled document is created,
            // SOLIDWORKS Design increments the untitled document
            // count to 10
            // The next untitled document gets a name with
            // count = 10 ("Part10")
            Part2 = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swApp.ActivateDoc2("Part10", silent, ref longstatus);
            ModelView myModelView2 = null;
            myModelView2 = (ModelView)Part2.ActiveView;

            // Minimize Part10
            myModelView2.FrameState = (int)swWindowState_e.swWindowStateMinimized;

        }
        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Reset Untitled Document Count Example (VBA)

For new untitled documents, SOLIDWORKS Design increments the untitled document count and generates a document name by appending the count to the document type. By default, Part1 is for the first untitled document in a SOLIDWORKS Design session, Part2 for the next untitled document in that same SOLIDWORKS Design session, etc.). The untitled document count is automatically reset between SOLIDWORKS Design sessions.

You can use ISldWorks::ResetUntitledCount to reset the untitled document count within a single SOLIDWORKS Design session.

The following code example demonstrates how to use ISldWorks::ResetUntitledCount to reset the untitled document count. The code resets the untitled document count at the beginning of the SOLIDWORKS Design session, creates two untitled documents, maximizes one document, and minimizes the second document.

NOTE: Programs designed to run multiple times in the same SOLIDWORKS Design session should either assign titles to new documents or reset the untitled document count at the beginning of the SOLIDWORKS Design session. This ensures that SOLIDWORKS Design creates predictable names for untitled documents, resulting in predictable outcomes for each invocation of the program.

```
'-----
' Preconditions: Verify that the specified document template
' exists.
'
' Postconditions:
' 1. Creates two new untitled part documents, Part9 and Part10;
'    maximizes Part9 and minimizes Part10.
' 2. Minimize Part9 and maximize Part10.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim Part2 As SldWorks.ModelDoc2
Dim ModelView As SldWorks.ModelView
Dim ModelView2 As SldWorks.ModelView
Dim boolstatus As Boolean
Dim longstatus As Long
Dim longwarnings As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Reset the counts for untitled
    ' documents (parts, assemblies, and drawings)
    ' to 8
    swApp.ResetUntitledCount 8, 8, 8

    ' When a new untitled document is created,
    ' SOLIDWORKS Design increments the untitled document
    ' count to 9
    ' The next untitled document gets a name with
    ' count = 9 ("Part9")
    Set Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part9", False, longstatus
    Set Part = swApp.ActiveDoc
    Set ModelView = Part.ActiveView

    ' Maximize Part9
    ModelView.FrameState = swWindowState_e.swWindowMaximized

    ' When a new untitled document is created, SOLIDWORKS Design
    ' increments the untitled document count to 10
    ' The next untitled document gets a name with
    ' count = 10 ("Part10")
    Set Part2 = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part10", False, longstatus
    Set Part2 = swApp.ActiveDoc
    Set ModelView2 = Part2.ActiveView

    ' Minimize Part10
    ModelView2.FrameState = swWindowState_e.swWindowMinimized

End Sub
```

Reset Untitled Document Count Example (VB.NET)

For new untitled documents, SOLIDWORKS Design increments the untitled document count and generates a document name by appending the count to the document type. By default, Part1 is for the first untitled document in a SOLIDWORKS Design session, Part2 for the next untitled document in that same SOLIDWORKS Design session, etc.). The untitled document count is automatically reset between SOLIDWORKS Design sessions.

You can use ISldWorks::ResetUntitledCount to reset the untitled document count within a single SOLIDWORKS Design session.

The following code example demonstrates how to use ISldWorks::ResetUntitledCount to reset the untitled document count. The code resets the untitled document count at the beginning of the SOLIDWORKS Design session, creates two untitled documents, maximizes one document, and minimizes the second document.

NOTE: Programs designed to run multiple times in the same SOLIDWORKS Design session should either assign titles to new documents or reset the untitled document count at the beginning of the SOLIDWORKS Design session. This ensures that SOLIDWORKS Design creates predictable names for untitled documents, resulting in predictable outcomes for each invocation of the program.

```
'-----
' Preconditions: Verify that the specified document template
' exists.
'
' Postconditions:
' 1. Creates two new untitled part documents, Part9 and Part10;
'    maximizes Part9 and minimizes Part10.
' 2. Minimize Part9 and maximize Part10.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim Part As ModelDoc2 = Nothing
        Dim Part2 As ModelDoc2 = Nothing
        Dim boolstatus As Boolean = False
        Dim longstatus As Integer = 0
        Dim longwarnings As Integer = 0

        ' Reset the counts for untitled documents
        ' (parts, assemblies, and drawings) to 8
        swApp.ResetUntitledCount(8, 8, 8)

        ' When a new untitled document is created,
        ' SOLIDWORKS Design increments the untitled document
        ' count to 9
        ' The next untitled document gets a name with
        ' count = 9("Part9")
        Part = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swApp.ActivateDoc2("Part9", False, longstatus)
        Dim myModelView As ModelView = Nothing
        myModelView = Part.ActiveView

        ' Maximize Part9
        myModelView.FrameState = swWindowState_e.swWindowMaximized

        ' When a new untitled document is created,
        ' SOLIDWORKS Design increments the untitled document count to 10
        ' The next untitled document gets a name with
        ' count = 10("Part10")
        Part2 = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swApp.ActivateDoc2("Part10", False, longstatus)
        Dim myModelView2 As ModelView = Nothing
        myModelView2 = Part2.ActiveView

        ' Minimize Part10
        myModelView2.FrameState = swWindowState_e.swWindowMinimized

    End Sub
    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Reset Visibility of Sketches in Drawing View Example (C#)

This example shows how to reset the visibility of any hidden sketches in a drawing view so that the drawing view reflects the model.

```
//-----
// Preconditions: Verify that the specified drawing
// document to open exists.
//
// Postconditions:
// 1. Opens the specified drawing document
// 2. Examine the drawing, then press F5.
// 3. Activates a drawing view and hides
//    a sketch in that drawing view .
// 4. After examining the drawing to verify,
//    press F5.
// 5. Selects the drawing view with the hidden sketch
//    and resets the visibility of all sketches in
//    that drawing view so that the drawing view
//    reflects the model.
// 6. Examine the drawing to verify that the hidden
//    sketch is visible.
//
// NOTE: Because this drawing is used elsewhere, do
// not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace ResetSketchVisibilityCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawing = default(DrawingDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            View swView = default(View);
            string fileName = null;
            bool boolstatus = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\resetsketchvisibility.SLDDRW";

            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawing = (DrawingDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Examine the drawing, then press F5
            System.Diagnostics.Debugger.Break();

            // Select a drawing view where to hide a sketch
            boolstatus = swDrawing.ActivateView("Drawing View1");

            // Hide the selected sketch
            boolstatus = swModelDocExt.SelectByID2("Sketch1@resetsketchvisibility-7@Drawing View1", "SKETCH", 0, 0, 0, false, 0,
null, 0);
            swModel.BlankSketch();

            // Examine the drawing to verify that selected sketch is hidden, then press F5
            System.Diagnostics.Debugger.Break();

            // Select the drawing view with the hidden sketch
            boolstatus = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            swView = (View)swSelMgr.GetSelectedObject6(1, -1);

            // Reset the visibility of sketches in the selected
            // drawing view so that drawing view reflects the model
            swView.ResetSketchVisibility();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
```


Reset Visibility of Sketches in Drawing View Example (VBA)

This example shows how to reset the visibility of any hidden sketches in a drawing view so that the drawing view reflects the model.

```
'-----
' Preconditions: Verify that the specified drawing
' to open document exists.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Examine the drawing, then press F5.
' 3. Activates a drawing view and hides
'    a sketch in that drawing view.
' 4. After examining the drawing to verify,
'    press F5.
' 5. Selects the drawing view with the hidden sketch
'    and resets the visibility of all sketches in
'    that drawing view so that the drawing view reflects
'    the model.
' 6. Examine the drawing to verify that the hidden
'    sketch is visible.
'
' NOTE: Because this drawing is used elsewhere, do
' not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swView As SldWorks.View
Dim fileName As String
Dim boolstatus As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\resetsketchvisibility.SLDDRW"

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6(fileName, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawing = swModel
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    Stop ' Examine the drawing, then press F%

    ' Select a drawing view where to hide a sketch
    boolstatus = swDrawing.ActivateView("Drawing View1")

    ' Hide the selected sketch
    boolstatus = swModelDocExt.SelectByID("Sketch1@resetsketchvisibility-7@Drawing View1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    swModel.BlankSketch

    Stop ' Examine the drawing to verify that selected sketch is hidden, then press F5

    ' Select the drawing view with the hidden sketch
    boolstatus = swModelDocExt.SelectByID("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set swView = swSelMgr.GetSelectedObject6(1, -1)

    ' Reset the visibility of sketches in the selected
    ' drawing view so that the drawing view reflects
    ' the model
    swView.ResetSketchVisibility

End Sub
```

Reset Visibility of Sketches in Drawing View Example (VB.NET)

This example shows how to reset the visibility of any hidden sketches in a drawing view so that the drawing view reflects the model.

```

'-----
' Preconditions: Verify that the specified drawing
' to open document exists.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Examine the drawing, then press F5.
' 3. Activates a drawing view and hides
'    a sketch in that drawing view.
' 4. After examining the drawing to verify,
'    press F5.
' 5. Selects the drawing view with the hidden sketch
'    and resets the visibility of all sketches in
'    that drawing view so that the drawing view reflects
'    the model.
' 6. Examine the drawing to verify that the hidden
'    sketch is visible.
'
' NOTE: Because this drawing is used elsewhere, do
' not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swDrawing As DrawingDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swView As View
        Dim fileName As String
        Dim boolstatus As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\resetsketchvisibility.SLDDRW"

        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawing = swModel
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        Stop ' Examine the drawing, then press F5

        ' Select a drawing view where to hide a sketch
        boolstatus = swDrawing.ActivateView("Drawing View1")

        ' Hide the selected sketch
        boolstatus = swModelDocExt.SelectByID2("Sketch1@resetsketchvisibility-7@Drawing View1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.BlankSketch()

        Stop ' Examine the drawing to verify that selected sketch is hidden, then press F5

        ' Select the drawing view with the hidden sketch
        boolstatus = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        swView = swSelMgr.GetSelectedObject6(1, -1)

        ' Reset the visibility of sketches in the selected
        ' drawing view so that the drawing view reflects the model
        swView.ResetSketchVisibility()

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Resolve All Lightweight Components and Fix a Component (C#)

This example shows how to:

- resolve all lightweight components
- fix a component

in the an assembly.

```
//-----
// Preconditions:
// 1. Specified file to open exists.
// 2. Open the Immediate window.
// 3. Run the macro.
//
// Postconditions:
// 1. Resolves all lightweight components
//    in the assembly.
// 2. Fixes the selected component.
// 3. Examine the Immediate window to verify.
//
// NOTE: Because this assembly is used elsewhere,
// do not save any changes when closing it.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ResolveLightWeightFixAssemblyCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            AssemblyDoc swAssy;
            ModelDocExtension swModelDocExt;
            Component2 swComp;
            SelectionMgr swSelMgr;
            string fileName;
            int errors = 0;
            int warnings = 0;
            bool status;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\key_pad_1.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swAssy = (AssemblyDoc)swModel;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Resolve all lightweight components
            errors = swAssy.ResolveAllLightWeightComponents(true);
            Debug.Print("All lightweight components resolved (0 = All components resolved)? " + errors);

            // Fix the selected component
            status = swModelDocExt.SelectByID2("Pad_1-1@key_pad_1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swAssy.FixComponent();
            status = swModelDocExt.SelectByID2("Pad_1-1@key_pad_1", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swComp = swSelMgr.GetSelectedObjectsComponent3(1, -1);
            Debug.Print("Selected component fixed? " + swComp.IsFixed());

            /// <summary>
            /// The SldWorks swApp variable is pre-assigned for you.
            /// </summary>

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Resolve All Lightweight Components and Fix a Component (VBA)

This example shows how to:

- resolve all lightweight components
- fix a component

in the an assembly.

```
'-----
' Preconditions:
' 1. Specified file to open exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Resolves all lightweight components
'    in the assembly.
' 2. Fixes the selected component.
' 3. Examine the Immediate window to verify.
'
' NOTE: Because this assembly is used elsewhere,
' do not save any changes when closing it.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssy As SldWorks.AssemblyDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swComp As SldWorks.Component2
Dim swSelMgr As SldWorks.SelectionMgr
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\key pad_1.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swAssy = swModel
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    ' Resolve all lightweight components
    errors = swAssy.ResolveAllLightWeightComponents(True)
    Debug.Print ("All lightweight components resolved (0 = All components resolved)? " & errors)

    ' Fix the selected component
    status = swModelDocExt.SelectByID2("Pad_1-1@key pad_1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swAssy.FixComponent
    status = swModelDocExt.SelectByID2("Pad_1-1@key pad_1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swComp = swSelMgr.GetSelectedObjectsComponent3(1, -1)
    Debug.Print ("Selected component fixed? " & swComp.IsFixed)

End Sub
```

Resolve All Lightweight Components and Fix a Component (VB.NET)

This example shows how to:

- resolve all lightweight components
- fix a component

in the an assembly.

```
-----
' Preconditions:
' 1. Specified file to open exists.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Resolves all lightweight components
'    in the assembly.
' 2. Fixes the selected component.
' 3. Examine the Immediate window to verify.
'
' NOTE: Because this assembly is used elsewhere,
' do not save any changes when closing it.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swAssy As AssemblyDoc
    Dim swModelDocExt As ModelDocExtension
    Dim swComp As Component2
    Dim swSelMgr As SelectionMgr
    Dim fileName As String
    Dim errors As Integer
    Dim warnings As Integer
    Dim status As Boolean

    Sub Main()

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\key pad_1.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swAssy = swModel
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        ' Resolve all lightweight components
        errors = swAssy.ResolveAllLightWeightComponents(True)
        Debug.Print("All lightweight components resolved (0 = All components resolved)? " & errors)

        ' Fix the selected component
        status = swModelDocExt.SelectByID2("Pad_1-1@key pad_1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swAssy.FixComponent()
        status = swModelDocExt.SelectByID2("Pad_1-1@key pad_1", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swComp = swSelMgr.GetSelectedObjectsComponent3(1, -1)
        Debug.Print("Selected component fixed? " & swComp.IsFixed)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Return Box Extents Example (C++ Dispatch)

This example shows how to return the box extents of assembly components.

NOTE: There is no checking to see if the components are suppressed.

```
void GetComponentExtents()
{
    LPDISPATCH pDispatch;
    IModelDoc ModelDoc;
    IConfiguration Configuration;
    IComponent RootComponent;
    VARIANT varChildren;
    SafeArray* psaChildren;
    VARIANT varBox;
    SafeArray* psaBox;
    IComponent Child;
    long i, nChildren;
    double extent[6];
    double value;
    long j;

    // Get the AssemblyDoc
    pDispatch = GetSOLIDWORKS()->GetActiveDoc();
    ModelDoc.AttachDispatch(pDispatch);
    if ( ModelDoc.GetType() == swDocASSEMBLY )
    {

        // Get the direct children of this assembly
        if ( (pDispatch = ModelDoc.GetActiveConfiguration()) != NULL )
        {
            Configuration.AttachDispatch(pDispatch);

            if ( (pDispatch = Configuration.GetRootComponent()) != NULL )
            {
                RootComponent.AttachDispatch(pDispatch);

                varChildren = RootComponent.GetChildren();
                psaChildren = V_ARRAY(&varChildren);
                SafeArrayGetUBound(psaChildren, 1, &nChildren);
                for ( i=0; i<=nChildren; i++ ) // Note Ubound 0 is 1 child
                {
                    SafeArrayGetElement(psaChildren, &i, &pDispatch);
                    Child.AttachDispatch(pDispatch);

                    // Extract the extents from the components
                    varBox = Child.GetBox(FALSE, FALSE);
                    psaBox = V_ARRAY(&varBox);
                    SafeArrayLock(psaBox);
                    for ( j=0; j<6; j++ )
                    {
                        SafeArrayGetElement(psaBox, &j, (VOID *)&(value));
                        extent[j] = value;
                    }

                    // Destroy the Box SafeArray
                    SafeArrayUnlock(psaBox);
                    SafeArrayDestroy(psaBox);
                    Child.ReleaseDispatch();
                }

                // Destroy the children SafeArray
                SafeArrayDestroy(psaChildren);
            }
        }
    }
}
```

```
return;  
}
```

Create Trimmed Curve Example (C#)

This example shows how to create a trimmed curve on a selected face.

```
//-----
// Preconditions:
// 1. Open a part and select a face.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a trimmed curve.
// 2. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CreateTrimmedCurve_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SketchManager swSketchMgr = default(SketchManager);
            Face2 swFace = default(Face2);
            Surface swSurf = default(Surface);
            double[] vFaceUV = null;
            SurfaceParameterizationData surfParam = default(SurfaceParameterizationData);
            Curve swCurveU = default(Curve);
            double[] vUIsoStartPt = null;
            double[] vUIsoEndPt = null;
            const double nChordTol = 0.001;
            const double nLengthTol = 0.001;
            double nStartParam = 0;
            double nEndParam = 0;
            bool bIsClosed = false;
            bool bIsPeriodic = false;
            double[] vStartPt = null;
            double[] vEndPt = null;
            double[] vTessPts = null;
            SketchSegment swSketchSeg = default(SketchSegment);
            int i = 0;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            swSurf = (Surface)swFace.GetSurface();

            vFaceUV = (double[])swFace.GetUVBounds();
            surfParam = swSurf.Parameterization2();

            vUIsoStartPt = (double[])swSurf.Evaluate((vFaceUV[0] + vFaceUV[1]) / 2.0, vFaceUV[2], 0, 0);
            vUIsoEndPt = (double[])swSurf.Evaluate((vFaceUV[0] + vFaceUV[1]) / 2.0, vFaceUV[3], 0, 0);

            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Face:");
            Debug.Print("   uRange = [" + vFaceUV[0] + " , " + vFaceUV[1] + "]");
            Debug.Print("   vRange = [" + vFaceUV[2] + " , " + vFaceUV[3] + "]");
            Debug.Print(" Surface:");
            Debug.Print("   uRange = [" + surfParam.UMin + " , " + surfParam.UMax + "]");
            Debug.Print("   vRange = [" + surfParam.VMin + " , " + surfParam.VMax + "]");
            Debug.Print(" U Curve:");
            Debug.Print("   Start Pt = (" + vUIsoStartPt[0] * 1000 + " , " + vUIsoStartPt[1] * 1000 + " , " + vUIsoStartPt[2] * 1000
+ " ) mm");
            Debug.Print("   End Pt = (" + vUIsoEndPt[0] * 1000 + " , " + vUIsoEndPt[1] * 1000 + " , " + vUIsoEndPt[2] * 1000 + "
mm");

            // Create untrimmed U curve
            double uvValue;
            uvValue = (vFaceUV[0] + vFaceUV[1]) / 2.0;
            swCurveU = swSurf.MakeIsoCurve2(false, ref uvValue);

            // Trim U curve to start and end of U range
            swCurveU = swCurveU.CreateTrimmedCurve2(vUIsoStartPt[0], vUIsoStartPt[1], vUIsoStartPt[2], vUIsoEndPt[0],
vUIsoEndPt[1], vUIsoEndPt[2]);

            Debug.Print(" Trimmed U curve:");
            bRet = swCurveU.GetEndParams(out nStartParam, out nEndParam, out bIsClosed, out bIsPeriodic);
            Debug.Print(" Start parameter is " + nStartParam);
            Debug.Print(" End parameter is " + nEndParam);
        }
    }
}
```

```

Debug.Print("    Is closed? " + bIsClosed);
Debug.Print("    Is periodic? " + bIsPeriodic);

vStartPt = (double[])swCurveU.Evaluate2(nStartParam, 0);
vEndPt = (double[])swCurveU.Evaluate2(nEndParam, 0);

vTessPts = (double[])swCurveU.GetTessPts(nChordTol, nLengthTol, (vStartPt), (vEndPt));

swSketchMgr = (SketchManager)swModel.SketchManager;

swSketchMgr.Insert3DSketch(false);
swModel.SetAddToDB(true);

for (i = 0; i <= vTessPts.GetUpperBound(0) - 3; i += 3)
{
    swSketchSeg = (SketchSegment)swModel.CreateLine2(vTessPts[i + 0], vTessPts[i + 1], vTessPts[i + 2], vTessPts[i + 3], vTessPts[i + 4], vTessPts[i + 5]);
}

swModel.SetAddToDB(false);
swSketchMgr.Insert3DSketch(true);

bRet = swModel.EditRebuild3();

}

public SldWorks swApp;
}
}

```

Create Trimmed Curve Example (VBA)

This example shows how to create a trimmed curve on a selected face.

```

'-----
' Preconditions:
' 1. Open a part and select a face.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a trimmed curve.
' 2. Inspect the Immediate window.
'-----

```

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace As SldWorks.Face2
    Dim swSurf As SldWorks.Surface
    Dim vFaceUV As Variant
    Dim surfParam As SldWorks.SurfaceParameterizationData
    Dim swCurveU As SldWorks.Curve
    Dim vUIsoStartPt As Variant
    Dim vUIsoEndPt As Variant
    Const nChordTol As Double = 0.001
    Const nLengthTol As Double = 0.001
    Dim nStartParam As Double
    Dim nEndParam As Double
    Dim bIsClosed As Boolean
    Dim bIsPeriodic As Boolean
    Dim vStartPt As Variant
    Dim vEndPt As Variant
    Dim vTessPts As Variant
    Dim swSketchMgr As SldWorks.SketchSegment
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)
    Set swSurf = swFace.GetSurface

    vFaceUV = swFace.GetUVBounds
    Set surfParam = swSurf.Parameterization2

    vUIsoStartPt = swSurf.Evaluate((vFaceUV(0) + vFaceUV(1)) / 2#, vFaceUV(2), 0, 0)
    vUIsoEndPt = swSurf.Evaluate((vFaceUV(0) + vFaceUV(1)) / 2#, vFaceUV(3), 0, 0)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Face:"
    Debug.Print "   uRange = [" & vFaceUV(0) & " , " & vFaceUV(1) & "]"
    Debug.Print "   vRange = [" & vFaceUV(2) & " , " & vFaceUV(3) & "]"
    Debug.Print " Surface:"
    Debug.Print "   uRange = [" & surfParam.UMin & " , " & surfParam.UMax & "]"
    Debug.Print "   vRange = [" & surfParam.VMin & " , " & surfParam.VMax & "]"
    Debug.Print " U Curve:"
    Debug.Print "   Start Pt = (" & vUIsoStartPt(0) * 1000 & " , " & vUIsoStartPt(1) * 1000 & " , " & vUIsoStartPt(2) * 1000 & " )
mm"
    Debug.Print "   End Pt = (" & vUIsoEndPt(0) * 1000 & " , " & vUIsoEndPt(1) * 1000 & " , " & vUIsoEndPt(2) * 1000 & " ) mm"

    ' Create untrimmed U curve
    Set swCurveU = swSurf.MakeIsoCurve2(False, (vFaceUV(0) + vFaceUV(1)) / 2#)

    ' Trim U curve to start and end of U range
    Set swCurveU = swCurveU.CreateTrimmedCurve2(vUIsoStartPt(0), vUIsoStartPt(1), vUIsoStartPt(2), vUIsoEndPt(0), vUIsoEndPt(1), vUIsoEndPt(2))

    Debug.Print " Trimmed U curve:"
    bRet = swCurveU.GetEndParams(nStartParam, nEndParam, bIsClosed, bIsPeriodic)
    Debug.Print "   Start parameter is " & nStartParam
    Debug.Print "   End parameter is " & nEndParam
    Debug.Print "   Is closed? " & bIsClosed
    Debug.Print "   Is periodic? " & bIsPeriodic

    vStartPt = swCurveU.Evaluate2(nStartParam, 0)
    vEndPt = swCurveU.Evaluate2(nEndParam, 0)

    vTessPts = swCurveU.GetTessPts(nChordTol, nLengthTol, (vStartPt), (vEndPt))

    Dim swSketchMgr as SldWorks.SketchManager
    Set swSketchMgr = swModel.SketchManager

    swSketchMgr.Insert3DSketch False
    swModel.SetAddToDB True

```

```
' Disable VB range checking since tessellation points
' cannot be multiples of 6
On Error Resume Next

For i = 0 To UBound(vTessPts) Step 3
    Set swSketchSeg = swModel.CreateLine2(vTessPts(i + 0), vTessPts(i + 1), vTessPts(i + 2), vTessPts(i + 3), vTessPts(i + 4),
vTessPts(i + 5))
Next i

On Error GoTo 0
swModel.SetAddToDB False
swSketchMgr.Insert3DSketch True
bRet = swModel.EditRebuild3

End Sub
```

Create Trimmed Curve Example (VB.NET)

This example shows how to create a trimmed curve on a selected face.

```

'-----
' Preconditions:
' 1. Open a part and select a face.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a trimmed curve.
' 2. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swSketchMgr As SketchManager
        Dim swFace As Face2
        Dim swSurf As Surface
        Dim vFaceUV As Object
        Dim surfParam As SurfaceParameterizationData
        Dim swCurveU As Curve
        Dim vUIsoStartPt As Object
        Dim vUIsoEndPt As Object
        Const nChordTol As Double = 0.001
        Const nLengthTol As Double = 0.001
        Dim nStartParam As Double
        Dim nEndParam As Double
        Dim bIsClosed As Boolean
        Dim bIsPeriodic As Boolean
        Dim vStartPt As Object
        Dim vEndPt As Object
        Dim vTessPts As Object
        Dim swSketchSeg As SketchSegment
        Dim i As Integer
        Dim bRet As Boolean

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swFace = swSelMgr.GetSelectedObject6(1, -1)
        swSurf = swFace.GetSurface

        vFaceUV = swFace.GetUVBounds
        surfParam = swSurf.Parameterization2

        vUIsoStartPt = swSurf.Evaluate((vFaceUV(0) + vFaceUV(1)) / 2.0#, vFaceUV(2), 0, 0)
        vUIsoEndPt = swSurf.Evaluate((vFaceUV(0) + vFaceUV(1)) / 2.0#, vFaceUV(3), 0, 0)

        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" Face:")
        Debug.Print(" uRange = [" & vFaceUV(0) & " , " & vFaceUV(1) & "]")
        Debug.Print(" vRange = [" & vFaceUV(2) & " , " & vFaceUV(3) & "]")
        Debug.Print(" Surface:")
        Debug.Print(" uRange = [" & surfParam.UMin & " , " & surfParam.UMax & "]")
        Debug.Print(" vRange = [" & surfParam.VMin & " , " & surfParam.VMax & "]")
        Debug.Print(" U Curve:")
        Debug.Print(" Start Pt = (" & vUIsoStartPt(0) * 1000 & " , " & vUIsoStartPt(1) * 1000 & " , " & vUIsoStartPt(2) * 1000 &
") mm")
        Debug.Print(" End Pt = (" & vUIsoEndPt(0) * 1000 & " , " & vUIsoEndPt(1) * 1000 & " , " & vUIsoEndPt(2) * 1000 &
") mm")

        ' Create untrimmed U curve
        swCurveU = swSurf.MakeIsoCurve2(False, (vFaceUV(0) + vFaceUV(1)) / 2.0#)

        ' Trim U curve to start and end of U range
        swCurveU = swCurveU.CreateTrimmedCurve2(vUIsoStartPt(0), vUIsoStartPt(1), vUIsoStartPt(2), vUIsoEndPt(0), vUIsoEndPt(1),
vUIsoEndPt(2))

        Debug.Print(" Trimmed U curve:")
        bRet = swCurveU.GetEndParams(nStartParam, nEndParam, bIsClosed, bIsPeriodic)
        Debug.Print(" Start parameter is " & nStartParam)
        Debug.Print(" End parameter is " & nEndParam)
        Debug.Print(" Is closed? " & bIsClosed)
        Debug.Print(" Is periodic? " & bIsPeriodic)

        vStartPt = swCurveU.Evaluate2(nStartParam, 0)
        vEndPt = swCurveU.Evaluate2(nEndParam, 0)

        vTessPts = swCurveU.GetTessPts(nChordTol, nLengthTol, (vStartPt), (vEndPt))
    End Sub

```

```

swSketchMgr = swModel.SketchManager

swSketchMgr.Insert3DSketch(False)
swModel.SetAddToDB(True)

' Disable VB range checking since tessellation points
' cannot be multiples of 6

On Error Resume Next

For i = 0 To (UBound(vTessPts) - 3) Step 3
    swSketchSeg = swModel.CreateLine2(vTessPts(i + 0), vTessPts(i + 1), vTessPts(i + 2), vTessPts(i + 3), vTessPts(i + 4),
vTessPts(i + 5))
Next i

On Error GoTo 0

swModel.SetAddToDB(False)
swSketchMgr.Insert3DSketch(True)

bRet = swModel.EditRebuild3

End Sub

Public swApp As SldWorks

End Class

```

Reverse Order of Decals Example (VBA)

This example shows how to reverse the order of the decals applied on a model.

```
'-----
' Preconditions:
' 1. Open a model that has two decals
'    applied on it.
' 2. Click the DisplayManager tab and click
'    each decal to see its order.
' 3. Click a blank spot in the DisplayManager.
'
' Postconditions:
' 1. Reverses the order of the decals.
' 2. Click each decal to see its new order.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swDecal As SldWorks.Decal
Dim lNbrDecals As Long
Dim boolstatus As Boolean
Dim arrDecals As Variant
Dim i As Long
Dim iDecalID As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    arrDecals = swModelDocExt.GetDecals()

    'Get last decal ID
    lNbrDecals = swModelDocExt.GetDecalsCount

    ' Reverse order of decals
    boolstatus = swModelDocExt.ReverseDecalsOrder(lNbrDecals)

    ' Force a rebuild of the model
    boolstatus = swModel.ForceRebuild3(True)

End Sub
```

Roll Back Model Example (C#)

This example shows how to step through the FeatureManager design tree of a model by rolling back to each feature in reverse sequence. Running an example like this can provide insight into the design intent of the user.

```
//-----
// Preconditions:
// 1. Open a part or assembly document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Observe the FeatureManager design tree
//    while the macro executes.
// 2. Examine the Immediate window to see the
//    names of the features rolled back and forward.
//
// NOTE: The delay between steps is set to 1 second.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;

namespace EditRollbackCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelView swModelView = default(ModelView);
            PartDoc swPart = default(PartDoc);
            AssemblyDoc swAssy = default(AssemblyDoc);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            object[] featFace = null;
            Face2 swFace = default(Face2);
            object rect = null;
            string featureName = null;
            ArrayList featName = new ArrayList();
            string nameString;
            int docType = 0;
            int i = 0;
            int j = 0;
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelView = (ModelView)swModel.ActiveView;
            rect = null;
            swFeatMgr = (FeatureManager)swModel.FeatureManager;

            swFeat = (Feature)swModel.FirstFeature();
            docType = swModel.GetType();
            switch (docType)
            {
                case (int)swDocumentTypes_e.swDocPART:
                    swPart = (PartDoc)swModel;
                    break;
                case (int)swDocumentTypes_e.swDocASSEMBLY:
                    swAssy = (AssemblyDoc)swModel;
                    break;
                default:
                    Debug.Print("Open a part or assembly document, then rerun this macro.");
                    break;
            }

            while ((swFeat != null))
            {
                featureName = swFeat.Name;
                featName.Add(featureName);
                swFeat = (Feature)swFeat.GetNextFeature();
            }

            // Playback each feature in the FeatureManager design tree
            for (i = 0; i <= ((featName.Count) - 1); i++)
            {
                nameString = (string)featName[i];
                Debug.Print(nameString);
                status = swFeatMgr.EditRollback((int)swMoveRollbackBarTo_e.swMoveRollbackBarToAfterFeature, nameString);
                // Do not assert because you may be trying to roll back or roll forward
                // to a feature that cannot be rolled back or forward to; for example,
                // the Lighting or Annotations folder
                //Debug.Assert status

                // Remove any previous highlights
                swModelView.GraphicsRedraw(rect);
                // Highlight feature if it has any geometry
                switch (docType)
                {
                    case (int)swDocumentTypes_e.swDocPART:
                        swPart = (PartDoc)swModel;
                        swFeat = (Feature)swPart.FeatureByName(nameString);
                        break;
                    case (int)swDocumentTypes_e.swDocASSEMBLY:
                        swAssy = (AssemblyDoc)swModel;
                        swFeat = (Feature)swAssy.FeatureByName(nameString);
```

```

        break;
    }

    featFace = (object[])swFeat.GetFaces();
    if (featFace != null)
    {
        for (j = 0; j <= ((featFace.GetUpperBound(0))); j++)
        {
            swFace = (Face2)featFace[j];
            swFace.Highlight(true);
        }
    }

    // Only pause if rollback is successful
    if (status)
    {
        // Pause for 1 second
        System.Threading.Thread.Sleep(1000);
        // Allow SOLIDWORKS Design to refresh screen
        System.Windows.Forms.Application.DoEvents();
    }
}

// Remove highlight from last feature
swModelView.GraphicsRedraw(rect);
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Roll Back Model Example (VBA)

This example shows how to step through the FeatureManager design tree of a model by rolling back to each feature in reverse sequence. Running an example like this can provide insight into the design intent of the user.

```

'-----
' Preconditions:
' 1. Open a part or assembly document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Observe the FeatureManager design tree
'    while the macro executes.
' 2. Examine the Immediate window to see the
'    names of the features rolled back and forward.
'
' NOTE: The delay between steps is set to 1 second.
'-----

```

Option Explicit

Sub Main()

```

' Delay in seconds
Const DELAY As Single = 1#

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelView As SldWorks.ModelView
Dim swPart As SldWorks.PartDoc
Dim swAssy As SldWorks.AssemblyDoc
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim featFace As Variant
Dim swFace As SldWorks.Face2
Dim rect As Variant
Dim featName() As String
Dim timeNow As Single
Dim docType As Long
Dim i As Long
Dim j As Long
Dim status As Boolean

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swModelView = swModel.ActiveView
Set rect = Nothing
Set swFeatMgr = swModel.FeatureManager
Set swFeat = swModel.FirstFeature

docType = swModel.GetType
Select Case docType
    Case swDocPART
        Set swPart = swModel
    Case swDocASSEMBLY
        Set swAssy = swModel
    Case Else
        Debug.Print "Open a part or assembly document, then rerun this macro."
        Exit Sub
End Select

ReDim featName(0)

Do While Not swFeat Is Nothing
    featName(UBound(featName)) = swFeat.Name
    ReDim Preserve featName(UBound(featName) + 1)
    Set swFeat = swFeat.GetNextFeature
Loop

' Loop over-allocates array by 1, so remove last (empty) entry
ReDim Preserve featName(UBound(featName) - 1)

' Playback each feature in the FeatureManager design tree
For i = 0 To UBound(featName)
    Debug.Print featName(i)
    status = swFeatMgr.EditRollback(swMoveRollbackBarToAfterFeature, featName(i))

    ' Do not assert because you may be trying to roll back or roll forward
    ' to a feature that cannot be rolled back or forward to; for example,
    ' the Lighting or Annotations folder

    'Debug.Assert status

    ' Remove any previous highlights
    swModelView.GraphicsRedraw (rect)

    ' Highlight feature if it has any geometry
    Select Case docType
        Case swDocPART
            Set swFeat = swPart.FeatureByName(featName(i))
        Case swDocASSEMBLY
            Set swFeat = swAssy.FeatureByName(featName(i))
    End Select

    featFace = swFeat.GetFaces
    If Not IsEmpty(featFace) Then
        For j = 0 To UBound(featFace)
            Set swFace = featFace(j)

```

```
        swFace.Highlight True
    Next j
End If

' Only pause if rollback is successful
If status Then
    timeNow = Timer
    While timeNow + DELAY > Timer
        ' Allow SOLIDWORKS Design to refresh screen
        DoEvents
    Wend
End If
Next i

' Remove highlight from last feature
swModelView.GraphicsRedraw (rect)

End Sub
```